

MsgHub

Tracer

Mono<Msg>

Pipeline

SkillBox

Memory

# AgentScope-Java

深度剖析

从原理到生产的 LLM Agent 开发指南

陈瑶 著

Flux<Event>

ReAct

---

编者絮语

---

本书旨在为读者呈现 *AgentScope-Java* 的核心架构与实现细节，

但由于作者水平有限，书中疏漏之处在所难免，

恳请广大读者批评指正，不吝赐教。

**如需反馈，请联系：23938340@qq.com**

# AgentScope-Java 深度剖析：从原理到生产的 LLM Agent 开发指南

## 前言

2025 年底，阿里巴巴通义实验室开源了 AgentScope-Java——一个面向企业级场景的多智能体开发框架。与 Python 生态的 LangChain、AutoGen 不同，它选择了 Java 17 + Project Reactor 的全栈响应式架构，在 LLM Agent 框架中独树一帜。

本书基于 AgentScope-Java v1.0.11 源码的逐行分析，从理论基础到生产部署，完整呈现一个 LLM Agent 框架的设计哲学与实现细节。每一章都包含精确的源码引用（文件路径+行号），确保论述有据可查。

## 读者对象

- 使用 AgentScope-Java 构建生产级 Agent 应用的 Java 开发者
- 需要评估 Agent 框架选型的架构师
- 对 LLM Agent 系统设计感兴趣的研究者
- 希望扩展 AgentScope-Java 的贡献者

## 如何阅读本书

- **快速入门**: 阅读第 1-3 章理论基础 → 第 21 章实战教程 → 开始编码
  - **深入理解**: 按章节顺序阅读, 每章包含设计评价和源码引用
  - **生产部署**: 重点阅读第 16-20 章生产工程部分
  - **框架对比**: 直接跳到第 25 章选型指南
-

# 目录

## 第一篇：理论基础

- 第 1 章 Agent 范式演进——从脚本到智能体
- 第 2 章 LLM 作为 Agent 大脑——工作原理与工程约束
- 第 3 章 响应式编程与 Agent 框架——为什么选择 Reactor

## 第二篇：核心架构深度剖析

- 第 4 章 Agent 体系——接口设计哲学与实现机制
- 第 5 章 消息模型——多模态内容的统一抽象
- 第 6 章 模型集成——七家 LLM 的统一适配
- 第 7 章 工具系统——从函数调用到 MCP 生态
- 第 8 章 记忆系统——从短期对话到长期认知
- 第 9 章 PlanNotebook——结构化规划系统
- 第 10 章 SkillBox——动态技能系统
- 第 11 章 优雅关闭——生产环境的守护者

## 第三篇：多 Agent 协作与分布式

- 第 12 章 Pipeline 编排与多 Agent 协作模式
- 第 13 章 A2A 协议——Agent 互联网
- 第 14 章 RAG 检索增强——让 Agent 拥有知识
- 第 15 章 Human-in-the-Loop——人机协作的艺术

## 第四篇：生产工程

- 第 16 章 状态持久化与容错
- 第 17 章 可观测性——全链路追踪与监控
- 第 18 章 Spring Boot 集成与微服务部署
- 第 19 章 性能调优
- 第 20 章 安全设计

## 第五篇：实战与生态

- 第 21 章 从零构建你的第一个 Agent
  - 第 22 章 多 Agent 游戏实战——Werewolf
  - 第 23 章 生产级多 Agent 系统实战——Boba Tea Shop
  - 第 24 章 框架扩展开发指南
  - 第 25 章 框架对比与选型指南
-

# 第一篇：理论基础

## 第 1 章 Agent 范式演进——从脚本到智能体

### 1.1 Agent 的定义与本质

"智能体 (Agent)" 的概念源于哲学与人工智能的交叉领域。在计算机科学中，一个 Agent 需要满足四个核心属性：

- 自主性 (Autonomy)**：无需人类直接干预即可运行，对自身行为和内部状态有控制权
- 反应性 (Reactivity)**：感知环境变化并及时响应
- 主动性 (Proactivity)**：不只是被动响应，还能主动发起目标导向的行为
- 社会性 (Sociality)**：能与其他 Agent (或人类) 通过某种协议交互

传统软件系统具备反应性，但缺乏自主性和主动性——它们严格按照预编程的逻辑执行。而 LLM Agent 的革命性在于：LLM 的推理能力赋予了系统自主决策和主动行为的能力，使得"脚本"进化为"智能体"成为可能。

# 1.2 从规则引擎到 LLM Agent 的技术演进

时间线：Agent 技术演进

| 1960s-1980s | 1990s-2010s    | 2020s             | 2023+      |
|-------------|----------------|-------------------|------------|
| 规则系统        | BDI 架构         | LLM Agent         | 多 Agent 系统 |
|             |                |                   |            |
| 专家系统        | 信念-愿望-意图       | Tool Use          | Supervisor |
| (MYCIN)     | (Jason, JACK)  | (ChatGPT Plugins) | Handoffs   |
| 产生式规则       | 多 Agent 系统     | ReAct             | Routing    |
| (CLIPS)     | (JADE, FIPA)   | (Yao et al.)      | Workflow   |
|             |                | Plan-and-Execute  | A2A 协议     |
| 有限状态机       | 对话系统           | RAG               | (标准化)      |
|             | (ELIZA → Siri) | 记忆增强              |            |

## 第一代：规则系统 (1960s-1980s)

MYCIN 是最早的专家系统之一，包含 600 条规则，用于诊断血液感染。它展示了"知识编码"的威力，也暴露了规则系统的致命缺陷——知识获取瓶颈和缺乏灵活性。当遇到规则未覆盖的情况时，系统完全无能为力。

## 第二代：BDI 架构 (1990s-2010s)

BDI (Belief-Desire-Intention) 架构引入了认知科学的概念。Agent 维护一个信念库（世界模型），根据愿望（目标）形成意图（行动计划），并在执行过程中根据感知更新信念。JADE 和 FIPA 标准试图为多 Agent 系统建立通用通信协议，但过于学术化，未能广泛工业化。

## 第三代：LLM Agent (2022-)

ChatGPT 的 Function Calling 能力改变了一切。LLM 不再只是"聊天机器人", 它具备了理解工具描述并决定何时调用的能力、根据工具返回结果进行推理的能力、多步推理和自我纠正的能力——这就是 ReAct 范式的基础。

#### 第四代：多 Agent 系统 (2023-)

当单个 Agent 的能力达到瓶颈时，多 Agent 协作成为自然延伸。Google 的 A2A (Agent-to-Agent) 协议尝试建立 Agent 间的通信标准，就像 HTTP 之于 Web 服务。

## 1.3 ReAct 范式深度解析

### 1.3.1 认知科学基础

ReAct (Reasoning + Acting) 范式的理论基础来自认知科学中的"双过程理论":

- **System 1 (快思考)**：直觉式、自动化的反应——对应 LLM 的直接回答
- **System 2 (慢思考)**：深思熟虑、逐步推理——对应 ReAct 的推理链

人类解决复杂问题的方式不是一次性给出答案，而是交替进行思考和行动：思考下一步该做什么，执行行动，观察结果，再思考。ReAct 将这一认知模式形式化为 LLM Agent 的执行框架。

### 1.3.2 ReAct 论文核心思想

Yao et al. (2022) 的《ReAct: Synergizing Reasoning and Acting in Language Models》提出：

传统方法对比：

CoT (Chain of Thought):

Question → 思考 1 → 思考 2 → ... → 思考 N → 答案

问题：思考过程可能与事实脱节，产生"幻觉"

Act-only:

Question → 行动 1 → 观察结果 1 → 行动 2 → 观察结果 2 → ... → 答案

问题：缺乏推理规划，行动可能漫无目的

ReAct:

Question → 思考 1 → 行动 1 → 观察结果 1 → 思考 2 → 行动 2 → 观察结果 2 → ... →

答案

优势：推理指导行动，行动反馈推理，形成闭环

### 1.3.3 AgentScope-Java 的 ReAct 实现

AgentScope-Java 的 **ReActAgent** 是 ReAct 范式的完整实现，其核心循环在

**ReActAgent.java:460-631**：

```
// 简化的 ReAct 循环
while (iteration < maxIters) {
    ReasoningResult result = reasoning(messages).block(); // 推理
    if (result.hasToolCalls()) {
        ActingResult actingResult = acting(result.getToolCalls()).block();
        // 行动
        if (actingResult.isSuspended()) return
        handleSuspension(actingResult);
        messages.addAll(actingResult.getMessages());
    } else {
        return result.getMessage(); // 任务完成
    }
    iteration++;
}
```

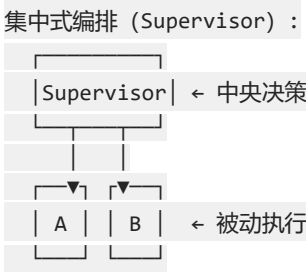
```
}  
return summarizing(messages).block(); // 强制总结
```

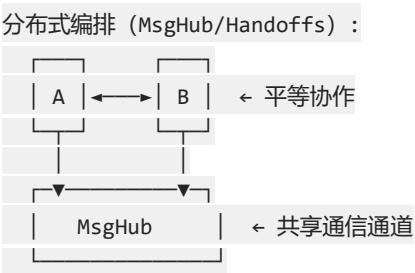
1.3.4 ReAct 的局限性与改进方向

| 局限性    | 表现                | AgentScope-Java 的应对              |
|--------|-------------------|----------------------------------|
| 长链推理退化 | 超过 5-6 轮迭代后推理质量下降 | AutoContextMemory 渐进压缩           |
| 缺乏全局规划 | 逐步推理可能迷失方向        | PlanNotebook 结构化规划               |
| 工具选择失误 | LLM 可能选择错误工具      | ToolGroup 隔离 + SkillBox 按需加载     |
| 无记忆累积  | 每次对话从零开始          | LongTermMemory (Mem0/ReMe)       |
| 不可中断   | 长时间执行无法干预         | Interrupt 机制 + Graceful Shutdown |

1.4 多 Agent 协作的理论模型

1.4.1 集中式 vs 分布式编排





**集中式**的优势：简单可控、职责明确、易于调试。劣势：单点瓶颈、Supervisor能力上限决定系统上限。

**分布式**的优势：可扩展、无单点故障、Agent 自主性高。劣势：协调复杂、一致性难保证、调试困难。

### 1.4.2 A2A 协议的设计理念

A2A (Agent-to-Agent) 协议借鉴了微服务架构的理念：

| 微服务概念             | A2A 对应                 | 说明              |
|-------------------|------------------------|-----------------|
| API Gateway       | AgentCard              | 服务自描述（能力、端点、认证） |
| Service Discovery | Well-Known URI / Nacos | 动态发现 Agent 实例   |
| REST/RPC          | JSON-RPC over HTTP/SSE | 请求-响应 + 流式传输    |
| Message Queue     | RocketMQ Transport     | 异步消息传递          |
| Load Balancer     | Nacos + 多实例注册          | Agent 水平扩展      |

A2A 的核心价值：**将 Agent 视为网络服务，使不同团队、不同语言开发的 Agent 可以互操作。**

## 思考题

1. ReAct 范式相比纯 CoT 和纯 Act-only 的核心优势是什么？在什么场景下 ReAct 反而不如 CoT？
2. 集中式编排 (Supervisor) 和分布式编排 (MsgHub) 各适合什么业务场景？请举出具体例子。
3. A2A 协议借鉴了微服务架构的哪些概念？这种借鉴的合理性和局限性分别是什么？

**\*\*参考： \*\***

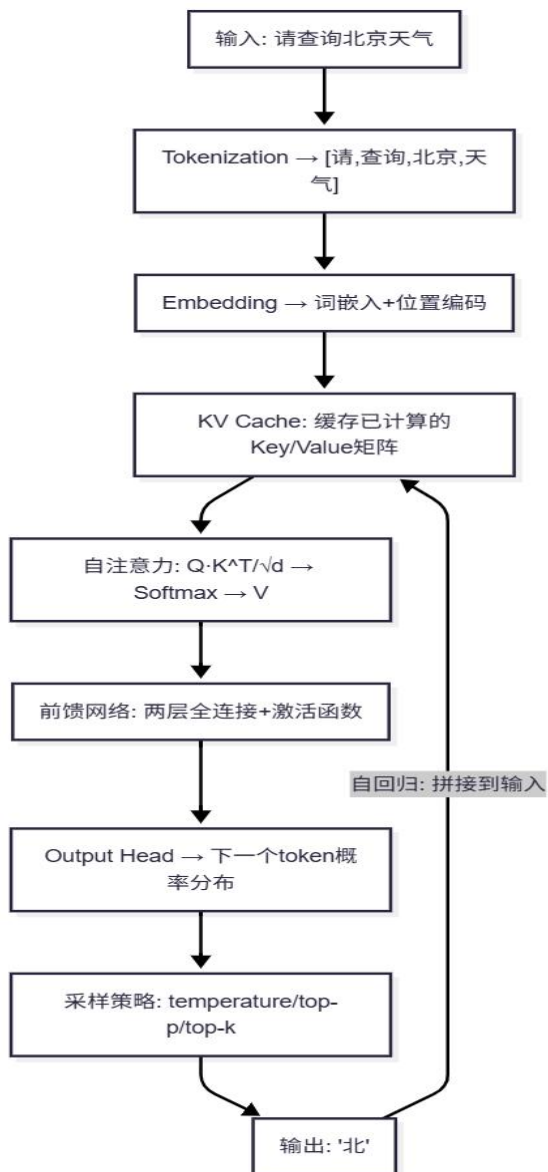
1. ReAct 核心优势是推理与行动的交错——CoT 纯推理无法获取外部信息，Act-only 纯行动缺乏规划。ReAct 在需要实时数据（如查库、调 API）的场景远优于 CoT。但纯数学推理、代码生成等无需外部交互的场景，CoT 更高效——ReAct 的额外工具调用反而引入噪音和延迟。
  2. Supervisor 适合有明确入口的业务（客服工单→路由到技术/财务/物流 Agent）；MsgHub 适合无中心的对等协作（多人会议、多人游戏）。关键区别：Supervisor 有单点路由 Agent，MsgHub 是去中心化广播。
  3. 借鉴了服务注册发现 (NacosA2aRegistry↔Nacos/Eureka)、API 网关 (AgentCard↔OpenAPI Spec)、传输协议 (JSON-RPC↔REST/gRPC)。合理性：Agent 确实是网络服务，需要同样的治理能力。局限：Agent 有状态性（对话上下文），不能像微服务一样无状态水平扩展。
-

# 第 2 章 LLM 作为 Agent 大脑——工作原理与工程约束

LLM 是 Agent 的"大脑"——它负责理解指令、推理决策、生成行动。但这个大脑有固有的工程约束，理解这些约束是设计 Agent 框架的前提。

## 2.1 Transformer 推理机制——理解 Agent 性能的根源

大语言模型基于 Transformer 的 Decoder-Only 架构。核心推理过程：



## KV Cache—流式输出的工程基础

自回归生成必须逐 token 串行——每个新 token 依赖之前所有 token 的计算结果。如果每次生成都重新计算全部历史 token，一个 4000 token 的上下文需要重复计算 4000 次，这是不可接受的。

**KV Cache** 是关键优化：每层 Transformer 缓存已计算的 Key 和 Value 矩阵，新 token 只需计算自己的 Query 并与缓存的 K/V 做注意力运算。这使得：

- 首个 token 生成 (prefill)：需要处理全部输入，延迟高 (~1-3 秒)
- 后续每个 token 生成 (decode)：只需计算 1 个 token 的注意力，延迟低 (~20-50ms)

这个 prefill/decode 的延迟差异直接影响了 Agent 框架的设计——首次调用 LLM 有明显的冷启动延迟，后续流式输出的每个 token 延迟很低。

`Flux<ChatResponse>` 的设计正是映射了这个物理现实：第一个 `ChatResponse` 较慢，后续的 token 快速流出。

## 注意力模式与提示词工程

自注意力的核心公式  $\text{Softmax}(\mathbf{QK}^T/\sqrt{d})\mathbf{V}$  意味着：**每个 token 对所有其他 token 计算注意力权重**。但实证研究 ("Lost in the Middle", Liu et al. 2023) 发现，LLM 对序列**开头和结尾**的信息注意力最高，中间部分注意力衰减。

这对 Agent 框架的消息排列有直接影响：

- 系统提示词（最高优先级）→ 放在**开头**
- 最新的对话消息（最相关）→ 放在**结尾**
- 历史消息（中等优先级）→ 放在**中间**（最容易被"遗忘"的位置）

AgentScope-Java 的 `AutoContextMemory` 正是基于这个发现设计——保留开头（系统提示词）和结尾（最近消息），压缩中间部分。

## 2.2 上下文窗口的限制与突破策略

| 模型         | 上下文窗口 | 实际有效利用 | 原因                             |
|------------|-------|--------|--------------------------------|
| GPT-4o     | 128K  | ~80K   | Lost in the Middle<br>+ 指令遵循衰减 |
| Claude 3.5 | 200K  | ~120K  | 注意力稀释                          |
| Qwen-Max   | 32K   | ~24K   | 窗口较小，衰减更显著                     |

"实际有效利用"远低于标称窗口，原因有三：

1. **Lost in the Middle**：中间位置信息被忽略
2. **指令遵循衰减**：上下文越长，LLM 对系统指令的遵循度越低
3. **成本非线性增长**：注意力计算  $O(n^2)$ ，8000 token 的推理成本是 4000 token 的 4 倍

AgentScope-Java 通过 `AutoContextMemory` 实现 6 级渐进压缩（详见第 8 章），核心思想是**主动管理上下文窗口**——与其被动等待 LLM 忽略中间信息，不如主动压缩为摘要，让重要信息始终在"注意力高峰区"。压缩策略遵循"先损枝叶、后损主干"原则：从工具调用细节开始压缩，逐步到历史对话，最后才压缩当前轮内容。

## 2.3 Function Calling——LLM 如何"使用工具"

Function Calling 并非 LLM "调用"了函数，而是 LLM 输出一个结构化的"调用意图"，由框架负责实际执行。理解这个分离是理解 Agent 工具系统的关键。

### 三种结构化输出方案的演进

| 方案                           | 原理                      | 优势          | 劣势                |
|------------------------------|-------------------------|-------------|-------------------|
| <b>**JSON Mode**</b>         | LLM 在文本中输出 JSON 字符串     | 通用，所有模型支持   | 格式不可靠，可能输出非法 JSON |
| <b>**Function Calling**</b>  | 训练模型输出专用 tool_use 结构    | 格式可靠，原生并行支持 | 需要模型专门训练，非通用      |
| <b>**Structured Output**</b> | 约束解码，保证输出符合 JSON Schema | 100% 格式正确   | 仅少数模型支持，延迟略高      |

AgentScope-Java 的 `ToolValidator` 在执行前验证 LLM 输出的参数是否符合 JSON Schema，这是"信任但验证"的策略——即使模型声称使用了 Function Calling，输出仍可能格式异常。`Toolkit.callTools()` 通过 `Flux.merge()` 并行执行多个工具调用。

### Function Calling 的执行流程

1. 用户消息 + 工具描述 → LLM
2. LLM 输出: { "tool\_call": { "name": "get\_weather", "arguments": {"city": "北京"} } }
3. 框架执行 `get_weather("北京")` → "北京今天晴, 25°C"
4. 工具结果 + 历史消息 → LLM
5. LLM 输出: "北京今天天气晴朗, 气温 25 度。"

关键洞察：步骤 2-3 是 LLM 和框架的**协作**——LLM 负责“决定做什么”（推理），框架负责“实际去做”（执行）。这种分离是 Agent 可靠性的基础——框架可以拦截不安全的工具调用、重试失败的操作、注入人工审批节点。

## 2.4 流式响应——不仅是体验优化，更是架构必需

LLM 推理是逐 token 生成的，单次推理可能耗时 10-60 秒。如果不使用流式输出，用户在整个推理期间看到的是空白等待——这在 Agent 场景下尤其严重，因为一个 ReAct 循环可能包含多轮 LLM 调用。

流式在 Agent 框架中有四个不可替代的架构作用：

1. **超时规避**：长推理通过流式心跳保持连接存活，避免 HTTP 网关超时
2. **中途干预**：流式输出过程中可以检测到 Agent 产生有害内容，及时中断（HITL 的基础。HITL 是 Human-in-the-loop（人工审批/人机协同）的缩写。这是一种“AI 处理 + 人类把关 + 数据反哺”的交互范式。它不是要完全用 AI 替代人类，而是将人类的判断力、创造力和道德标准引入 AI 的工作流中，在关键环节进行干预，既保证结果的准确性与安全性，又利用人类的反馈数据来持续优化 AI 模型。结合当前的 AI 发展趋势（特别是 2026 年的视角），HITL 已经从早期的“数据标注”演变成了“智能体（Agent）控制”的核心机制）

3. **工具可见性**：流式输出 ToolUseBlock 时，前端可以实时展示 Agent 正在调用哪个工具

4. **背压控制**：消费者可以按自己的速率消费 token 流，而非被生产者压垮

AgentScope-Java 选择 Project Reactor 作为全栈响应式基础，

`Flux<ChatResponse>` 从模型层一路透传到 SSE 输出层，中间无需任何格式转换或缓冲。这种"端到端流式"是框架最核心的架构决策（详见第 3 章）。

---

## 思考题

1. "Lost in the Middle"现象对 Agent 的消息历史管理有什么启示？如何设计消息排列策略来缓解？
2. 并行工具调用和串行工具调用各有什么优劣？什么情况下应该强制串行？
3. 为什么 Agent 框架必须是流式的？如果使用同步框架，最关键的功能缺失是什么？

*\*\*参考：\*\**

1. *Lost in the Middle* 说明 LLM 对中间位置信息注意力衰减。启示：重要指令放首尾，历史消息按相关性重排而非时间序。AutoContextMemory 的 lastKeep 参数正是此思路——保留最近 N 条和首条系统提示，压缩中间部分。

2. *并行优势*：多工具同时执行减少总延迟（如同时查天气+查日历）；*劣势*：结果间可能有依赖，token 消耗翻倍。强制串行场景：工具 B 的输入依赖工具 A 的输出（如先查订单号再查物流），或工具间有副作用冲突（如同时修改同一资源）。

3. *流式是 Agent 框架的刚需*：①用户需要实时看到推理过程

(StreamingSSE) , 否则长请求体验极差; ②HITL 需要在推理中间插入人工决策; ③工具执行期间需要显示进度。同步框架最关键缺失是无法在 LLM 流式输出过程中拦截和修改——ReAct 的"观察"阶段就无法实现。

---

# 第 3 章 响应式编程与 Agent 框架——为什么选择 Reactor

## 3.1 从拉到推——为什么 Agent 框架必须是响应式的

传统同步编程采用"拉"模型：调用方发起请求，阻塞当前线程等待响应。这在 I/O 密集场景下有一个根本性问题——**线程是昂贵资源**。一个 JVM 线程默认栈大小 1MB，2000 个并发请求就需要 2GB 栈内存，而线程大部分时间在等待 I/O（等待 LLM 返回 token、等待数据库查询、等待 HTTP 响应），CPU 利用率极低。

LLM Agent 的三个核心特征使同步模型尤其不适合：

1. **流式输出**：LLM 逐 token 生成（~50 token/s），一个完整响应可能持续 10-30 秒。同步模型下，每个请求占一个线程 30 秒，200 并发就需要 200 个线程空等
2. **I/O 密集交织**：一个 ReAct 循环内包含 LLM 调用→工具执行→LLM 调用→工具执行的多轮 I/O 交互，同步模型下每次 I/O 切换都浪费线程时间
3. **事件驱动**：HITL（人工审批）需要 Agent 挂起等待外部事件，可能持续数小时甚至数天，同步模型不可能为此阻塞线程

响应式编程的"推"模型解决了这些问题——数据就绪后主动推送给消费者，等待期间线程不阻塞，可以服务其他请求。**背压 (Backpressure)** 机制确保生产者不会压垮消费者——当客户端消费速度跟不上 LLM 输出速度时，框架可以丢弃中间的 **ReasoningChunk** 或缓冲等待，而非让内存无限增长。

## 3.2 Project Reactor 核心概念

### Mono 与 Flux—异步容器的两个维度

**Mono<T>** 表示 0 或 1 个元素的异步结果，语义等价于 **CompletableFuture<T>**

但支持背压和操作符链。Agent 框架中几乎所有的单次操作都是 **Mono**：

```
// AgentBase.call() 的签名——一个 Agent 调用返回一个结果
public final Mono<Msg> call(Msg msg)

// Session.save() — 持久化操作返回完成信号
public Mono<Void> save(Session session, String sessionId)
```

**Flux<T>** 表示 0 到 N 个元素的异步流，天然适配流式场景：

```
// AgentBase.stream() 的签名——流式返回多个事件
public final Flux<Event> stream(Msg msg)

// LLM 模型的流式响应
public Flux<ChatResponse> streamChat(ChatRequest request)
```

### 操作符链—声明式编程风格

Reactor 的核心设计是**声明式**——描述"要什么"而非"怎么做"。操作符链将数据转换、过滤、组合表达为流水线：

```
agent.call(msg)
    .doOnNext(result -> tracer.trace("agent_complete", result))
    .onErrorResume(e -> fallbackAgent.call(msg))
    .subscribeOn(Schedulers.boundedElastic());
```

每个操作符返回新的 **Mono/Flux**，不修改原始对象（不可变性），这确保了操作符链的线程安全。

## 调度器—线程池的响应式抽象

| 调度器              | 线程池                  | 适用场景     | Agent 框架用途                    |
|------------------|----------------------|----------|-------------------------------|
| `boundedElastic` | 弹性有界（默认<br>10×CPU 核） | I/O 阻塞操作 | Agent.doCall()、<br>Session 读写 |
| `parallel`       | 固定（CPU 核数）           | CPU 密集计算 | 消息压缩、<br>Embedding 计算         |
| `immediate`      | 当前线程                 | 不切换线程    | Hook 执行、上下文<br>传播             |
| `single`         | 单线程                  | 串行化需求    | 状态机更新、CAS 锁<br>检查             |

关键设计决策：AgentBase 的 `doCall()` 默认在 `boundedElastic` 上执行，因为 Agent 调用链中穿插了 LLM API 调用（网络 I/O）和工具执行（可能是文件 I/O 或数据库查询），这些都是阻塞操作。

### 为什么 AgentBase.doCall() 默认使用 boundedElastic?

这是整个设计中最关键的决策。

1、痛点：LLM 的 API 调用（HTTP/HTTPS）是典型的高延迟网络 I/O，而 Agent 调用的工具（如查询数据库、读写文件）往往是阻塞式 I/O。

2、风险：如果在 `parallel`（通常用于 CPU 密集型）或 `single` 线程上执行这些阻塞操作，会直接“饿死”整个事件循环，导致系统吞吐量瞬间归零。

3、解决方案：boundedElastic 是一个弹性线程池。它允许在需要时创建新线程来处理阻塞任务，并在任务完成后回收。将其作为 Agent 调用的默认“跑道”，相当于给每个 Agent 任务分配了一个独立的“工位”，互不干扰。

在 2026 年，随着 DeepAgents（深层智能体）和长运行任务的普及，除了基础的线程调度，还需要关注以下几点：

### 1、超时与熔断

Agent 的调用链可能很长（ReAct 循环）。在 boundedElastic 中，必须为每个 doCall() 配置严格的超时策略（例如使用 timeout 操作符），防止某个卡死的工具调用耗尽整个弹性线程池。

### 2、HITL 的异步挂起

当 Agent 遇到需要 HITL 的步骤时，当前线程不应死等。应利用响应式流的特性，将当前状态“挂起”并持久化，释放 boundedElastic 线程资源，等待人类用户回调后再重新调度。

## 其他调度器的分工

### 1、parallel (CPU 密集型)

用途：消息压缩、Embedding 计算、复杂的 JSON 解析。

理由：这些操作不涉及等待外部响应，纯粹消耗 CPU 算力。限制线程数为 CPU 核数可以最大化利用硬件资源，避免过多的线程上下文切换。

### 2、immediate (当前线程)

用途：Hook 执行、上下文传播。

理由：这些操作极快（微秒级），且必须保证在调用者的上下文中执行（例如获取当前的 Trace ID 或 User Context），切换线程反而增加开销且可能丢失上下文。

### 3、single (单线程)

用途：状态机更新、CAS 锁检查。

理由：保证串行化。Agent 往往维护着复杂的对话状态（Memory/State），使用单线程调度器可以避免多线程并发修改状态导致的数据竞争（Race Condition），充当一种轻量级的“逻辑锁”。

## 架构全景图：Agent 的生命周期与线程流转

结合上述，一个典型的 Agent 请求处理流程如下：

- 1、请求接入：用户请求进入网关，分配一个轻量级事件循环线程（IO Worker）。
- 2、上下文初始化 (immediate)：快速提取 Token、Trace ID，不切换线程。
- 3、Agent 核心执行 (publishOn(boundedElastic))。

思考：调用 LLM（网络 I/O，异步等待）。

行动：LLM 返回需要调用工具（如 search\_db）。

执行工具：在 boundedElastic 的线程中同步执行数据库查询（阻塞 I/O）。

- 4、结果处理 (parallel)

拿到工具返回的大量数据后，切换到 parallel 线程池进行数据清洗或 Embedding 向量化。

## 5、状态更新 (single)

将新的对话历史写入 Redis 或本地状态机，确保串行写入，防止覆盖。

## 6、响应返回：流式返回给用户

## 上下文传播—替代 ThreadLocal

传统 Java 用 `ThreadLocal` 存储请求级别的上下文 (traceId、userId 等)，但响应式模型下线程会切换，`ThreadLocal` 丢失。Reactor 提供不可变的 `Context` (底层是 `Map`)，通过操作符链自动传播：

```
Mono.deferContextual(ctx -> {  
    String traceId = ctx.get("traceId"); // 任何操作符都能获取  
    return agent.call(msg).doOnNext(r -> log.info("[{}] result={}", traceId,  
r));  
});
```

AgentScope-Java 的 `TracerRegistry` 通过 Reactor 全局 Hook

(`Hooks.onEachOperator`) 实现跨操作符的追踪上下文传播——每个操作符执行时自动注入 `traceId`，无需手动传递。

## 3.3 Mono.using——Agent 生命周期管理的基石

Agent 框架中一个核心模式是**资源获取-使用-释放**：获取 Agent 锁→执行推理→释放锁。Reactor 的 `Mono.using()` 专为这种模式设计，类似 try-with-resources：

```
// AgentBase.call() 的核心逻辑 (简化)
public final Mono<Msg> call(Msg msg) {
    return Mono.using(
        () -> {
            acquireLock();          // 获取资源 (CAS 锁)
            return new AgentContext(this, msg);
        },
        ctx -> doCall(ctx),         // 使用资源执行业务逻辑
        ctx -> releaseLock()        // 无论成功/失败都释放资源
    );
}
```

`Mono.using()` 的关键保证：**即使 `doCall()` 抛出异常或被取消，`releaseLock()` 也一定会执行。**这比手动写 `doFinally()` 更安全，因为 `doFinally()` 在某些取消场景下可能不被调用。

如果不使用 `Mono.using()` 而用同步代码：

```
// 同步版本——看似简单，但缺少取消安全性和背压支持
public Msg call(Msg msg) {
    acquireLock();
    try {
        return doCall(msg);
    } finally {
        releaseLock();
    }
}

// 问题：如果调用方在等待时取消，doCall()内部的 LLM 流式调用无法被中断，
// 导致资源泄漏。Mono.using()配合 Flux 的取消信号可以优雅处理。
```

## 3.4 Agent 场景的背压实战

背压不是理论概念——在 Agent 框架中它直接影响用户体验和系统稳定性。

### 场景 1：SSE 客户端消费慢

```
// Agent 流式输出 → SSE 推送给浏览器
agent.stream(msg)
    .onBackpressureBuffer(100)    // 缓冲最多 100 个事件
```

```
.onOverflowDrop() // 超出后丢弃最旧的事件
.subscribe(event -> sseEmitter.send(event));
```

如果浏览器网络慢，消费速率低于 LLM 生成速率（50 token/s），无背压处理会导致服务端内存溢出。`onBackpressureBuffer(100)` + `onOverflowDrop()` 的组合确保：大部分用户体验流畅（100 个事件的缓冲足够），极端情况下丢弃中间的 `ReasoningChunk`（用户看到思考过程有跳帧，但不影响最终结果）。

## 场景 2：MsgHub 广播风暴

MsgHub 将一个 Agent 的输出广播给 N 个参与者。如果某个参与者消费慢，不能阻塞其他参与者：

```
// MsgHub 广播的简化逻辑
sourceAgent.stream(msg)
    .flatMap(event ->
        Flux.fromIterable(participants)
            .flatMap(p -> p.onEvent(event)) // 并行广播
            .onBackpressureDrop() // 慢消费者丢弃中间事件
    );
```

1、`flatMap` 的核心作用是将一个流中的每个元素转换成一个流，然后将这些流合并成一个单一的流。它的关键特性就是可以并发地订阅和处理这些内部流。

- `Flux.fromIterable(participants).flatMap(p -> p.onEvent(event))`
- `Flux.fromIterable(participants)` 创建了一个包含所有参与者的流
- `flatMap` 会为流中的每一个参与者 `p` 调用 `p.onEvent(event)`

由于 `flatMap` 默认是并发的（并发度为 256），它会同时（或在一个批次内）向 `participants` 列表中的多个智能体推送事件，而不是等待前一个参与者处理完再处理下一个。

2、`onBackpressureDrop()`: 这是一个非常重要的背压（Backpressure）处理策略。

- 场景：如果某个参与者 p 的处理速度很慢（即它是一个“慢消费者”），它的内部队列可能会满
- 作用：onBackpressureDrop() 会直接丢弃那些无法被慢速参与者及时处理的事件，从而防止整个广播流程被一个慢节点拖垮，保证了系统的整体响应性

### 如何实现真正的串行广播？

```
Flux.fromIterable(participants).concatMap(p -> p.onEvent(event)) // 串行执行，保证顺序
```

concatMap 会严格按照顺序订阅内部流，只有当前一个参与者的 onEvent 流程完成后，才会开始下一个。

### 场景 3：RAG 批量检索

```
knowledge.query(query, topK=20)
    .onBackpressureBuffer(50) // 检索结果缓冲
    .take(10) // 只取前 10 个（忽略多余的）
    .collectList()
    .flatMap(results -> llm.chat(prompt + results));
```

这是一个非常经典的 RAG（检索增强生成）“检索-聚合”模式，通过响应式流的三个关键操作符，实现了一个高效且鲁棒的数据处理管道。

#### 1、knowledge.query(query, topK=20)：宽泛检索

- 目的：从向量数据库中召回比最终所需数量更多的候选结果。
- 为什么是 20 而不是 10？这是一种“宽进严出”的策略。先召回 20 个可能相关的文档片段，为后续的筛选提供充足的“原料”。这 20 个结果通常是基于向量相似度粗筛出来的，可能包含一些相关性不高的“噪音”。

#### 2、.onBackpressureBuffer(50)：应对背压

- 场景：这是一个非常关键的工程化设计。knowledge.query 可能是一个快速的内存或数据库操作，而下游的 LLM 调用则非常慢。这会在两者之间形成“背压”（Backpressure），即生产者速度快于消费者。
- 作用：onBackpressureBuffer(50) 创建了一个容量为 50 的缓冲区。当 LLM 还在处理上一个请求时，新的检索结果可以被暂时存放在这里，而不是直接报错或导致上游阻塞。这就像一个蓄水池，平滑了生产与消费之间的速度差异，提升了系统的吞吐量和稳定性。

### 3、.take(10)：精准筛选

- 目的：从 20 个候选结果中，只选取最相关的前 10 个。
  - 关键点：这里的 .take(10) 通常不是简单地取列表中的前 10 个。在实际的 RAG 架构中，这个操作符之前往往会隐式地包含一个 重排序（Re-ranking）步骤。
- 流程：query(topK=20) → Re-ranker 模型精排 → .take(10) → LLM。
- 解释：先用一个轻量级模型对 20 个结果进行更精确的相关性打分，然后 .take(10) 取出分数最高的 10 个。这确保了喂给 LLM 的上下文是质量最高、最相关的，避免了“垃圾进，垃圾出”的问题。

## 3.5 为什么是 Reactor 而不是其他方案

### Reactor vs. CompletableFuture

CompletableFuture 是 Java 8 引入的异步原语，为什么不直接用它？

| 维度  | CompletableFuture               | Reactor Mono/Flux    |
|-----|---------------------------------|----------------------|
| 流式  | 不支持（单值）                         | Flux 原生支持            |
| 背压  | 无                               | 内置                   |
| 操作符 | 极少<br>(thenApply/exceptionally) | 100+                 |
| 取消  | 不支持真正取消                         | Disposable.dispose() |
| 重试  | 手动实现                            | retry()/retryWhen()  |
| 上下文 | 依赖 ThreadLocal                  | Context 自动传播         |

关键差距：**CompletableFuture 不支持流式**——LLM 的 token 逐个返回，无法用 `CompletableFuture<String>` 表示一个逐渐增长的字符串。

`Flux<ChatResponse>` 天然表示这个流。

## Reactor vs. Java 21 Virtual Threads

Virtual Threads（虚拟线程，JEP 444）是 Java 21 的重量级特性，它让同步代码获得类似异步的并发能力——每个虚拟线程只占用几 KB 而非 1MB。那为什么不用 Virtual Threads？

```
// Virtual Thread 版本的 Agent—看起来更简单
ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();

Msg result = executor.submit(() -> {
    acquireLock();
    try {
        return doCall(msg); // 同步调用，虚拟线程自动让出
    } finally {
        releaseLock();
    }
}).get();
```

Virtual Threads 的优势是**代码简单**——写同步风格的代码，获得异步级别的并发。但 AgentScope-Java 选择 Reactor 有三个不可替代的原因：

1. **流式不可替代**：Virtual Threads 让阻塞代码不占平台线程，但 `doCall()` 返回的是 `Mono<Msg>` 而非 `Msg`——调用方需要流式消费中间状态 (ReasoningChunk)，Virtual Threads 无法表达"逐步产生结果"
2. **取消语义**：Virtual Thread 的取消依赖 `Thread.interrupt()`，但很多 I/O 操作不响应中断。Reactor 的 `Disposable.dispose()` 能精确取消操作符链中的每一步
3. **背压不可替代**：Virtual Threads 没有背压概念——如果 LLM 输出 50 token/s 而消费者只能处理 10 token/s，Virtual Thread 只能缓冲或 OOM

**但 Virtual Threads 与 Reactor 不冲突**——在 Agent 框架中，Virtual

Threads 最适合工具执行层（阻塞 I/O 操作），Reactor 负责编排层。

AgentScope-Java 的 `boundedElastic` 调度器实际上已经与 Virtual Threads 有类似的语义（弹性线程池处理阻塞 I/O）。

## Reactor vs. Kotlin Coroutines

Kotlin Coroutines 提供了类似 `async/await` 的语法糖，底层可以编译为

Reactor 操作符。选择 Reactor 的原因：

1. **纯 Java 生态**：AgentScope-Java 是 Java 框架，不应依赖 Kotlin 编译器
2. **调试可见性**：Reactor 的 `Hooks.onEachOperator()` 可以在每个操作符前后插入逻辑（用于 Tracer），Coroutines 的 `suspend` 函数对框架层不透明

3. **冷流语义**：Reactor 的 Mono/Flux 是冷流（创建时不执行，subscribe 时才执行），这保证了 Agent 的延迟初始化——只有真正需要时才创建 Agent 实例

## 3.6 响应式的代价

响应式不是银弹，它的代价必须正视：

1. **调试困难**：异常堆栈不反映调用链（操作符链在编译后是嵌套的 Lambda），Reactor 提供了 `Hooks.onOperatorDebug()` 生成增强堆栈，但生产环境不建议开启（性能开销 ~30%）
2. **学习曲线陡峭**：Mono/Flux 的冷/热语义、调度器切换时机、Context 传播规则需要大量实践才能掌握。团队中必须有人深入理解 Reactor 才能排查生产问题
3. **阻塞调用风险**：在 Reactor 链中意外调用阻塞操作（如 `Thread.sleep()` 或同步 HTTP 请求）会占住事件循环线程，导致整个应用卡顿。必须在 `boundedElastic` 调度器上执行阻塞操作
4. **全局 Hook 污染**：`Hooks.onEachOperator()` 是 JVM 级别的全局设置，同一 JVM 中的多个 Reactor 应用会互相影响

**设计取舍**：AgentScope-Java 选择了更高的性能上限和更自然的流式支持，代价是更陡的学习曲线。对于面向生产部署的企业级 Agent 系统（高并发、流式交互、多 Agent 协作），这是合理的取舍。对于快速原型或低并发内部工具，LangChain4j 的同步 API 可能更合适。

---

## 思考题

1. 响应式编程的背压机制在 Agent 场景中如何体现？什么情况下需要主动调节背压？
2. Reactor 的全局 Hook (`Hooks.onEachOperator`) 会影响 JVM 中所有 Reactor 操作符。这种设计有什么风险？
3. 对比同步框架和响应式框架在 LLM Agent 场景的性能差异，什么并发量级下差异开始显著？

**\*\*参考：\*\***

1. Agent 场景的背压体现：LLM 流式输出速率 (~50 token/s) 远快于客户端消费速率时，缓冲区堆积。需要主动调节的场景：SSE 客户端网络慢、MsgHub 广播给多个慢消费者、RAG 检索结果大量涌入时。策略：  
`onBackpressureDrop()` 丢弃中间 `ReasoningChunk`、  
`onBackpressureBuffer(cap)` 限制缓冲。

2. 风险：①多 Agent 应用共存同一 JVM 时，一个应用的 Hook 可能干扰另一个；②调试困难——全局 Hook 的副作用隐式传播；③性能影响——每个操作符都经过 Hook 链。改进：使用 `Context.put()/get()` 传递局部状态替代全局 Hook，或为每个 Agent 创建独立的 Hooks 实例。

3. 差异在 I/O 密集场景最显著：LLM API 调用是典型 I/O 密集（网络等待占 90%+ 时间），同步框架每个请求占一个线程，响应式框架用事件循环。经验值：并发 < 10 差异可忽略 (< 5%)，50+ 并发响应式开始领先 (30-50%)，200+ 并发差异可达 5-10 倍（同步受线程池限制，响应式几乎线性扩展）。

---

## 第二篇：核心架构深度剖析

---

# 第 4 章 Agent 体系——接口设计哲学与实现机制

## 4.1 接口分层设计

AgentScope-Java 采用了 **接口组合 + 抽象基类** 的分层设计，这是框架中最精巧的设计之一：

Agent (顶层接口)

extends CallableAgent, StreamableAgent, ObservableAgent

AgentBase (抽象基类)

implements StateModule, Agent

StructuredOutputCapableAgent (抽象类)

extends AgentBase

ReActAgent (核心具体类)

extends StructuredOutputCapableAgent

UserAgent (人机交互代理)

extends AgentBase

**设计意图分析：**

- `CallableAgent`：定义 `call()` 方法族，返回 `Mono<Msg>`（响应式），是 Agent 最核心的"调用"能力
- `StreamableAgent`：定义 `stream()` 方法族，返回 `Flux<Event>`（响应式流），支持实时事件推送
- `ObservableAgent`：定义 `observe()` 方法，使 Agent 可以被动接收消息而不必回复——这是多 Agent 协作的关键

这种接口组合设计遵循了 **Interface Segregation Principle (ISP)**，不同角色只需关注所需的能力接口。例如，一个纯观察者 Agent 只需实现 `ObservableAgent`，而不必承担 `call()` 的契约。

**设计评价：** 这种设计优于单一庞大接口，但存在一个权衡——Agent 接口将三种能力强制组合，所有 Agent 都必须同时实现三种能力，降低了灵活性。如果某些 Agent 只需要流式能力但不需要观察，就无法做到。不过考虑到框架的定位（多 Agent 协作），这个权衡是合理的。

## 4.2 AgentBase 生命周期：框架的"脊梁"

`AgentBase` (`agentscope-core/.../agent/AgentBase.java`，约 800 行) 是整个框架最重要的基类，提供了：

### 4.2.1 执行生命周期保障

```
public final Mono<Msg> call(List<Msg> msgs) {
    return Mono.using(
        this::acquireExecution,          // 1. 获取执行锁 (CAS)
        resource -> TracerRegistry.get()
```

```

        .callAgent(this, msgs, () ->
            notifyPreCall(msgs)          // 2. 前置 Hook
            .flatMap(this::doCall)// 3. 子类业务逻辑
            .flatMap(this::notifyPostCall) // 4. 后置 Hook + 广播
            .onErrorResume(createErrorHandler(msgs))), // 5. 错误/中
        断处理
        this::releaseExecution,          // 6. 释放锁（保证执行）
        true);
    }

```

`Mono.using` 模式保证了无论成功、失败还是取消，`releaseExecution` 都会被调用。这是 Reactor 中资源管理的标准范式，类似于 try-with-resources。

**并发控制：**使用 `AtomicBoolean running` 进行 CAS 锁定，确保**单个 Agent 实例不会被并发执行**。这是一个有意的设计约束——同一 Agent 实例串行执行，避免状态竞争。

**设计评价：**`call()` 声明为 `final` 是一个关键决策——子类无法覆盖调用流程，只能通过 `doCall()` 注入业务逻辑。这确保了生命周期管理的完整性（Hook、Tracer、广播等始终执行），但也牺牲了子类的灵活性。这是一个好的权衡，因为 Agent 执行的生命周期管理是横切关注点，不应由业务代码控制。

## 4.2.2 Hook 系统

Hook 按 priority 排序执行（值越小优先级越高），分为：

- `PreCallEvent` / `PostCallEvent`：Agent 调用前后
- `PreReasoningEvent` / `PostReasoningEvent` / `ReasoningChunkEvent`：推理步骤前后
- `PreActingEvent` / `PostActingEvent`：工具执行前后

**设计亮点：** Hook 事件是 sealed class，编译器强制穷举所有事件类型，避免遗漏。Hook 事件中部分是可修改的（如 `setInputMessages()`），一部分是只读的（如 `ReasoningChunkEvent`），这种区分使得框架可以在不破坏封装的前提下允许扩展。

### 4.2.3 中断机制

使用 `AtomicBoolean interruptFlag + AtomicReference<Msg> userInterruptMessage` 实现协作式中断。子类在 Mono 链的检查点调用 `checkInterruptedAsync()` 检测中断信号。

**设计评价：** 协作式中断比抢占式中断更安全，不会导致资源泄漏，但依赖子类正确放置检查点。如果 `doCall()` 实现中遗漏了检查点，中断可能不会及时响应。

### 4.2.4 MsgHub 订阅者管理与自动广播

每次 `call()` 完成后，结果自动广播给所有 MsgHub 订阅者：

```
private final Map<String, List<AgentBase>> hubSubscribers = new
ConcurrentHashMap<>();
```

```
// notifyPostCall 中:
.flatMap(msg -> broadcastToSubscribers(msg).thenReturn(msg));
```

这是**多 Agent 协作的核心通信机制**——Agent 的回复自动通知其他参与者，无需显式消息传递。

**执行逻辑：**

- 1、拿到 Agent 产出的消息 msg。

- 2、执行 `broadcastToSubscribers(msg)`：这是一个异步操作，负责遍历 `hubSubscribers` 列表，将消息推送给所有订阅者。
- 3、`.thenReturn(msg)`：这是一个非常巧妙的写法。它的意思是：“等广播操作完成后，忽略广播的返回值（通常是 `Void`），继续返回原始的消息 `msg`”。确保广播行为不会改变 Agent 原本的返回值，保证了业务流程的纯净性，同时完成了消息的“旁路广播”。

## 4.3 ReActAgent 执行引擎：框架的灵魂

`ReActAgent` (`agentscope-core/.../ReActAgent.java`, 约 1744 行) 是框架唯一的核心具体 Agent，实现了 **ReAct (Reasoning + Acting)** 范式。

### 4.3.1 核心依赖

```
private final Memory memory;           // 对话历史
private final String sysPrompt;        // 系统提示词
private final Model model;              // LLM 模型
private final int maxIters;             // 最大推理-行动迭代次数 (默认 10)
private final Toolkit toolkit;          // 可用工具集
private final ExecutionConfig modelExecutionConfig;
private final ExecutionConfig toolExecutionConfig;
private final GenerateOptions generateOptions;
private final PlanNotebook planNotebook; // 计划本
private final ToolExecutionContext toolExecutionContext;
private final StatePersistence statePersistence;
```

### 4.3.2 ReAct 循环（核心执行流）

用户消息 → [Reasoning → Acting] × N → 最终回复

1. **Reasoning 阶段** (`reasoning()` 方法)：

- 准备消息（系统提示 + Memory 全量历史）
- 调用 Model 进行流式推理
- 解析模型响应中的 ToolUseBlock（工具调用意图）
- 将推理消息添加到 Memory

## 2. **Acting 阶段** (`acting()` 方法)：

- 提取 ToolUseBlock 列表
- 执行工具调用（并行或串行）
- 将工具结果添加到 Memory

## 3. **循环控制**：

- 如果模型不再请求工具 → 返回最终文本回复
- 如果达到 `maxIters` → 生成摘要并返回
- 支持 `gotoReasoning()`（从 Hook 中跳回推理阶段重新思考）

**设计评价：** ReAct 循环是 LLM Agent 的经典范式，AgentScope-Java 的实现非常完整——支持流式推理、并行工具执行、计划本、中断、状态持久化等。

1744 行的代码量虽然较大，但考虑到功能的完备性，是合理的。不过，

`ReActAgent` 作为唯一的具具体 Agent 实现，职责略重。如果框架未来需要支持其他范式（如 Reflexion、Plan-and-Execute），当前架构的扩展点在于 `doCall()` 方法，但需要大量重写。

### 4.3.3 Builder 模式

ReActAgent 使用 Builder 模式构造，支持超过 20 个配置项。这是框架的入口点设计——所有复杂性封装在 Builder 中，用户只需关注需要的配置。

---

#### 思考题

4. Agent 接口强制组合 (Callable+Streamable+Observable) 的权衡是什么？如果你来设计，会怎么改进？
5. AgentBase.call()声明为 final 是否过于严格？在什么场景下子类可能需要覆盖调用流程？
6. PendingToolRecoveryHook 自动生成错误结果是否总是最优策略？还有什么替代方案？

**\*\*参考：\*\***

1. 强制组合的优势：API 一致性——所有 Agent 都支持三种调用方式，调用方无需判断类型。劣势：简单 Agent 被迫实现空方法（如不关心事件的 Agent 也要实现 Observable）。改进思路：提供 AbstractAgent 基类，默认空实现流式和事件方法，只在需要时覆盖。
2. final 过严的场景：A2A 代理需要完全覆盖调用流程（不走 AgentBase 的 Mono.using 生命周期）；需要自定义重试策略而非框架内置的。但放宽也有风险——子类可能跳过 CAS 锁检查导致并发问题。折中：保留 final，但开放更多扩展点（如 beforeCall/afterCall 模板方法）。
3. 不是最优——对用户关键操作（如付款确认），静默返回错误结果可能导致 Agent 误判执行成功。替代方案：①将 PendingTool 结果标记为 pending 状态，由 Agent 自行决定是否继续；②抛出特定异常让上层逻辑处理；③对于标记为 critical 的工具，直接中断当前 ReAct 循环。

# 第 5 章 消息模型——多模态内容的统一抽象

消息是 Agent 系统的血液——LLM 输入输出是消息，Agent 间通信是消息，Memory 存储的是消息，Pipeline 传递的还是消息。一个设计良好的消息模型直接决定了框架的表达能力和扩展性。

## 5.1 Msg——Agent 世界的通用信封

```
Msg (核心消息类)
├─ role: MsgRole (USER, ASSISTANT, SYSTEM, TOOL)
├─ content: List<ContentBlock> -- 多模态内容
└─ metadata: Map<String, Object>
```

Msg 的设计遵循“信封-内容”分离模式：`role` 和 `metadata` 是信封信息（路由、追踪），`content` 是信件内容（多模态数据块）。这种分离使得同一内容可以在不同角色间流转——用户发的内容（USER）和 Agent 返回的引用内容（ASSISTANT）共享相同的 `ContentBlock` 类型，只是信封不同。

`metadata` 使用 `Map<String, Object>` 而非强类型，因为不同场景需要不同的元数据：追踪场景需要 `traceId`，HITL 场景需要 `approvalStatus`，A2A 场景需要 `taskId`。强类型元数据会导致接口膨胀，`Map` 提供了足够的灵活性。代价是类型安全性——取值时需要强制转换，且无编译期校验。

# 5.2 ContentBlock——为什么选择 sealed interface

## 类型层次

```
ContentBlock (sealed interface)
├─ TextBlock          -- 文本
├─ ImageBlock         -- 图像 (URL 或 Base64)
├─ AudioBlock         -- 音频
├─ VideoBlock         -- 视频
├─ ThinkingBlock      -- 推理/思考过程
├─ ToolUseBlock        -- 工具调用请求
└─ ToolResultBlock    -- 工具执行结果
```

## sealed interface vs. enum：一个关键的设计选择

ContentBlock 选择 sealed interface 而非 enum，这并非随意决定。两者都可以实现类型穷举，但语义完全不同：

| 维度   | enum            | sealed interface          |
|------|-----------------|---------------------------|
| 实例数量 | 固定单例（每个枚举值一个实例） | 每次创建新实例                   |
| 携带数据 | 需额外字段（构造时固定）    | 每个实例自带不同数据                |
| 模式匹配 | switch 穷举       | switch + pattern matching |
| 扩展性  | 编译期封闭           | 同包/同模块内可扩展                |

**选择 sealed interface 的核心原因：**ToolUseBlock 每次实例携带不同的工具名和参数，ImageBlock 每次携带不同的图片数据。如果用 enum，每个"类型"只能有一个实例，无法表达"多个不同的工具调用"。

```
// sealed interface 允许每个实例携带不同数据
ContentBlock toolCall1 = new ToolUseBlock("get_weather", Map.of("city",
"Beijing"));
ContentBlock toolCall2 = new ToolUseBlock("search_web", Map.of("query",
"React"));

// enum 只能表示类型，数据需要额外关联
enum ContentType { TEXT, IMAGE, TOOL_USE, ... } // 数据去哪了？
```

## 模式匹配的编译期安全保障

sealed interface 的最大优势：编译器强制穷举。当框架新增 `VideoBlock` 时，所有 `switch` 语句都会编译报错，提醒开发者处理新类型：

```
String format(ContentBlock block) {
    return switch (block) {
        case TextBlock t -> t.text();
        case ImageBlock i -> "[image]";
        case ToolUseBlock t -> t.name() + "(" + t.arguments() + ")";
        // 编译器检查：如果遗漏 VideoBlock，直接报错！
    };
}
```

这个特性在 Formatter 体系中尤为关键——7 个 Provider 的 Formatter 都需要处理每种 ContentBlock，编译期穷举检查确保不会遗漏任何类型。

## 局限与扩展策略

sealed interface 的局限：**只有同包（Java 17+ 同模块）的类才能实现它**，用户无法自定义 ContentBlock 子类型。如果业务需要扩展（如自定义的 `EmbeddingBlock` 或 `TableBlock`），有三种策略：

1. **访问者模式**：注册自定义 `ContentBlockHandler`，框架在遇到未知类型时回调
2. **TextBlock 携带结构化数据**：将自定义内容编码为 TextBlock + metadata 标记，Formatter 层按标记特殊处理

3. **框架改造**：将 sealed interface 改为普通 interface + type 字段，牺牲编译期安全换取开放扩展

## 5.3 ThinkingBlock——写时全量、读时过滤

**ThinkingBlock** 是 AgentScope-Java 消息模型中最精巧的设计之一。它记录了 LLM 的推理过程（如“我需要先查天气再决定是否带伞”），但这个内容**不能原样发送给 LLM API**——大多数 Provider 的 API 不接受 thinking 内容作为输入，直接发送会导致 API 报错或 LLM 行为异常。

框架采用“写时全量、读时过滤”的策略：

1. **写入 Memory 时**：**ThinkingBlock** 原样保存——推理链是调试和审计的宝贵数据
2. **发送给 LLM API 时**：Formatter 层过滤掉 **ThinkingBlock**——只发送 TextBlock、ToolUseBlock 等 API 认识的类型
3. **流式输出时**：**ThinkingBlock** 作为 **ReasoningChunkEvent** 推送给前端——用户可以看到 Agent 的思考过程

这种模式在软件工程中有广泛的应用：数据库的 WAL (Write-Ahead Log) 写时全量记录、读时按需回放；日志系统写时全量采集、读时按级别过滤。核心思想是：**存储成本远低于信息丢失的代价，过滤可以在读取时灵活控制。**

## 5.4 设计评价

消息模型是框架中最稳定的基础层——`Msg` + `ContentBlock` 的设计在 `AgentScope-Java` 的整个生命周期中几乎没有变化，这说明初始设计足够前瞻。`sealed interface` 为多模态扩展预留了空间（`AudioBlock`、`VideoBlock` 的加入没有破坏任何现有代码），`ThinkingBlock` 的“写时全量、读时过滤”策略则精准地解决了推理链保存与 API 兼容性之间的矛盾。

主要不足：`metadata` 的 `Map<String, Object>` 缺乏类型安全；`ContentBlock` 的 `sealed` 限制使得框架用户无法扩展自定义内容类型，在垂直领域（如医疗影像、工业传感器数据）这可能成为瓶颈。

## 5.5 Source 体系——URL vs Base64 的工程权衡

内容来源有两种抽象：

- URLSource**：外部 URL 引用（如 `https://cdn.example.com/image.png`）
- Base64Source**：内联 Base64 编码（数据直接嵌入消息体）

这不是简单的“二选一”——两者在不同场景下有 10 倍以上的效率差异：

| 场景      | URLSource            | Base64Source     | 推荐           |
|---------|----------------------|------------------|--------------|
| 1MB 图片  | ~50 bytes (URL)      | ~1.33MB (Base64) | URLSource    |
| 10KB 图标 | 1 HTTP 请求 + 50 bytes | ~13KB            | Base64Source |
| 临时截图    | URL 可能过期             | 数据自包含            | Base64Source |

|         |      |      |           |
|---------|------|------|-----------|
| CDN 可缓存 | 天然支持 | 每次重传 | URLSource |
|---------|------|------|-----------|

AgentScope-Java 的 Formatter 层负责将 `Source` 转换为各 Provider 要求的格式：OpenAI 接受 URL 和 Base64 两种；Anthropic 只接受 Base64；Gemini 偏好 URL。框架在 `ImageBlock` 构建时自动选择最优编码方式，用户无需关心底层差异。

### 思考题

1. ContentBlock 使用 sealed interface 的设计有什么优势和局限？如果需要支持自定义 ContentBlock 类型，如何扩展？
2. ThinkingBlock 在 Formatter 层被过滤但在 Memory 中保留——这种"写时全量、读时过滤"的模式在哪些其他场景中也有应用？
3. URLSource 和 Base64Source 的传输效率差异在什么场景下最显著？如何选择？

**\*\*参考： \*\***

1. *sealed interface* 优势：编译期穷举检查，*switch* 模式匹配不会遗漏类型；局限：无法在框架外扩展新类型。扩展方式：①将 *sealed* 改为普通 *interface*，提供 *type* 标识；②使用访问者模式，注册自定义 *ContentBlockHandler*；③利用 *Java SPI* 注册新类型及其对应的 *Formatter* 处理逻辑。
2. 类似模式广泛存在：①数据库 WAL（写时全量记录、读时按需回放）；②日志系统（写时全量采集、读时按级别过滤）；③可观测性 *Trace*（写时全链路记录、读时按采样率展示）。核心思想：存储成本低，过滤灵活，避免信息丢失后无法恢复。

3. 差异在图像场景最显著: Base64 编码体积膨胀 33%, 一张 1MB 图片编码后 1.33MB 嵌入 JSON, 而 URL 仅占几十字节。选择策略: ①小文件 (<100KB) 且调用频次低→Base64 (减少一次 HTTP 请求); ②大文件或高频调用→URLSource (传输效率+CDN 缓存); ③涉及隐私/临时数据→Base64 (避免 URL 泄露)。

---

# 第 6 章 模型集成——七家 LLM 的统一适配

Agent 框架的模型集成层面临一个核心挑战：7 家 LLM 提供商的 API 格式各不相同，但 Agent 的上层逻辑（ReAct 循环、工具调用、流式输出）不应感知这些差异。模型集成层的设计决定了框架对新模型的适配成本。

## 6.1 三层模型架构

Model (接口)

└─ stream(messages, tools, options) -> Flux<ChatResponse>

ChatModelBase (抽象类, final stream())

└─ 模板方法: Tracer 包装 → Formatter 转换 → doStream() → 响应解析

OpenAIChatModel / AnthropicChatModel / DashScopeChatModel / GeminiChatModel  
/ OllamaChatModel

└─ 具体实现: doStream() + Provider 特有的 HTTP 交互逻辑

### 为什么 stream() 声明为 final

`ChatModelBase.stream()` 声明为 `final`，所有子类只能覆盖 `doStream()`。这保证了每次模型调用都经过基类的模板方法——Tracer 包装、度量收集、错误处理。这是模板方法模式的经典应用：不变的部分（横切关注点）在基类中固化，可变的（API 协议适配）留给子类。

如果允许子类覆盖 `stream()`，某个实现可能忘记 Tracer 包装，导致可观测性链路断裂——在生产环境中，一个没有 Tracer 的模型调用是“黑箱”，出问题时无法排查。

## 流式抽象—`Flux<ChatResponse>` 如何抹平 Provider 差异

不同 Provider 的流式传输实现差异很大：

| Provider  | 流式协议   | Chunk 格式                                                          | 结束标识                                         |
|-----------|--------|-------------------------------------------------------------------|----------------------------------------------|
| OpenAI    | SSE    | <code>`data: {"choices":[{"delta": "..."}]}`</code>               | <code>`data: [DONE]`</code>                  |
| Anthropic | SSE    | <code>`event: content_block_delta\ndata: {"delta": "...}"`</code> | <code>`event: message_stop`</code>           |
| DashScope | SSE    | 自定义 event type                                                    | <code>`event: conversation.completed`</code> |
| Gemini    | SSE    | <code>`candidates[0].content.parts[0].text`</code>                | <code>`finishReason: STOP`</code>            |
| Ollama    | NDJSON | <code>`{"response": "token", "done": false}`</code>               | <code>`{"done": true}`</code>                |

`Flux<ChatResponse>` 统一了这些差异——每个 Provider 的 `doStream()` 负责将各自的流式协议转换为 `Flux<ChatResponse>`，上层代码只看到统一的事件流。

**`doStream()` 的核心转换逻辑**（以 OpenAI 兼容模式为例）：

```
@Override
public Flux<ChatResponse> doStream(List<Msg> msgs, GenerateOptions options)
{
    // 1. Formatter 将统一消息转为 Provider 格式
    List<Map<String, Object>> apiMessages = chatFormatter.format(msgs,
        toolkit);
    Map<String, Object> requestBody = buildRequest(apiMessages, options);

    // 2. 发起 SSE 请求
    Flux<String> sseChunks = webClient.post()
        .uri("/chat/completions")
        .bodyValue(requestBody)
```

```
        .retrieve()
        .bodyToFlux(String.class);    // 每个 SSE data 行

// 3. 解析 SSE + 增量拼接 + 结束检测
return sseChunks
    .takeUntil(data -> "[DONE].equals(data)) // 结束检测
    .filter(data -> !"[/DONE].equals(data))
    .map(this::parseChunk)                // 解析 JSON
    .scan(this::accumulateDelta)          // 增量拼接 tool_call
    .map(this::toChatResponse);           // 转为统一格式
}
```

这段伪代码展示了四个关键转换步骤：(1) **格式化**——将框架的 `Msg` 列表转为 Provider 要求的 JSON 格式；(2) **SSE 请求**——发起 HTTP 流式请求；(3) **解析 + 拼接**——解析每个 chunk，拼接工具调用的分片参数；(4) **统一转换**——转为框架的 `ChatResponse`。步骤 3 是最复杂的部分——OpenAI 的 `tool_call` 参数可能分 5-10 个 chunk 到达，需要缓冲拼接直到参数完整。

### 错误重试策略

模型调用的失败类型和重试策略：

| 错误类型                  | 典型场景        | 重试策略                           |
|-----------------------|-------------|--------------------------------|
| Rate Limit (429)      | 并发过高        | 指数退避 + Respect `Retry-After` 头 |
| Server Error (5xx)    | Provider 故障 | 最多 3 次，间隔 1s/2s/4s             |
| Timeout               | 网络慢/模型负载高   | 超时后重试，可配置降级模型                  |
| Invalid Request (4xx) | 参数格式错误      | <b>**不重试**</b> ，直接失败           |
| Connection Reset      | 网络抖动        | 1 次立即重试                        |

AgentScope-Java 目前未内置自动重试——调用方（如 `ReActAgent`）负责在 `onErrorResume()` 中处理失败。这个设计选择是正确的：Agent 场景下的重试需

要业务感知（如重试前是否需要修改提示词、是否应该切换模型），框架层面的通用重试可能掩盖问题。

## 6.2 Formatter 体系：Provider 适配的核心

Formatter 是整个模型集成层的**关键抽象**，解决了"统一消息模型 → 多种 API 格式"的适配问题：

| Provider  | Chat Formatter                        | Multi-Agent Formatter                       | Conversation Merger                        |
|-----------|---------------------------------------|---------------------------------------------|--------------------------------------------|
| OpenAI    | <code>`OpenAIChatFormatter`</code>    | <code>`OpenAIMultiAgentFormatter`</code>    | <code>`OpenAIConversationMerger`</code>    |
| DeepSeek  | <code>`DeepSeekFormatter`</code>      | <code>`DeepSeekMultiAgentFormatter`</code>  | 继承 OpenAI                                  |
| GLM       | <code>`GLMFormatter`</code>           | <code>`GLMMultiAgentFormatter`</code>       | 继承 OpenAI                                  |
| Anthropic | <code>`AnthropicChatFormatter`</code> | <code>`AnthropicMultiAgentFormatter`</code> | <code>`AnthropicConversationMerger`</code> |
| DashScope | <code>`DashScopeChatFormatter`</code> | <code>`DashScopeMultiAgentFormatter`</code> | <code>`DashScopeConversationMerger`</code> |
| Gemini    | <code>`GeminiChatFormatter`</code>    | <code>`GeminiMultiAgentFormatter`</code>    | <code>`GeminiConversationMerger`</code>    |
| Ollama    | <code>`OllamaChatFormatter`</code>    | <code>`OllamaMultiAgentFormatter`</code>    | <code>`OllamaConversationMerger`</code>    |

## OpenAI 兼容模式—DeepSeek 和 GLM 的继承策略

DeepSeek 和 GLM 继承 OpenAI 的 ConversationMerger，因为它们的 API 协议与 OpenAI 高度兼容。这不是偶然——越来越多的模型提供商选择兼容 OpenAI API 格式作为事实标准。这种"OpenAI-compatible"模式带来了明显的维护收益：3 个 Provider 共享同一套合并逻辑。

但 Chat Formatter 层面仍然需要独立实现，因为即使是"兼容 OpenAI"的 Provider 也有微妙差异（如 DeepSeek 的 reasoning\_content 字段、GLM 的 tool\_call ID 格式）。这验证了一个设计原则：**协议兼容 ≠ 实现兼容**，协议层面的兼容性不能替代 Formatter 层的精细化适配。

### 每个 Provider 最独特的格式差异

| Provider  | 关键差异                                  | Formatter 处理方式                   |
|-----------|---------------------------------------|----------------------------------|
| OpenAI    | tool_calls 在 choices.delta 中分片发送      | 缓冲拼接完整 tool_call                 |
| Anthropic | 严格 USER/ASSISTANT 交替 + thinking block | ConversationMerger + thinking 过滤 |
| DashScope | 多种模型（qwen 系列）参数不一致                    | 按模型名分发不同格式                       |
| Gemini    | tools 定义在 tools[] 而非 functions[]      | 格式转换 + function_declarations     |
| DeepSeek  | reasoning_content 字段（推理链）             | 映射为 ThinkingBlock                |

|        |                   |                |
|--------|-------------------|----------------|
| Ollama | 本地模型，NDJSON 非 SSE | JSON 流解析替代 SSE |
|--------|-------------------|----------------|

**设计评价：**Formatter 体系正确地识别了"消息格式转换"作为核心变化轴。但 7×3=21 个 Formatter 类的维护成本较高。更好的做法是引入 `OpenAiCompatibleFormatter` 基类，将 OpenAI 协议的通用逻辑抽取出来，DeepSeek/GLM/Ollama 等兼容 Provider 只需覆盖差异方法。

### 6.3 ConversationMerger：多 Agent 对话压缩

当多个 Agent 在同一 Memory 中交互时，会产生连续的同角色消息。不同 API 对消息交替格式有不同要求——最严格的是 Anthropic，要求 USER/ASSISTANT 严格交替。ConversationMerger 的处理策略：

- 1. **连续 assistant 消息：**合并为单条（含多个 tool\_use）
- 2. **连续 user 消息：**合并为单条（拼接文本内容）
- 3. **无法合并时：**插入空 user 消息满足交替约束

第 3 种策略有争议——空 user 消息可能误导模型。更好的做法是让多 Agent 的消息通过 metadata 标记来源，Formatter 层按来源分组而非按角色合并，但这需要更深层的架构改造。

### 6.4 GenerateOptions：统一配置对象

`GenerateOptions` 聚合了所有 Provider 的配置参数（apiKey、baseUrl、temperature、maxTokens、thinkingBudget 等），并支持**分层合并**：

`mergeOptions(primary, fallback)` 实现逐级覆盖——请求级 > Builder 级 > 系统默认级。

**设计评价：** 单一配置对象简化了用户接口，但 `thinkingBudget`（仅 Anthropic）等 Provider 特有字段混入通用对象，编译期无法校验。更理想的设计是使用组合模式——`GenerateOptions` 包含通用字段 + `Map<String, Object> providerExtras`，Provider 特有配置放入 `extras` 并在运行时校验。当前方案在简洁性和完备性之间选择了前者。

---

## 思考题

1. Formatter 体系的代码膨胀问题（ $7 \times 3 = 21$  个类）如何解决？设计一个兼容 OpenAI 协议的通用 Formatter 基类。
2. Anthropic 要求严格 USER/ASSISTANT 交替——如果 Agent 连续两次调用工具，`ConversationMerger` 如何处理？
3. `GenerateOptions` 包含 Provider 特有字段（如 `thinkingBudget` 仅对特定 Provider 有效），这种设计有什么问题？如何改进？

**\*\*参考：\*\***

1. 解决方案：设计 OpenAI 兼容的通用基类

`OpenAiCompatibleFormatter`，覆盖 70% 的格式逻辑。Provider 差异通过策略方法暴露（如 `buildSystemMessage()`、`handleThinking()`），子类只需覆盖差异部分。参考 Spring 的 `AbstractRoutingDataSource` 模式——通用逻辑在基类，路由决策在子类。

2. `ConversationMerger` 在连续两次 assistant 消息间插入空 user 消息以满足

交替要求。但这引入了语义空白——空 *user* 消息可能误导模型。更好的做法：合并连续的 *tool\_result* 为单次 *assistant*+多 *tool\_call*，而非拆分为多条 *assistant* 消息。

3. 问题：字段语义依赖运行时 *Provider* 类型，编译期无法校验 (*thinkingBudget* 对非 *Anthropic* 模型无效但不报错)。改进：①使用 *Provider* 特有子类 (*AnthropicGenerateOptions* extends *GenerateOptions*)；②使用 *Map<String,Object> extraParams* 存储 *Provider* 特有字段，运行时校验；③*Builder* 模式按 *Provider* 类型限制可用字段。

---

# 第 7 章 工具系统——从函数调用到 MCP 生态

工具是 Agent 与外部世界交互的手——没有工具，Agent 只能“想”不能“做”。

工具系统的设计直接影响 Agent 的能力边界和安全性。

## 7.1 工具注册——四种方式，一个门面

框架支持多种工具注册方式，覆盖从简单到复杂的全部场景：

1. **注解驱动**（推荐）：在方法上标注 `@Tool` 和 `@ToolParam`，通过反射自动提取 Schema
2. **`AgentTool` 接口**：直接实现 `getName()`、`getDescription()`、`getParameters()`、`callAsync()`
3. **MCP 协议**：通过 `McpClientBuilder` 连接 MCP 服务器，自动导入工具
4. **SubAgentTool**：将任意 Agent 封装为工具，支持多轮会话

### @Tool 注解→JSON Schema 的生成原理

注解驱动的设计大幅降低了工具定义门槛。背后的原理是：框架通过反射提取方法签名、`@Tool` 描述和 `@ToolParam` 约束，自动生成符合主流 LLM 平台（如 OpenAI, Anthropic 等）要求的、精简版的 JSON Schema，用于描述工具的参数结构：

```
@Tool(description = "查询指定城市的天气")
public String getWeather(
    @ToolParam(description = "城市名称") String city,
    @ToolParam(description = "温度单位", required = false) String unit
) { ... }
```

生成的 JSON Schema:

```
{
  "name": "getWeather",
  "description": "查询指定城市的天气",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {"type": "string", "description": "城市名称"},
      "unit": {"type": "string", "description": "温度单位"}
    },
    "required": ["city"]
  }
}
```

这个 Schema 会发送给 LLM，LLM 据此生成结构化的工具调用请求。**Schema 的质量直接影响 LLM 的工具选择准确率**——描述模糊的参数会导致 LLM 误调。

注解驱动 vs. 接口实现—何时选择哪种

| 场景            | 推荐方式         | 原因             |
|---------------|--------------|----------------|
| Agent 自有业务逻辑  | @Tool 注解     | 简单、类型安全、IDE 友好 |
| 需要动态参数 Schema | AgentTool 接口 | Schema 可在运行时生成 |
| 第三方服务集成       | MCP 协议       | 服务端维护工具定义，自动发现 |
| 子 Agent 封装    | SubAgentTool | 需要多轮对话上下文      |

Toolkit 是工具管理的统一门面，内部委托给：

- ToolRegistry: ConcurrentHashMap 存储

- `ToolGroupManager`：工具分组管理（详见第 10 章 SkillBox）
- `ToolSchemaProvider`：JSON Schema 生成
- `McpClientManager`：MCP 客户端生命周期
- `ToolExecutor`：工具执行引擎

## 7.2 工具执行——从意图到结果

`ToolExecutor` 的执行流程：

```
工具调用请求 → 工具查找与验证 → 分组激活检查 → 参数验证 (JSON Schema)
→ 上下文合并 (预设参数注入) → 工具调用 → 结果转换 → 错误处理
```

### 并行 vs. 串行执行

当 LLM 一次输出多个工具调用时，框架需要决定执行策略：

- **并行** (`Flux.mergeSequential`)：多个工具同时执行，总延迟 =  $\max(\text{单个工具延迟})$ 。适用于独立工具（如同时查天气+查日历）
- **串行** (`Flux.concat`)：工具按顺序执行，总延迟 =  $\text{sum}(\text{所有工具延迟})$ 。适用于有依赖的工具（如先查订单号再查物流）

当前框架默认并行执行，串行需要调用方在 Agent 层面编排。这是合理的默认选择——大多数工具调用是独立的，串行依赖应通过 ReAct 循环自然解决（第一轮调工具 A，第二轮基于 A 的结果调工具 B）。

### `ToolSuspendException`——用异常做流程控制的争议

`ToolSuspendException` 的设计很巧妙——通过抛出异常来挂起执行流，保持 Reactor 响应式链的一致性。这种“异常流控”模式在语义上有争议：

**支持方：**在 Reactor 的 `onErrorResume` 范式下，这是自然的表达方式。挂起本质上是一种"中断"，与异常的语义接近

**反对方：**异常应该表示错误，挂起是正常的业务流程。调用方需要区分"工具挂起"和"工具异常"两种 error，增加复杂度

更优雅的替代方案是使用 Sealed Result 类型 (`ToolResult.Suspend / ToolResult.Success / ToolResult.Error`)，但这需要对 `ToolExecutor` 的返回类型做破坏性改造，当前方案在 Reactor 生态内是务实的折中。

## 7.3 SubAgentTool——Agent 即工具

`SubAgentTool` 是工具系统中一个容易被忽视但极具架构价值的设计——它将任意 Agent 封装为工具，使得 Supervisor Agent 可以通过统一的工具调用接口委派子任务。

```
// 将一个完整的 ReActAgent 封装为工具
Agent subAgent = ReActAgent.builder()
    .name("research_agent")
    .toolkit(researchTools)
    .memory(new AutoContextMemory(config, model))
    .build();

SubAgentTool researchTool = SubAgentTool.builder()
    .agent(subAgent)
    .description("负责深度研究和分析任务")
    .build();
```

### Agent 即工具的架构意义

这个抽象实现了**能力分层**：Supervisor Agent 负责路由和协调，Sub Agent 负责专业执行。从 Supervisor 的视角看，SubAgentTool 和普通工具没有区别——都是接收输入、返回结果。这保证了 Pipeline 编排的一致性。

## 风险与防护

1. **信息丢失**：SubAgentTool 的返回类型是 `ToolResult`（字符串），Sub Agent 的丰富输出（ThinkingBlock、中间工具调用）被压缩为单一结果。如果需要保留完整过程，需要在 metadata 中附加
2. **循环依赖**：Agent A 的 SubAgentTool 包含 Agent B，Agent B 的 SubAgentTool 又包含 Agent A。框架没有内置循环检测，需要开发者自律
3. **资源消耗**：每个 SubAgentTool 内部是一个完整的 Agent 实例（含 Memory、Hook 链、LLM 客户端），嵌套层数过多会显著增加延迟和 token 消耗

## 7.4 MCP 集成——接入外部工具生态

MCP (Model Context Protocol) 集成支持三种传输方式：

- **StdIO**：本地进程通信（stdin/stdout）——适合本地工具服务
- **SSE**：HTTP Server-Sent Events——适合远程工具服务
- **StreamableHTTP**：HTTP 流式传输——适合高吞吐场景

McpTool 实现了 AgentTool 接口，将 MCP 服务器的工具桥接到 AgentScope 工具系统。McpContentConverter 负责将 MCP 内容类型转换为 AgentScope 的 ContentBlock。

### MCP vs. @Tool : 工具生态的两种范式

| 维度     | @Tool 注解   | MCP 协议          |
|--------|------------|-----------------|
| 工具定义位置 | Agent 进程内  | 独立服务进程          |
| 发现方式   | 编译期确定      | 运行时动态发现         |
| 生命周期   | 与 Agent 相同 | 独立启停            |
| 跨语言    | 仅 Java     | 任意语言 (JSON-RPC) |
| 维护方    | Agent 开发者  | 工具服务团队          |
| 扩缩容    | 随 Agent 扩缩 | 独立扩缩            |

MCP 是 Agent 工具系统的"微服务化"——当工具逻辑复杂、需要独立部署和扩缩容时，MCP 是更好的选择。但 MCP 引入了网络延迟（一次工具调用需要一次 RPC），对于高频调用的轻量工具，@Tool 注解更高效。

## 7.5 设计评价

工具系统是 Agent 与外部世界交互的唯一通道，其设计直接决定了 Agent 的能力边界。注解驱动的注册方式将工具定义的门槛降到了最低——开发者只需写一个 Java 方法加两行注解。SubAgentTool 的"Agent 即工具"抽象实现了能力分层，保证了 Pipeline 编排的一致性。MCP 集成则打开了外部工具生态的大门。

主要不足：ToolSuspendException 的"异常流控"虽然在 Reactor 生态内可接受，但语义上仍有争议；SubAgentTool 缺乏循环依赖检测和嵌套深度限制；MCP SDK 的快速迭代意味着适配层需要频繁更新。

---

## 思考题

1. ToolSuspendException 使用异常进行流程控制是否合理？有什么替代设计？
2. MCP 协议与 AgentScope 原生@Tool 注解在工具定义上有什么本质差异？各适合什么场景？
3. SubAgentTool 将子 Agent 封装为工具——这种"Agent 即工具"的抽象有什么优势和风险？

*\*\*参考： \*\**

1. 异常做流程控制确实不够优雅——异常本意是处理错误，而非正常分支。替代方案：①返回 Sealed Result 类型 (Suspend / Continue / Error)，让调用方模式匹配处理；②用 Continuation/Coroutine 机制暂停恢复；③引入 ToolExecutionPolicy 枚举，工具声明自己的执行策略 (REQUIRE\_APPROVAL / AUTO)，由框架统一路由。
2. MCP：外部服务定义工具，Agent 通过协议发现和调用——适合第三方集成（如搜索引擎、数据库）；运行时动态发现。@Tool：Agent 内部定义工具，编译期确定——适合 Agent 自有能力；类型安全，IDE 支持好。选择：自有逻辑用@Tool，外部服务用 MCP。
3. 优势：统一抽象——Pipeline 编排无需区分 Agent 和工具，SubAgentTool 可透明插入 SequentialPipeline。风险：信息丢失——子

*Agent 的丰富输出被压缩为 ToolResult 字符串；循环依赖——A 调用 B 作为工具，B 又调用 A 作为工具。防护：限制嵌套深度+依赖 DAG 检测。*

---

# 第 8 章 记忆系统——从短期对话到长期认知

记忆是 Agent 区别于无状态 API 调用的关键——没有记忆，Agent 每次对话都从零开始，无法积累经验、理解上下文、保持连贯。记忆系统的设计直接决定 Agent 的"智商"上限。

人类记忆分为工作记忆（短期、容量有限）和长时记忆（长期、需要检索），AgentScope-Java 的记忆系统同样遵循这一双系统架构：短期记忆

（Memory）存储当前对话上下文，长期记忆（LongTermMemory）跨会话持久化关键信息。两者协作使 Agent 既能"记住当前任务"，也能"积累历史经验"。

## 8.1 短期记忆（Memory）——Agent 的工作台

短期记忆存储当前对话的消息历史，是 LLM 每次推理的直接输入。它的核心挑战不是"存什么"，而是"当上下文窗口不够时，如何取舍"。

```
Memory (接口)
├─ addMessage(), getMessages(), deleteMessage(), clear()
│   └─ InMemoryMemory      -- 默认, CopyOnWriteArrayList
└─ AutoContextMemory      -- 自动上下文压缩 (扩展)
```

### 8.1.1 InMemoryMemory

最简单的实现，使用 CopyOnWriteArrayList 保证线程安全。支持 StateModule 持久化。适合开发调试和短会话场景。

## 为什么用 `CopyOnWriteArrayList` 而非 `ArrayList + synchronized`? Agent

的消息访问模式是"读多写少"——每个 ReAct 循环中，LLM 调用需要读取全部消息 ( $O(n)$  遍历)，而写入只发生一次（追加新消息）。

`CopyOnWriteArrayList` 的写时复制策略虽然写入成本高 ( $O(n)$ )，但读操作无锁、无并发问题，完美匹配这个访问模式。

### 8.1.2 AutoContextMemory——6 级渐进压缩

`AutoContextMemory`（约 2000 行）是框架中最复杂的记忆实现。它解决的核心问题是：**当对话消息超过模型上下文窗口时，如何在保留关键信息的同时压缩上下文？**

#### 双存储架构

`AutoContextMemory` 采用双存储设计：

- `workingMemoryStorage`：压缩后的消息，用于实际 LLM 上下文（可变、可压缩）
- `originalMemoryStorage`：完整的原始消息，用于审计/回溯（只追加、不修改）

**为什么需要双存储？** 单一存储无法同时满足两个矛盾需求：LLM 需要精简的上下文（减少 token 消耗），但业务需要完整的对话记录（审计、调试、纠纷处理）。双存储让"推理用的压缩版"和"存档用的完整版"互不干扰。每次

`addMessage()` 同时写入两个存储，但只有 `workingMemoryStorage` 会执行压缩；

`getMessages()` 返回工作记忆的副本, `getOriginalMemoryMsgs()` 返回原始完整记录。

## 压缩触发机制

压缩检查发生在每次 LLM 推理之前 (通过

`AutoContextHook.handlePreReasoning()` 触发), 使用**双阈值 OR 判定**:

- **消息数阈值**: `workingMemoryStorage.size() >= msgThreshold` (默认 100 条)
- **Token 数阈值**: `estimatedTokens >= maxToken × tokenRatio` (默认 128K × 0.75 ≈ 98,304 tokens)

任一条件满足即触发压缩。Token 估算采用字符近似法 (`2.5 字符 ≈ 1 token`), 加上每条消息 5 token 的固定开销和工具调用 10 token 的额外开销。这种近似法无需加载分词器模型, 计算速度快, 但对中文的估算偏差较大 (中文实际约 1.5-2 字符/token), 可能高估 token 数导致过早触发压缩。

## 6 级渐进压缩策略

压缩策略按顺序尝试, **首个成功的策略立即返回**, 后续策略不再执行。这保证了每次压缩只做"最小的必要牺牲":

**策略 1: 压缩上轮工具调用**——找到上轮推理中连续的工具消息 (默认  $\geq 6$  条), 将其发送给 LLM 生成摘要, 替换为一行摘要 + 离线标记 `<!--`

`CONTEXT_OFFLOAD: uuid=... -->`。这是最温和的压缩——工具调用的细节往往对后续推理不再重要, 但保留了"做了什么"的摘要。

**策略 2：卸载大消息（保留最近 N 条）**——扫描工作记忆中超过

`largePayloadThreshold`（默认 5120 字符）的消息，将原文卸载到存储并替换为前 200 字符预览 + 离线标记。保护最近 `lastKeep` 条消息（默认 50 条）不被卸载。这一步是纯机械截断，**不消耗 LLM 调用**。

**策略 3：卸载大消息（不保留 lastKeep）**——与策略 2 相同逻辑，但只保护最新 assistant 响应及其之后的消息，不再保护 lastKeep 区域。这是策略 2 的加强版——当策略 2 无法释放足够空间时，牺牲更多的历史上下文。

**策略 4：摘要上轮对话**——找到最近一个 USER-FINAL\_ASSISTANT 对（不含工具调用的简单对话），将中间的工具调用链发送给 LLM 生成摘要。用户消息保留，仅替换 assistant 回复及中间过程。这一步开始"改写"对话内容，信息损失不可逆。

**策略 5：摘要本轮大消息**——在本轮（最近 USER 消息之后）中找到超过阈值的大消息，逐条调用 LLM 生成保守摘要（提示词强调"EXTRA CAREFUL and CONSERVATIVE"）。每条消息独立压缩，不会丢失整轮信息。

**策略 6：压缩本轮全部消息**——最激进的策略：将本轮所有消息（工具调用、工具结果、assistant 回复）合并为一条 ASSISTANT 摘要，目标压缩率为 30%（即摘要长度 = 原文 × 0.3）。压缩后的消息带有特殊标记 `_compressed_current_round`，防止被误认为真实 assistant 回复。

**压缩的成本与风险**

| 维度 | 分析 |
|----|----|
|----|----|

|          |                                                                |
|----------|----------------------------------------------------------------|
| Token 成本 | 策略 1/4/5/6 需要 LLM 调用生成摘要，每次压缩消耗额外 token（约为被压缩内容的 20-30%）       |
| 延迟影响     | LLM 压缩调用增加推理前延迟（约 1-3 秒/次），但在长对话中避免了超窗口错误                      |
| 信息损失     | 策略 2/3 是无损截断（原文仍可从 originalMemoryStorage 恢复），策略 1/4/5/6 是有损压缩  |
| 恢复机制     | 离线标记 `<!-- CONTEXT_OFFLOAD: uuid=... -->` 记录了原文的 UUID，理论上可按需恢复 |

**关键设计洞察：**6 级策略的渐进设计体现了"先损枝叶、后损主干"的原则——工具调用细节最不重要（策略 1），消息长度最不关键（策略 2/3），历史对话次重要（策略 4），当前轮的内容最重要（策略 5/6 最后才动）。这种优先级排序与人类对话的信息价值分布高度吻合。

## 8.2 长期记忆（LongTermMemory）

短期记忆解决"当前对话记住多少"的问题，长期记忆解决"跨对话积累什么"的问题。长期记忆允许 Agent 从历史交互中学习用户偏好、记录关键决策、积累领域知识。

```
LongTermMemory (接口)
└─ record(List<Msg>), retrieve(Msg) -> String
    └─ Mem0LongTermMemory    -- Mem0 向量存储后端
    └─ ReMeLongTermMemory    -- ReMe 后端
```

### 8.2.1 三种记忆模式

| 模式               | 机制                                                       | 适用场景        |
|------------------|----------------------------------------------------------|-------------|
| `AGENT_CONTROL`  | Agent 自主决定何时存储/检索<br><br>(通过<br><br>LongTermMemoryTools) | 用户偏好、关键决策   |
| `STATIC_CONTROL` | StaticLongTermMemoryHook<br><br>自动在调用前后触发                | 需要自动记录一切的场景 |
| `BOTH`           | 两者兼备                                                     | 最灵活但最复杂     |

**AGENT\_CONTROL 模式**将记忆操作暴露为工具（`LongTermMemoryRecordTool` 和 `LongTermMemoryRetrieveTool`），Agent 在推理过程中自主决定何时存储和检索。这符合 ReAct 范式的理念——Agent 把记忆当作一种“能力”来使用，而非被动接受的上下文。缺点是依赖 LLM 的判断力，可能遗漏重要记忆。

**STATIC\_CONTROL 模式**通过 `StaticLongTermMemoryHook` 实现对 Agent 透明的长期记忆：PreReasoning 阶段自动检索相关记忆并注入上下文，PostReasoning 阶段自动记录本轮对话。优点是不遗漏，缺点是每轮固定消耗额外 token（即使检索结果与当前任务无关）。

### 8.2.2 后端实现对比

| 后端   | 存储方式        | 检索方式      | 适用规模         |
|------|-------------|-----------|--------------|
| Mem0 | 向量数据库（自动嵌入） | 语义相似度检索   | 中等规模（<10 万条） |
| ReMe | 结构化记忆图谱     | 关键词+语义混合检 | 大规模、需要关联推    |

|  |  |   |   |
|--|--|---|---|
|  |  | 索 | 理 |
|--|--|---|---|

Mem0 后端将每条记忆自动嵌入为向量，检索时按语义相似度排序返回 Top-K。ReMe 后端维护记忆之间的关系图谱，支持"用户提到过 A，而 A 关联 B"的推理链。两者均通过 `record()` 存入、`retrieve()` 检索，接口统一但能力差异显著。

### 8.3 短期记忆与长期记忆的协作

在实际运行中，短期记忆和长期记忆的协作流程如下：

- PreReasoning 阶段：**AutoContextHook 检查并压缩工作记忆；  
STATIC\_CONTROL 模式同时检索长期记忆并注入上下文
- 推理阶段：**LLM 基于（可能压缩后的）工作记忆 + 注入的长期记忆进行推理；AGENT\_CONTROL 模式下 LLM 可能主动调用记忆工具
- PostReasoning 阶段：**新消息追加到工作记忆和原始记忆；  
STATIC\_CONTROL 模式自动记录本轮对话到长期记忆

**协作中的矛盾：**长期记忆的注入增加了工作记忆的 token 负担，可能加速触发压缩。这意味着"记住更多历史"和"保持当前上下文"之间存在张力——一种工程权衡是限制长期记忆注入的 token 数（如最多注入 2000 token 的检索结果），留出空间给当前对话。

*\*\*延伸阅读： \*\* 关于 AutoContextMemory 的参数调优，参见第 19.3 节；关于在 Boba Tea Shop 案例中使用 AutoContextMemory 的实践，参见第 23.2 节。*

**设计评价：** AutoContextMemory 的 6 级渐进压缩是工程上极其精巧的设计——它不是简单粗暴地截断上下文，而是像人类遗忘一样“先忘细节、再忘过程、最后忘结论”。双存储保证了审计可追溯性，离线标记提供了恢复可能性。但当前实现也存在不足：(1) Token 估算的字符近似法对中文偏差较大；(2) 压缩策略的触发阈值是静态配置，无法根据对话质量动态调整；(3) 6 级策略之间的粒度跳跃较大，缺乏中间过渡。长期记忆的三种模式覆盖了主要使用场景，但 AGENT\_CONTROL 和 STATIC\_CONTROL 的组合（BOTH 模式）缺乏冲突处理机制——当 Agent 主动检索的记忆与 Hook 自动注入的记忆重叠时，如何去重？

---

## 思考题

1. AutoContextMemory 的 6 级渐进压缩策略中，哪一级对推理质量影响最大？如何量化评估？
2. AGENT\_CONTROL 和 STATIC\_CONTROL 两种长期记忆模式在 token 消耗上有什么差异？
3. 双存储架构（workingMemoryStorage + originalMemoryStorage）的一致性如何保证？

**\*\*参考：\*\***

1. 影响最大的是第 3 级（摘要压缩）——直接改写原始内容，信息损失不可逆。量化方法：①用原始记忆和压缩记忆分别跑同一组测试问题，对比答案准确率；②用 LLM 评估压缩前后的语义保留度（1-5 分）；③跟踪压缩后 Agent 的工具调用准确率变化（能否基于压缩记忆选对工具）。

2. AGENT\_CONTROL 模式：每次对话 LLM 自主决定是否检索长期记忆，  
token 消耗 = 基础对话 + 每次检索的 query+结果。STATIC\_CONTROL 模式：系统每轮自动注入长期记忆摘要，token 消耗 = 基础对话 + 每轮固定的摘要长度。当长期记忆量大时，STATIC\_CONTROL 更耗 token（每轮固定成本），AGENT\_CONTROL 更省（按需检索），但 AGENT\_CONTROL 的 LLM 判断本身也消耗 token。

3. 一致性保证依赖写入顺序：先写 workingMemoryStorage（当前对话上下文），再写 originalMemoryStorage（原始完整记忆）。读时只读 workingMemoryStorage。风险点：崩溃发生在两次写之间——workingMemoryStorage 已更新但 originalMemoryStorage 未更新。缓解：使用 WAL（Write-Ahead Log）或幂等写入。

---

# 第 9 章 PlanNotebook——结构化规划系统

## 9.1 设计动机

ReAct 范式的 Agent 通过"推理-行动"循环逐步解决问题，但存在两个核心缺陷：

- 1. **缺乏全局视野**：每一步只考虑当前状态，容易迷失方向或重复工作
- 2. **无法处理复杂任务**：面对需要多步骤协调的任务（如"构建一个网站"），纯 ReAct 循环效率低下

PlanNotebook 通过引入**结构化规划**解决了这些问题（如本章所述）——Agent 在执行复杂任务前先创建计划，按步骤推进，每步完成后记录成果。

## 9.2 数据模型

```
PlanNotebook [PlanNotebook.java:104]
├─ currentPlan: Plan          ← 当前活跃计划
├─ planToHint: PlanToHint    ← 计划→提示转换器
├─ storage: PlanStorage      ← 历史计划存储
├─ maxSubtasks: Integer      ← 子任务上限 (null=无限制)
├─ needUserConfirm: boolean  ← 是否需要用户确认 (默认 true)
└─ changeHooks: Map<String, BiConsumer> ← 变更通知回调

Plan [Plan.java]
├─ name: String
└─ description: String
```

```

├ expectedOutcome: String
├ state: PlanState (ACTIVE, DONE, ABANDONED)
├ subtasks: List<SubTask>
└ createdAt / updatedAt

SubTask [SubTask.java]
├ name: String
├ description: String
├ expectedOutcome: String
├ outcome: String          ← 完成时填写的实际成果
└ state: SubTaskState (TODO, IN_PROGRESS, DONE, ABANDONED)

```

## 9.3 状态转移规则

SubTask 状态转移有严格的约束条件：

```

stateDiagram-v2
    [*] --> TODO
    TODO --> IN_PROGRESS: 前序任务完成
    TODO --> ABANDONED: 跳过
    IN_PROGRESS --> DONE: finishSubtask()
    IN_PROGRESS --> ABANDONED: 放弃
    DONE --> [*]
    ABANDONED --> [*]

    note right of IN_PROGRESS: 同时只能有一个
    note right of DONE: 必须提供 outcome\n 自动激活下一个

```

自动激活机制 ([PlanNotebook.java:717-728](#)):

```

// finishSubtask() 完成后的自动激活逻辑
private void autoActivateNextSubtask(Plan plan, int completedIndex) {
    for (int i = completedIndex + 1; i < plan.getSubtasks().size(); i++) {
        SubTask next = plan.getSubtasks().get(i);
        if (next.getState() == SubTaskState.TODO) {
            next.setState(SubTaskState.IN_PROGRESS);
            break;
        }
    }
}

```

# 9.4 十个工具函数

| #  | 工具名                       | 功能          | 关键约束                          |
|----|---------------------------|-------------|-------------------------------|
| 1  | `create_plan`             | 创建新计划       | 子任务数不超过 maxSubtasks; 已有计划会被替换 |
| 2  | `update_plan_info`        | 更新计划元数据     | null/空字符串表示不修改                |
| 3  | `revise_current_plan`     | 增加/修改/删除子任务 | 支持按索引操作                       |
| 4  | `update_subtask_state`    | 更新子任务状态     | 前序任务必须完成                      |
| 5  | `finish_subtask`          | 完成子任务       | 必须提供 outcome; 触发自动激活          |
| 6  | `view_subtasks`           | 查看子任务详情     | 按索引列表查看                       |
| 7  | `get_subtask_count`       | 统计子任务数量     | 按状态分组统计                       |
| 8  | `finish_plan`             | 完成或放弃计划     | 设置 PlanState                  |
| 9  | `view_historical_plans`   | 查看历史计划      | 从 PlanStorage 加载              |
| 10 | `recover_historical_plan` | 恢复历史计划      | 当前计划将被放弃                      |

**create\_plan 实现详解** (PlanNotebook.java:268-331):

```
@Tool(name = "create_plan", description = "Create a plan by given name and sub-tasks")
public Mono<String> createPlan(
    @ToolParam(name = "name", ...) String name,
    @ToolParam(name = "description", ...) String description,
    @ToolParam(name = "expected_outcome", ...) String expectedOutcome,
    @ToolParam(name = "subtasks", ...) List<Map<String, Object>>
subtasks) {
    // 转换 Map -> SubTask 对象
    List<SubTask> subtaskList = new ArrayList<>();
    for (Map<String, Object> subtaskMap : subtasks) {
        subtaskList.add(mapToSubTask(subtaskMap));
    }
    // 验证子任务数量上限
    if (maxSubtasks != null && subtaskList.size() > maxSubtasks) {
```

```

        return Mono.just("Cannot create plan: exceeds maximum limit...");
    }
    // 创建计划
    Plan plan = new Plan(name, description, expectedOutcome, subtaskList);
    // 替换当前计划 (如果存在)
    String message = currentPlan == null
        ? "Plan created successfully."
        : "Current plan replaced by new plan.";
    currentPlan = plan;
    return triggerPlanChangeHooks().thenReturn(message);
}

```

## 9.5 提示注入系统 (PlanToHint)

PlanNotebook 不仅提供工具，还在每次推理前自动注入上下文提示，引导 Agent 按计划执行。

**DefaultPlanToHint** ([plan/hint/DefaultPlanToHint.java:38-256](#)) 生成

<system-hint> 标签包裹的提示，覆盖 5 种场景：

场景 1：无计划 → "对于复杂任务，建议创建计划"

场景 2：计划开始（所有子任务 TODO）→ "计划已创建，建议从第一个子任务开始"

场景 3：子任务进行中 → "当前子任务详情 + 可用操作选项"

场景 4：无进行中子任务 → "请选择下一个子任务开始执行"

场景 5：所有子任务完成 → "所有子任务已完成，建议完成计划"

**提示注入机制**——PlanNotebook 通过注册为 Hook 实现提示注入：

1. `ReActAgent.Builder` 创建 PlanNotebook 时，自动将其工具注册到 Toolkit
2. PlanNotebook 的 `getHint()` 方法在 `PreReasoningEvent` 中被调用
3. 生成的提示追加到推理输入消息中

## 9.6 变更钩子（Change Hooks）

外部系统可以注册 `BiConsumer<PlanNotebook, Plan>` 回调，在每次计划变更时触发：

```
// PlanNotebook.java LINE 1024-1041
planNotebook.addChangeHook("ui_update", (notebook, plan) -> {
    // 更新前端 UI 显示当前计划状态
    uiService.updatePlanView(plan);
});
```

这使得 `PlanNotebook` 可以与实时 UI 集成，展示 Agent 的规划进度。

## 9.7 状态持久化

```
// PlanNotebook.java LINE 157-175
@Override
public void saveTo(Session session, SessionKey sessionKey) {
    // 即使 currentPlan 为 null 也保存（清除状态也需要持久化）
    session.save(sessionKey, keyPrefix + "_state", new
PlanNotebookState(currentPlan));
}

@Override
public void loadFrom(Session session, SessionKey sessionKey) {
    this.currentPlan = null; // 先清除，避免残留
    session.get(sessionKey, keyPrefix + "_state", PlanNotebookState.class)
        .ifPresent(state -> this.currentPlan = state.currentPlan());
}
```

**设计评价：** `PlanNotebook` 是框架中**最精巧的元认知系统**——它让 Agent 不仅能“做”，还能“计划做”。十个工具函数覆盖了计划的完整生命周期，状态转移规则保证了执行的有序性，自动激活机制减少了 Agent 的决策负担，`PlanToHint` 的上下文注入让 Agent 无需主动查询就能知道下一步该做什么。但这种设计也带来了复杂度——Agent 需要理解并正确使用这些工具，如果 LLM 的指令遵循能力不足，可能会出现计划与执行脱节的情况。

## 思考题

1. PlanNotebook 的自动激活机制在什么情况下可能导致问题？如何改进？
2. 如果 Agent 的 LLM 指令遵循能力较弱，PlanNotebook 的工具函数可能被误用。如何设计防护措施？
3. PlanToHint 的 5 种场景提示是否足够？是否需要增加“计划偏离”的场景提示？

**\*\*参考： \*\***

1. 自动激活问题：简单任务（如“今天天气”）被强制赋予 PlanNotebook，LLM 可能过度规划——为单步操作生成 10 个 SubTask。改进：①增加复杂度评估 Hook，仅当意图分类为复杂时激活；②提供@NoPlan 注解让开发者显式跳过；③阈值可配（如工具数>3 才激活）。
  2. 防护措施：①输入校验——SubTask 的 description 字段限长+正则白名单；②输出约束——限制单次计划 SubTask 数量上限（如 maxSubTasks=10）；③沙箱执行——PlanNotebook 的变更只在当前 ReAct 循环可见，循环结束后自动清理；④监控——检测异常模式（如连续 5 次创建又删除 SubTask）。
  3. 需要增加“计划偏离”提示。场景：Agent 执行 SubTask\_3 时发现前提假设错误，需要回退修改 SubTask\_1。新增 hint：PlanToHint.DEVIATION="当前执行结果与原计划不一致，请评估是否需要调整后续计划"。还需要：PARTIAL\_COMPLETE（部分完成无法继续）、RESOURCE\_LIMIT（token/时间不足需缩减计划）。
-

# 第 10 章 SkillBox——动态技能系统

## 10.1 设计动机

当 Agent 拥有大量工具时，两个问题随之而来：

1. **LLM 选择困难**：工具越多，LLM 越难选择正确的工具，误调率上升
2. **安全隔离**：某些工具只在特定场景下可用，不应始终暴露

### 工具数量 vs. 准确率的实证关系

这是一个被大量实证研究验证的问题。Google Research 2024 年的实验表明：当可用工具从 10 个增加到 50 个时，LLM 的工具选择准确率从 ~95% 下降到 ~70%；超过 100 个工具时，准确率进一步降至 ~50%。原因不仅是"选择太多"，更关键的是**工具描述之间的语义冲突**——多个工具的功能描述相似时，LLM 无法区分。

SkillBox 通过**动态技能加载**解决这些问题——技能默认不激活，只有当 Agent 明确加载某个技能后，相关工具才可见。这相当于在工具层面实现了"分页"——每次只向 LLM 展示当前任务相关的工具子集，将有效工具数控制在 10-20 个的高准确率区间。

## 10.2 核心架构

SkillBox [SkillBox.java:1-1144]

└─ SkillRegistry

← 技能注册表（版本管理）

|                            |                              |
|----------------------------|------------------------------|
| — AgentSkillPromptProvider | ← 技能提示生成器                    |
| — SkillToolFactory         | ← 技能工具包装器工厂                  |
| — Toolkit (外部引用)           | ← 工具注册/激活的底层执行者              |
| — CodeExecutionBuilder     | ← 代码执行子系统 (Shell/Read/Write) |

## 10.3 技能注册——Fluent Builder 模式

```
// SkillBox.java LINE 116-118, 301-539
skillBox.registration()
    .skill(mySkill)           // 必须: AgentSkill 实例
    .tool(myToolObject)      // 可选: @Tool 注解对象
    .agentTool(myAgentTool)  // 可选: AgentTool 实例
    .mcpClient(mcpWrapper)   // 可选: MCP 客户端
    .subAgent(provider, config) // 可选: 子 Agent
    .presetParameters(...)    // 可选: 隐藏参数
    .apply();                 // 执行注册
```

## 10.4 Tool Group 隔离机制

**关键设计：**每个技能的工具注册在独立的 ToolGroup 中，且**默认禁用**：

```
// SkillBox.java LINE 523-525
// 每个技能创建一个独立工具组，初始状态为禁用
String groupName = skillId + "_skill_tools";
toolkit.registration().group(groupName).enabled(false).apply();
```

这意味着 LLM 在推理时**看不到**这些工具——它们不会出现在

`toolkit.getToolSchemas()` 中。

## 10.5 激活/去激活流程

```
syncToolGroupStates() [SkillBox.java:149-173]
|
|— 遍历所有注册的技能
```

```

|   | — 技能激活 → toolkit.setGroupEnabled(skillId + "_skill_tools", true)
|   | — 技能未激活 → toolkit.setGroupEnabled(skillId + "_skill_tools", false)
|
| — deactivateAllSkills() [LINE 290-293]
    → 在每次 agent.call() 开始时调用
    → 所有技能设为未激活
    → Agent 需要重新加载技能才能使用

```

## 10.6 SkillHook——提示注入

**SkillHook** (**SkillHook.java:29-94**) 在 **PreReasoningEvent** (priority=55) 中注

入技能提示：

```

// SkillHook.java LINE 44-67
public <T extends HookEvent> Mono<T> onEvent(T event) {
    if (event instanceof PreReasoningEvent preReasoning) {
        String skillPrompt = promptProvider.getPrompt(skillBox);
        if (skillPrompt != null) {
            List<Msg> messages = preReasoning.getInputMessages();
            // 策略 1: 有 SYSTEM 消息 → 追加到现有 SYSTEM 消息
            if (hasSystemMessage) {
                existingSystemMsg.getContent().add(TextBlock.builder().text(skillPrompt).build());
            }
            // 策略 2: 无 SYSTEM 消息 → 前置插入新 SYSTEM 消息
            else {
                messages.add(0, Msg.builder().role(MsgRole.SYSTEM)
                    .content(TextBlock.builder().text(skillPrompt).build()).build());
            }
        }
    }
    return Mono.just(event);
}

```

**设计选择：**将技能提示合并到现有 SYSTEM 消息中（而非插入独立消息），保持了消息列表的简洁性，也避免了某些 API 对连续 SYSTEM 消息的限制。

## 10.7 代码执行子系统

SkillBox 内置了 `CodeExecutionBuilder` (LINE 575-1142) , 提供安全的代码执行能力:

```
skillBox.codeExecution()  
    .withShell()           // 注册 Shell 命令执行工具  
    .withRead()           // 注册文件读取工具  
    .withWrite()          // 注册文件写入工具  
    .allowedCommands("ls", "cat", "python") // Shell 命令白名单  
    .workingDirectory("/tmp/agent")         // 文件操作根目录  
    .build();
```

**安全机制:**

- **命令白名单:** 只有 `allowedCommands` 列表中的命令可执行
- **路径遍历防护:** `SkillFileFilter` 检查目标路径是否在技能目录内 (LINE 753:  
`targetPath.startsWith(skillDir)`)
- **临时目录:** 自动创建临时目录, 注册 JVM 关闭时的清理钩子 (LINE 662-668)

## 10.8 内置工具 : `load_skill_through_path`

SkillBox 注册了一个内置工具 `load_skill_through_path` (LINE 542-573) , 在 `skill-build-in-tools` 工具组中 (始终启用) 。LLM 可以通过此工具动态加载技能资源或 SKILL.md 内容。

*\*\*设计评价:\*\* SkillBox 的"默认禁用+按需激活"模式是解决工具爆炸问题的优雅方案。ToolGroup 隔离确保了安全性, SkillHook 的提示注入让 Agent 自然地感知可用技能。但 `deactivateAllSkills()` 在每次 call() 开始时重置所有技能状态, 意味着 Agent 必须在每次对话中重新加载技能——这增加了*

*token 消耗，但也防止了技能状态在不同对话间泄漏。对于需要跨对话保持技能激活的场景，可以考虑增加"持久技能"配置。*

## 思考题

1. SkillBox 的 deactivateAllSkills()在每次 call()时重置——这种设计的利弊分别是什么？
2. 如果一个技能需要另一个技能的输出作为输入，当前的 ToolGroup 隔离机制能否支持？如何设计技能间依赖？
3. ShellCommandTool 的多层安全防护中，哪一层最可能被绕过？为什么？

**\*\*参考：\*\***

1. 利：每次调用都是干净状态，避免上次激活的技能影响本次推理（防止"技能污染"）。弊：无法保持跨调用的技能状态（如多轮对话中用户说"继续用上次的功能"）。改进：增加 skillScope 参数（PER\_CALL / PER\_SESSION），Session 级别技能在会话期间保持激活。

2. 当前 ToolGroup 隔离机制不支持技能间依赖——每个 ToolGroup 的 Toolkit 互不可见。设计方案：①显式依赖声明——@Skill(requires="database\_query")，框架自动将依赖工具合并到同一 ToolGroup；②共享上下文——通过 Memory 传递中间结果（技能 A 写入 Memory，技能 B 从 Memory 读取）；③Pipeline 组合——将依赖技能编排为 SequentialPipeline 而非并行 ToolGroup。

3. 最可能被绕过的是 CommandValidator 的黑名单——黑名单本质是"已知的危险命令集"，无法覆盖所有变体。例如：用 `bash -c "$(echo rm -rf | base64 -d)"` 绕过关键词检测。加固：①改为白名单模式（只允许明确列出的命令）；②使用操作系统级沙箱（Docker/namespaces 隔离）；③审计日志+告警。

# 第 11 章 优雅关闭——生产环境的守护者

## 11.1 为什么需要优雅关闭

在 Agent 执行过程中强制终止（如 K8s Pod 漂移、Docker 容器重启）会导致严重后果：

- **工具调用半完成**：工具已执行但结果未写回 Memory，LLM 下次重试会重复执行
- **状态丢失**：推理过程、Memory、Toolkit 状态等全部丢失
- **客户端无响应**：HTTP 连接断开，客户端不知道 Agent 执行到了哪里

AgentScope-Java 的优雅关闭系统确保：**正在执行的 Agent 可以安全完成当前阶段，状态被持久化，客户端可以恢复执行。**

## 11.2 三状态生命周期

```
stateDiagram-v2
    [*] --> RUNNING
    RUNNING --> SHUTTING_DOWN: SIGTERM / shutdown()
    SHUTTING_DOWN --> TERMINATED: 所有活跃请求完成
    SHUTTING_DOWN --> TERMINATED: 超时强制中断
    TERMINATED --> [*]

    note right of RUNNING: 正常接受请求
```

note right of SHUTTING\_DOWN: 不接受新请求,等待完成

note right of TERMINATED: JVM 可安全退出

状态转换由 `GracefulShutdownManager` (LINE 41-321) 管理:

```
// GracefulShutdownManager.java LINE 46-53
private final AtomicReference<ShutdownState> state =
    new AtomicReference<>(ShutdownState.RUNNING);
private final ConcurrentHashMap<String, ActiveRequestContext>
activeRequestsByAgentId =
    new ConcurrentHashMap<>();
private final ConcurrentHashMap<String, ShutdownSessionBinding>
sessionBindings =
    new ConcurrentHashMap<>();
```

## 11.3 完整关闭流程

SIGTERM 信号到达 JVM

```
|
| └─ AgentScopeJvmShutdownHook.run() [AgentScopeJvmShutdownHook.java:34-70]
| |
| | └─ 1. performGracefulShutdown() [LINE 198-213]
| |   └─ state: RUNNING → SHUTTING_DOWN
| |     └─ 记录 shutdownStartedAt 时间戳
| |       └─ startMonitorIfNeeded() → 启动超时监控线程
| |
| | └─ 2. awaitTermination(timeout + 5s) [LINE 284-303]
| |   └─ 阻塞等待所有活跃请求完成
| |     └─ 每 1 秒检查 terminationLock
| |       └─ 超时后返回 false
| |
| └─ 3. HttpTransportFactory.shutdown() [最后执行]
    └─ 关闭所有 HTTP 连接
|
└─ (超时监控线程并行运行)
    enforceTimeoutAndInterrupt() [LINE 232-265]
    └─ 每秒检查一次
    └─ elapsed >= shutdownTimeout → 强制中断
        └─ shutdownTimeoutSignal.tryEmitEmpty() ← 通知工具执行器中止
            └─ 遍历 activeRequests: saveToSession() ← 保存状态
                └─ 遍历 activeRequests: interruptForShutdown() ← 强制中断
                    └─ 所有请求完成后: state → TERMINATED
```

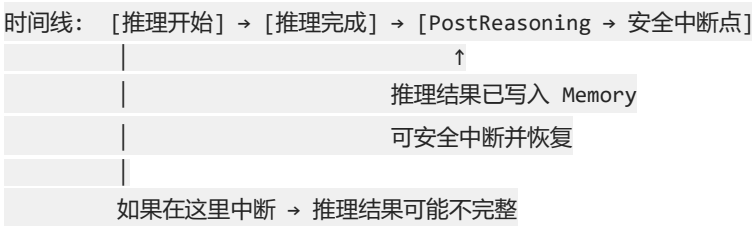
# 11.4 安全点检查——GracefulShutdownHook

GracefulShutdownHook (LINE 54-103, priority=0 最高优先级) 实现了"安全点

"检查——只在阶段完成后才中断，不浪费已生成的 Token:

```
GracefulShutdownHook.onEvent() [LINE 65-81]
|
|— PreCallEvent [LINE 67-69]:
|   |— deduplicateIfResuming() → 检测 shutdown 中断后的重试，去重输入
|
|— PostReasoningEvent [LINE 72-73]:
|   |— manager.interruptIfShuttingDown() → 推理完成后安全中断
|
|— PostActingEvent [LINE 74-75]:
|   |— manager.interruptIfShuttingDown() → 工具执行完成后安全中断
|
|— PostSummaryEvent [LINE 76-77]:
|   |— manager.interruptIfShuttingDown() → 摘要生成完成后安全中断
```

**关键设计:** 中断发生在 PostEvent 而非 PreEvent，确保当前阶段的结果被完整保存:



# 11.5 客户端重试与消息去重

当服务端因关闭而中断后，客户端通常会使用相同的用户消息重试。但 Memory 中已存在该用户消息，直接重试会导致重复。

GracefulShutdownHook 在 PreCallEvent 中处理这个问题:

```
// GracefulShutdownHook.java LINE 88-97
private void deduplicateIfResuming(PreCallEvent event) {
    if (manager.checkAndClearShutdownInterrupted(event.getAgent())) {
        // 检测到 shutdown 中断后的重试

        // 将输入消息替换为空列表 → Agent 从 Memory 恢复执行
        event.setInputMessages(List.of());
    }
}
```

`checkAndClearShutdownInterrupted()` (`GracefulShutdownManager.java:113-`

133) 从 Session 中读取 `shutdown_interrupted` 标志:

```
public boolean checkAndClearShutdownInterrupted(Agent agent) {
    ShutdownSessionBinding binding = sessionBindings.get(agent.getAgentId());
    Optional<ShutdownInterruptedState> flag =
        binding.session().get(binding.sessionKey(),
            ActiveRequestContext.SHUTDOWN_INTERRUPTED_KEY,
            ShutdownInterruptedState.class);
    if (flag.isPresent() && flag.get().interrupted()) {
        binding.session().delete(binding.sessionKey(),
            ActiveRequestContext.SHUTDOWN_INTERRUPTED_KEY);
        return true; // 检测到中断标志 → 去重
    }
    return false;
}
```

## 11.6 超时信号与工具执行器的协作

`GracefulShutdownManager` 提供了一个响应式超时信号，工具执行器可以与之竞争:

```
// GracefulShutdownManager.java LINE 93-95
public Mono<Void> getShutdownTimeoutSignal() {
    return shutdownTimeoutSignal.get().asMono();
}
```

`applyShutdownGuard()` (`ToolExecutor.java:408-418`) 利用此信号:

```
private Mono<ToolResultBlock> applyShutdownGuard(Mono<ToolResultBlock>
execution) {
    return Mono.firstWithSignal(
        execution,
        manager.getShutdownTimeoutSignal().then(Mono.error(new
AgentShuttingDownException()))
```

```
    ).next();  
}
```

当关闭超时信号触发时，工具执行会被中止——即使工具正在执行中。

## 11.7 PartialReasoningPolicy——不完整推理的处理

当 Agent 在推理阶段被中断时，ReasoningContext 可能只有部分累积的内容。`GracefulShutdownConfig` 提供两种策略：

| 策略          | 行为                      | 适用场景                    |
|-------------|-------------------------|-------------------------|
| `SAVE` (默认) | 保存部分推理内容到<br><br>Memory | 推理内容有价值，希望恢复时<br><br>继续 |
| `DISCARD`   | 丢弃部分推理内容                | 推理内容不完整，重试更安全           |

实现位置 (`ReActAgent.java:491-509`):

```
.onErrorResume(InterruptedExcption.class, error -> {  
    Msg msg = context.buildFinalMessage();  
    if (msg != null) {  
        boolean discard =  
            getSource() == InterruptSource.SYSTEM  
            && shutdownManager.getConfig().partialReasoningPolicy()  
            == PartialReasoningPolicy.DISCARD;  
        if (!discard) {  
            memory.addMessage(msg); // 保存部分推理  
        }  
        // SYSTEM + DISCARD -> 丢弃  
    }  
    return Mono.error(error);  
})
```

## 11.8 ActiveRequestContext——活跃请求的生命周期

```
ActiveRequestContext [ActiveRequestContext.java:29-77]
|
|— requestId: String          ← 唯一请求 ID
|— agent: AgentBase           ← 关联的 Agent
|— session: Session           ← 持久化后端
|— sessionKey: SessionKey     ← 会话键
|— interrupted: AtomicBoolean ← 防止双重中断
|
|— saveToSession() [LINE 58-68]:
|   |— agent.saveTo(session, sessionKey) ← 保存 Agent 全量状态
|   |— session.save(shutdown_interrupted, true) ← 标记中断
|
|— interruptForShutdown() [LINE 74]:
|   |— if (!interrupted.getAndSet(true)):
|       agent.interrupt(InterruptSource.SYSTEM) ← 发送系统中断
```

## 11.9 与 K8s 的配合

在 Kubernetes 环境中，优雅关闭与 Pod 生命周期配合：

```
# Pod spec 配置
spec:
  terminationGracePeriodSeconds: 120 # K8s 等待时间
  containers:
  - name: agent
    lifecycle:
      preStop:
        exec:
          command: ["/bin/sh", "-c", "sleep 10"] # 给 K8s 时间更新端点
```

配合 `GracefulShutdownConfig`：

```
GracefulShutdownConfig.builder()
    .shutdownTimeout(Duration.ofSeconds(100)) // 小于
    terminationGracePeriodSeconds
```

```
.partialReasoningPolicy(PartialReasoningPolicy.SAVE)
.build();
```

**\*\*设计评价：优雅关闭系统是框架\*\*生产级可用性的关键保障\*\*。三状态生命周期、安全点检查、超时强制中断、消息去重、部分推理策略——这些设计共同确保了 Agent 在面对进程终止时的数据安全和可恢复性。与 K8s 的自然配合使其适合云原生部署。但当前实现有一个限制：`sessionBindings` 使用 `agentId` 作为键，同一 Agent 实例只能绑定一个 Session，如果同一 Agent 被多个会话复用，会话恢复可能出现问题。**

---

#### ReActAgent

- └─ StructuredOutputCapableAgent ← 继承，提供结构化输出能力
  - └─ "generate\_response" 伪工具 + StructuredOutputHook
- └─ PlanNotebook ← 作为工具注册到 Toolkit，通过提示注入引导 Agent
  - └─ 10 个工具函数 + PlanToHint + 状态机
- └─ SkillBox ← 作为工具注册到 Toolkit，通过 SkillHook 注入提示
  - └─ ToolGroup 隔离 + 动态激活 + 代码执行
- └─ GracefulShutdownHook ← 作为系统 Hook (priority=0) 注册
  - └─ 安全点检查 + 消息去重 + 状态保存

#### 交叉点：

- StructuredOutputHook 通过 gotoReasoning() 与 ReAct 循环交互
- PlanNotebook 和 SkillBox 都使用 ToolGroup 机制控制工具可见性
- GracefulShutdownHook 的安全点恰好是 PlanNotebook/SkillBox Hook 的执行点
- 优雅关闭时，PlanNotebook 的状态通过 StateModule.saveTo() 被持久化
- SkillBox 的 deactivateAllSkills() 在 call() 开始时执行，与 GracefulShutdownHook 的 deduplicateIfResuming() 在 PreCallEvent 中协作

---

## 思考题

1. GracefulShutdown 的安全点检查为什么选择 PostEvent 而非 PreEvent?  
如果选择 PreEvent 会有什么问题?
2. PartialReasoningPolicy 的 SAVE 和 DISCARD 策略各适合什么场景? 如何自动选择?
3. 在 K8s 环境中, terminationGracePeriodSeconds 和 shutdownTimeout 的关系如何设计?

**\*\*参考: \*\***

1. 选择 PostEvent 因为它是操作完成后的安全点——Agent 已完成一轮推理, 状态一致。如果选 PreEvent: ①可能在操作中间打断, 导致半完成状态 (如工具调用已发出但结果未收到); ②LLM 正在流式输出时中断, 会产生截断的无效消息; ③状态保存不完整, 恢复后无法继续。

2. SAVE 适合: 长时推理任务 (如复杂代码生成), 中间推理结果有价值, 恢复后可继续。DISCARD 适合: 实时交互任务 (如客服对话), 部分结果已展示给用户, 恢复后应重新开始。自动选择策略: 根据已执行步数和 token 消耗——进度>50%选 SAVE (已投入成本高), 进度<30%选 DISCARD (重做成本低)。

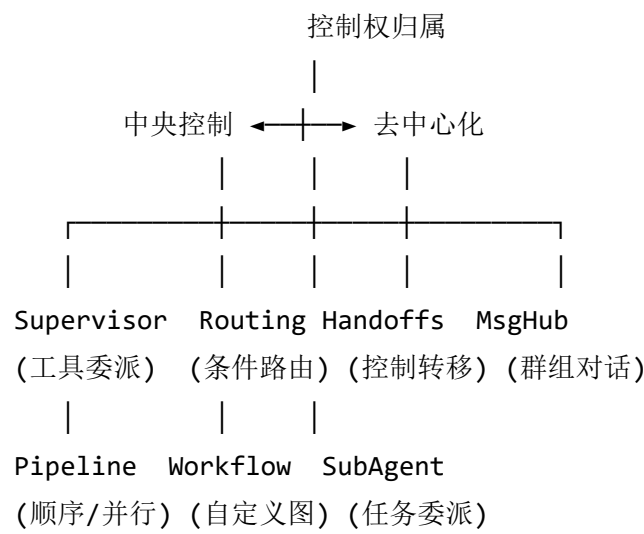
3. terminationGracePeriodSeconds 必须大于 shutdownTimeout+preStop 缓冲期。推荐值: shutdownTimeout=30s, preStop=15s, terminationGracePeriodSeconds=60s。关系: K8s 发送 SIGTERM→preStop hook 开始→应用进入 GracefulShutdown→shutdownTimeout 内完成在途请求→SIGKILL 强制终止。

# 第三篇：多 Agent 协作与分布式

## 第 12 章 Pipeline 编排与多 Agent 协作模式

### 12.1 模式全景图

AgentScope-Java 通过 `multiagent-patterns` 示例项目展示了 7 种多 Agent 协作模式。每种模式解决不同的编排需求，选择合适的模式是系统设计的关键决策。



## 12.2 Supervisor 模式——中央委派

**核心机制：**将子 Agent 注册为工具，Supervisor 的 LLM 通过 ReAct 循环自主决定调用哪个子 Agent。

```
Supervisor Agent
├── Toolkit
│   ├── schedule_event (SubAgentTool → CalendarAgent)
│   └── send_email (SubAgentTool → EmailAgent)
└── ReAct 循环:
    用户: "帮我安排明天下午 3 点跟张三开会，然后发邮件通知他"
    → 推理: 需要先安排会议，再发邮件
    → 调用 schedule_event 工具 → CalendarAgent 执行
    → 调用 send_email 工具 → EmailAgent 执行
    → 返回综合结果
```

**关键代码** (`supervisor/SupervisorConfig.java:109-122`):

```
@Bean("supervisorAgent")
public ReActAgent supervisorAgent(Model model,
    ReActAgent calendarAgent, ReActAgent emailAgent) {
    Toolkit toolkit = new Toolkit();
    toolkit.registration().subAgent(() -> calendarAgent).apply();
    toolkit.registration().subAgent(() -> emailAgent).apply();
    return ReActAgent.builder()
        .name("personal_assistant")
        .sysPrompt(SUPERVISOR_PROMPT)
        .model(model).toolkit(toolkit)
        .memory(new InMemoryMemory())
        .build();
}
```

**SubAgentTool 的工作原理：**

- `toolkit.registration().subAgent(provider)` 创建一个 `SubAgentTool`
- 子 Agent 的 `.name()` 成为工具名，`.description()` 成为工具描述
- LLM 看到的是一个普通工具，不知道背后是另一个 Agent
- 子 Agent 支持多轮会话（通过 `session_id` 参数）

- 调用时通过 `SubAgentProvider.getObject()` 获取新实例或复用已有实例

**适用场景：**

- 明确的功能分工（不同领域由不同 Agent 处理）
- 需要中央协调的复杂任务
- 子任务之间有依赖关系

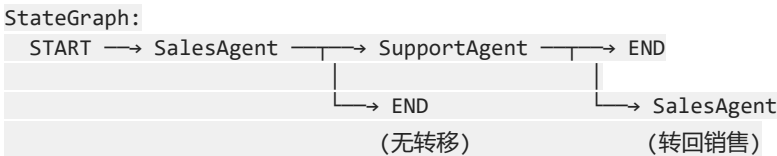
**优劣分析：**

- 优势：简单直觉，LLM 自主决策，无需硬编码路由逻辑
- 劣势：Supervisor 的 LLM 可能误选子 Agent；所有请求都经过 Supervisor，增加延迟和 token 消耗

---

## 12.3 Handoffs 模式——控制权转移

**核心机制：**使用 `StateGraph` + 条件边 + 转移工具。每个 Agent 可以通过调用特殊工具将对话控制权转交给另一个 Agent。



**控制权转移：**

```
SalesAgent 调用 TransferToSupportTool
→ ToolContextHelper.getStateForUpdate().put("active_agent",
"support_agent")
→ 条件边 RouteAfterSalesAction 检查 active_agent
→ 路由到 SupportAgent
```

### 转移工具 (handoffs/.../TransferToSalesTool.java:39-60):

```
@Tool(name = "transfer_to_sales", description = "Transfer to sales agent...")
public String transferToSales(
    @ToolParam(name = "reason", description = "Brief reason") String
    reason,
    ToolContext toolContext) {
    // 关键: 通过 ToolContext 修改图状态
    ToolContextHelper.getStateForUpdate(toolContext)
        .ifPresent(update -> update.put("active_agent", "sales_agent"));
    return "Transferred to sales agent. Reason: " + reason;
}
```

### StateGraph 构建 (handoffs/.../AgentScopeHandoffsConfig.java:128-176):

```
StateGraph graph = new StateGraph("agent_scope_handoffs", () -> {
    Map<String, KeyStrategy> strategies = new HashMap<>();
    strategies.put("messages", new AppendStrategy(false)); // 消息追加
    strategies.put("active_agent", new ReplaceStrategy()); // 活跃代理替换
    return strategies;
});

graph.addNode("sales_agent", salesAgent.asNode());
graph.addNode("support_agent", supportAgent.asNode());

// 初始路由
graph.addConditionalEdges(START, new RouteInitialAction(), Map.of(
    "sales_agent", "sales_agent",
    "support_agent", "support_agent"));

// Sales 后的路由
graph.addConditionalEdges("sales_agent", new RouteAfterSalesAction(),
    Map.of(
        "support_agent", "support_agent",
        "__end__", END));

// Support 后的路由
graph.addConditionalEdges("support_agent", new RouteAfterSupportAction(),
    Map.of(
        "sales_agent", "sales_agent",
        "__end__", END));
```

### 条件边逻辑 (handoffs/.../RouteAfterSalesAction.java:32-40):

```
public CompletableFuture<Command> apply(OverAllState state, RunnableConfig
    config) {
    String active =
        state.value("active_agent").map(Object::toString).orElse("");
    String target = "support_agent".equals(active) ? "support_agent" :
```

```
"__end__";  
    return CompletableFuture.completedFuture(new Command(target));  
}
```

### 适用场景：

- 不同专业领域的 Agent 需要无缝切换
- 对话上下文需要在 Agent 间保持连续
- Agent 可以自主决定是否转交控制权

### 优劣分析：

- 优势：Agent 自主决定何时转移，灵活性强；StateGraph 提供可视化流程
  - 劣势：需要为每个 Agent 编写转移工具和条件边，代码量大；状态管理复杂
- 

## 12.4 Routing 模式——条件路由

**核心机制：**使用 `AgentScopeRoutingAgent` 或 `StateGraph` + 条件边。与

Handoffs 不同，Routing 的路由决策由一个专门的路由器完成，而非 Agent 自主转移。

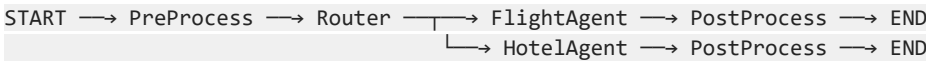
**简单路由** (`routing/.../simple/RoutingConfig.java:73-132`):

```
AgentScopeRoutingAgent routingAgent = AgentScopeRoutingAgent.builder()  
    .name("router")  
    .model(model)  
    .sysPrompt(ROUTER_PROMPT)  
    .addAgent(flightAgent)  
    .addAgent(hotelAgent)  
    .build();
```

路由器查看所有子 Agent 的 name 和 description，由 LLM 决定将请求路由到哪个 Agent。

**图路由** (`routing/.../graph/GraphRoutingConfig.java`): 将路由器嵌入

StateGraph, 支持预处理和后处理节点:



**适用场景:**

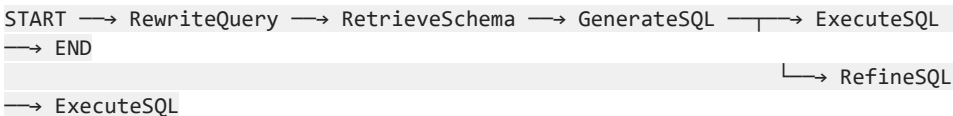
- 请求分类明确, 路由规则固定
- 需要在路由前后添加通用处理 (如日志、翻译)
- 不需要 Agent 间转移控制权

---

## 12.5 Workflow 模式——自定义 workflow

**核心机制:** 使用 `StateGraph` 定义完全自定义的节点和边, 实现复杂的业务流程。

**SQL Agent 工作流** (`workflow/` 子模块):



每个节点是一个 `AgentScopeAgent`, 边可以是条件边 (如 SQL 生成是否成功决定走执行还是优化路径) 。

**适用场景:**

- 有明确流程定义的业务场景
- 需要条件分支和循环

- 流程节点包含人工审核等非 LLM 步骤

## 12.6 Pipeline 模式——顺序/并行/循环

**核心机制：** `SequentialAgent`、`ParallelAgent`、`LoopAgent` 提供最简单的编排方式。

```
// 顺序执行
SequentialAgent seq = SequentialAgent.builder()
    .addAgents(researcher, writer, reviewer).build();

// 并行执行
ParallelAgent par = ParallelAgent.builder()
    .addAgents(translatorEN, translatorZH, translatorJA).build();

// 循环执行（带条件退出）
LoopAgent loop = LoopAgent.builder()
    .addAgents(coder, tester)
    .maxIterations(5).build();
```

**适用场景：** 流水线式的任务处理，无需复杂的条件逻辑。

## 12.7 模式选型决策矩阵

| 模式         | 控制权      | 灵活性 | 复杂度 | 典型场景        |
|------------|----------|-----|-----|-------------|
| Supervisor | 中央 Agent | 中   | 低   | 多领域个人助手     |
| Handoffs   | Agent 自主 | 高   | 高   | 客服（销售↔技术支持） |

|          |      |   |    |                  |
|----------|------|---|----|------------------|
| Routing  | 路由器  | 中 | 低  | 请求分类分发           |
| Workflow | 图定义  | 高 | 高  | SQL 生成+执行<br>+修复 |
| Pipeline | 固定   | 低 | 最低 | 翻译→审校→发<br>布     |
| MsgHub   | 去中心化 | 高 | 中  | 群组讨论/游戏          |

12.7.1 失败传播——选择协作模式时最容易被忽视的维度

不同模式下，子 Agent 失败的传播方式截然不同，这直接影响系统的可靠性设计：

| 模式         | 子 Agent 失败行为   | 影响范围     | 恢复策略                     |
|------------|----------------|----------|--------------------------|
| Supervisor | 子 Agent 返回错误结果 | 仅当前任务    | Supervisor 可重试或切换子 Agent |
| Handoffs   | 控制权已转移，无法回退    | 整个对话     | 需要设计"归还"机制               |
| Routing    | 路由到的 Agent 失败  | 仅当前分支    | 路由器可重新路由                 |
| Workflow   | 节点失败，图执行中断     | 整个工作流    | 需要补偿事务或回滚                |
| Pipeline   | 某个步骤失败         | 后续步骤全部阻塞 | 跳过/重试/降级                 |
| MsgHub     | 某个参与者无响应       | 取决于角色    | 超时剔除或默认值                 |

**关键洞察：**Handoffs 模式的失败传播最危险——控制权已转移，如果接收方 Agent 崩溃或产出无效结果，Supervisor 没有自然的"回收"机制。实践中的解

决方案是在 Handoffs 的 transfer 工具中增加超时和回退逻辑，但框架层面未提供标准化的失败恢复路径。

### 12.7.2 死锁风险

Handoffs 模式存在循环转移的风险：Agent A 转移给 Agent B，Agent B 又转移给 Agent A。AgentScope-Java 通过 StateGraph 的有限状态机设计部分缓解了这个问题——每次转移是一个确定的状态转换，无法回到之前的状态。但开发者仍需注意避免设计"乒乓"式的转移逻辑。

*\*\*设计评价：\*\* 多 Agent 协作模式的多样性是 AgentScope-Java 的差异化优势。选择哪种模式取决于业务场景的控制权需求、协作复杂度、失败容忍度和可调试性要求。没有"最好的"模式，只有"最合适的"模式。最重要的选型维度往往不是"功能是否能实现"，而是"失败时如何恢复"。*

---

#### 思考题

1. Supervisor 模式和 Handoffs 模式在用户体验上有什么差异？哪种更适合客服场景？
2. 如果需要一个"拍卖"场景（多 Agent 竞价），应该选择哪种协作模式？为什么？
3. MsgHub 的 autoBroadcast 机制在什么场景下需要关闭？关闭后如何保证信息传递？

*\*\*参考：\*\**

1. Supervisor: 用户始终与中心 Agent 对话，子 Agent 对用户透明——适合

客服场景（用户不需要知道转接给了谁）。Handoffs：用户感知到 Agent 切换，上下文传递更自然——适合专家咨询（用户想和专业 Agent 直接沟通）。客服场景选 Supervisor 更优：统一入口、统一语气、避免用户困惑。

2. 拍卖场景选择 MsgHub 广播模式：①所有竞价 Agent 同时收到拍卖信息（autoBroadcast=true）；②竞价 Agent 独立出价（autoBroadcast=false，秘密竞价）；③拍卖师 Agent 汇总结果。为什么不用 Supervisor：竞价需要并发而非串行；为什么不用 Handoffs：竞价 Agent 间无需切换。

3. 需要关闭 autoBroadcast 的场景：①投票/竞价——参与者不能看到他人选择（如 Werewolf 投票）；②敏感信息讨论——部分 Agent 间不应互相看到消息（如谈判双方）。关闭后信息传递方式：①通过 SupervisorAgent 中转；②指定 target 参数单播；③使用 Memory 共享结果（最终统一汇总）。

---

# 第 13 章 A2A 协议——Agent 互联网

## 13.1 为什么需要 Agent 间通信协议

当我们从单 Agent 走向多 Agent 系统，一个根本性问题浮现：**不同进程、不同机器、甚至不同团队开发的 Agent 如何互操作？**

传统方案各有局限：

- **共享数据库**：耦合太强，无法跨组织协作
- **自定义 HTTP API**：每个团队定义自己的接口，无法互操作
- **消息队列**：传输层解决了，但语义层（Agent 如何描述自己、如何协商能力）没有标准

A2A (Agent-to-Agent) 协议应运而生——它定义了 Agent 间通信的标准化协议，包括身份描述（AgentCard）、请求路由（JSON-RPC）、流式传输（SSE）和传输层扩展（SPI）。

AgentScope-Java 实现了完整的 A2A 协议栈，核心组件：**A2aAgent**（客户端代理）+ **AgentScopeA2aServer**（服务端组装器）+ **TransportWrapper**（SPI 可插拔传输层，含 JSON-RPC over HTTP 和 RocketMQ 两种实现）。

```
graph TB
    subgraph 客户端
        A2aAgent -->|"1. 解析 AgentCard"| CardResolver
        A2aAgent -->|"2. 构建传输层"| ClientBuilder
        A2aAgent -->|"3. 发送消息"| A2aClient
    end
```

```
subgraph 服务端
    Server -->|"SPI 加载"| TransportBuilders
    Server -->|"组装"| RequestHandler
    Server -->|"执行"| AgentExecutor
end
A2AClient -->|"JSON-RPC / RocketMQ"| TransportBuilders
TransportBuilders --> RequestHandler
RequestHandler --> AgentExecutor
AgentExecutor -->|"AgentRunner.stream()"| Agent 实例
```

## 13.2 AgentCard——Agent 的数字名片

AgentCard 是 A2A 协议的核心概念，它是 Agent 的"名片"，让其他 Agent 知道你是谁、能做什么、如何联系你。

### AgentCard 核心字段

| 字段                 | 类型          | 说明               |
|--------------------|-------------|------------------|
| name               | String      | Agent 唯一标识名      |
| description        | String      | 能力描述             |
| url                | String      | 服务端点地址           |
| provider           | Provider    | 提供者信息（组织、URL）    |
| version            | String      | 协议版本，固定`"0.3.0"` |
| skills             | List<Skill> | Agent 声明的技能列表    |
| securitySchemes    | Map         | 支持的安全认证方案        |
| preferredTransport | String      | 首选传输协议类型         |

### ConfigurableAgentCard vs AgentCard

`ConfigurableAgentCard` 是用户配置阶段的原始卡片，包含占位符和可选字段。

`AgentScopeAgentCardConverter` 负责将配置卡片转换为最终可发布的

`AgentCard`：

- 注入实际传输端点信息（从 `TransportProperties` 获取 URL、端口等）
- 合并 Agent 运行时能力（从 `AgentRunner` 提取工具列表映射为 Skills）
- 固定 `protocolVersion = "0.3.0"`

### 三种 AgentCard 解析策略

A2aAgent 在构建时需要获取远程 Agent 的 AgentCard，框架提供了三种解析器：

#### 1. FixedAgentCardResolver

(`a2a/client/agent/card/FixedAgentCardResolver.java:24`)

最简单的实现——直接包装一个已构造好的 AgentCard，原样返回。适用于测试或已知远程 Agent 信息的场景：

```
// 构建时传入已知的 AgentCard
A2aAgent.builder()
    .name("remote-calculator")
    .agentCard(knownCard) // 自动包装为 FixedAgentCardResolver
    .build();
```

#### 2. WellKnownAgentCardResolver

(`a2a/client/agent/card/WellKnownAgentCardResolver.java:37`)

遵循 A2A 规范的 Well-Known URI 模式：`baseUrl + /.well-known/agent-card.json`。构造时传入 `baseUrl`、`relativeCardPath`（默认为规范路径）和可选的 `authHeaders`。解析时执行 HTTP GET 请求获取卡片。

### 3. NacosAgentCardResolver

(`nacos/a2a/discovery/NacosAgentCardResolver.java:59`)

基于 Nacos 的动态服务发现。维护 `ConcurrentHashMap<String, AgentCard>` 缓存，首次调用时通过 `aiService.subscribeAgentCard()` 同时获取卡片并注册监听器。当远程 Agent 在 Nacos 中更新卡片时，内部 `AgentCardUpdater` 自动刷新缓存——实现了 Agent 能力的热更新。

*\*\*设计洞察\*\*：三种解析器覆盖了从静态到动态的完整场景。Nacos 解析器的缓存+监听模式尤其值得关注——它让 Agent 的能力描述可以随部署动态变化，而客户端无需重启。*

## 13.3 A2aAgent——客户端代理的完整生命周期

A2aAgent 是 AgentBase 的子类，代表一个远程 Agent。它将本地 Agent 调用转换为 A2A 协议请求。

### 构造与 Builder 模式

A2aAgent 只能通过 Builder 创建 (`a2a/client/agent/A2aAgent.java:261`)。

关键设计决策：

- `agentCard(AgentCard)` 自动包装为 `FixedAgentCardResolver`
- `agentCardResolver(AgentCardResolver)` 允许注入自定义解析策略
- 两者都不设置时，`build()` 抛出 `IllegalArgumentException`
- Agent 的 `description` 在构建时从 `AgentCard` 提取（失败时静默返回 `null`）

## 请求执行流程

```
sequenceDiagram
    participant Caller as 调用方
    participant A2a as A2aAgent
    participant Hook as A2aClientLifecycleHook
    participant Client as A2A Client
    participant Remote as 远程 Agent

    Caller->>A2a: call(msg)
    A2a->>Hook: PreCallEvent
    Hook->>Hook: 生成 requestId(UUID)
    Hook->>Client: buildA2aClient()
    Note over Hook: 每次调用创建新的 Client
    A2a->>A2a: memory.addMessage(msg)
    A2a->>Client: sendMessage(message, handlers)
    Client->>Remote: JSON-RPC / RocketMQ
    Remote-->>Client: TaskEvent / MessageEvent
    Client->>Hook: ClientEventHandlerRouter
    Hook->>A2a: sink.success(resultMsg)
    A2a->>A2a: memory.addMessage(resultMsg)
    A2a->>Hook: PostCallEvent
    Hook->>Hook: tryReleaseResource()
```

核心设计点：

1. **每次调用创建新 Client**： `A2aClientLifecycleHook`（优先级 500）在 `PreCallEvent` 中生成新的 `requestId`（UUID）并构建全新 A2A Client，在 `PostCallEvent` / `ErrorEvent` 中释放资源。这确保了 HTTP 连接不会泄漏，但也意味着每次调用都有连接建立开销。
2. **消息转换桥梁**： `MessageConvertUtil.convertFromMsg()` 将本地 `Msg` 转换为 A2A 协议的 `Message` 格式，反过来 `MessageEventHandler` 将远程 `Message` 转回 `Msg`。
3. **Reactor Sink 桥接**： `doExecute()` 使用 `Mono.create(sink -> {...})` 将回调式的 A2A Client API 桥接到响应式的 `Mono<Msg>` 返回类型。最终

`MessageEventHandler` 或 `TaskUpdateEventHandler` 调用 `sink.success(msg)`

完成 `Mono`。

## 客户端事件路由

`ClientEventHandlerRouter` 维护三种事件处理器：

| 事件类型            | 处理器                    | 行为                                                |
|-----------------|------------------------|---------------------------------------------------|
| TaskEvent       | TaskEventHandler       | 存储任务对象到上下文（用于后续取消）                                |
| MessageEvent    | MessageEventHandler    | 转换为 <code>Msg</code> ，完成 <code>Mono</code> （终端事件） |
| TaskUpdateEvent | TaskUpdateEventHandler | 拆分为状态更新和产物更新两个子处理器                                |

`TaskUpdateEventHandler` 是最复杂的：

- **TaskStatusUpdateEventHandler**：如果 `isFinal=true`，收集所有 `task artifacts` 转 `Msg` 并完成 `sink`；否则通过 `Hook` 发射 `ReasoningChunkEvent` 实现流式进度展示
- **TaskArtifactUpdateEventHandler**：每个 `artifact` 转 `Msg` 并发射 `ReasoningChunkEvent`，支持增量输出

## 任务取消

`A2aAgent` 支持任务中断。当 `interrupt()` 被调用时，它通过 `a2aClient.cancelTask()` 发送 `CancelTask` 请求，携带当前 `taskId`（从 `TaskEventHandler` 存储的上下文中获取）。

## 13.4 AgentScopeA2aServer——服务端组装器

**关键认知：** `AgentScopeA2aServer` 不是网络监听器。正如源码注释所言

(`a2a/server/AgentScopeA2aServer.java:68`) : "The Server is not listen ports and export endpoints. The main logic for this Server class is build and assemble all component for handle a2a request."

它是一个**组装器**，负责将 Agent 能力、请求处理器和传输层组合在一起。

### 组装流程 (`Builder.build()`)

1. **创建 AgentExecutor:** `AgentScopeAgentExecutor` 包装 `AgentRunner`，实现 A2A 的 `AgentExecutor` 接口
2. **创建 RequestHandler:** `AgentScopeA2aRequestHandler` 包装规范库的 `DefaultRequestHandler`，配置 `TaskStore`、`QueueManager`、`PushNotification` 等默认内存实现
3. **SPI 加载传输构建器:** `loadTransportBuilders()` 使用 `ServiceLoader.load(TransportWrapperBuilder.class)` 发现 classpath 上的所有传输实现
4. **过滤可用传输:** 只有同时拥有配置 (`TransportProperties`) 和 SPI 实现的传输才可用
5. **构建 AgentCard:** `AgentScopeAgentCardConverter` 合并配置、传输端点和 Agent 能力
6. **构建 TransportWrapper:** 为每个可用传输调用 `builder.build(agentCard, requestHandler, executor)`

## 服务端执行链

```
A2A 请求 → TransportWrapper.handleRequest()
          → RequestHandler.onSendTask() / onSendTaskStreaming()
          → AgentScopeAgentExecutor.execute()
          → AgentRunner.stream() (创建新 Agent 实例)
          → 阻塞/流式返回
```

注意：每次 A2A 请求都会通过 `AgentRunner.stream()` 创建一个全新的 Agent 实例。这意味着 Agent 之间无状态共享，但也带来了实例化开销。

## 13.5 JSON-RPC 传输层

`JsonRpcTransportWrapper` 是默认传输实现，基于 JSON-RPC 2.0 over HTTP，支持 SSE 流式推送。

### 请求类型映射

| A2A 方法               | RPC 类型 | 返回类型                 |
|----------------------|--------|----------------------|
| GetTask              | 请求-响应  | 单次 JSON-RPC Response |
| SendMessage          | 请求-响应  | 单次 JSON-RPC Response |
| SendStreamingMessage | 请求-流   | SSE 事件流              |
| CancelTask           | 请求-响应  | 单次 JSON-RPC Response |
| TaskResubscription   | 请求-流   | SSE 事件流              |

### 错误码映射

JSON-RPC 错误码与 A2A 语义错误的映射：

```
JsonParseException → JSONParseError (-32700)
MethodNotFoundError → MethodNotFoundError (-32601)
InvalidParamsError → InvalidParamsError (-32602)
```

```
TaskNotFoundError → TaskNotFoundError (-32001)
AgentCapabilityError → AgentCapabilityError (-32003)
```

## SSE 流式推送

`FluxSseSupport` 将 Reactor `Flux` 事件转为 SSE 格式推送。对于

`SendStreamingMessage`，服务端将 Agent 的流式输出逐条转为 SSE 事件发送，客户端的 `TaskUpdateEventHandler` 逐步消费。

## 13.6 RocketMQ 传输层

RocketMQ 传输层复用 `JsonRpcTransportWrapper` 的内部处理逻辑，通过消息队列替代 HTTP 直连。

### 架构设计

```
Producer → Topic(a2a-request-{agentName}) → Consumer(服务端)
服务端处理 → Topic(a2a-response-{requestId}) → Producer → Consumer(客户端)
```

### 关键实现细节

1. **Topic 命名**：请求主题按 Agent 名称隔离（`a2a-request-{agentName}`），响应主题按请求 ID 隔离
2. **CompletableFuture 超时**：使用 15 分钟超时的 `CompletableFuture` 跟踪请求完成状态。超时后自动取消任务
3. **FluxSseSupport 复用**：流式场景下，`Flux` 事件同样通过 `FluxSseSupport` 转为 SSE 格式，通过 RocketMQ 消息体传递
4. **无状态设计**：每个请求独立的响应 Topic + `CompletableFuture`，天然支持并发

**\*\*设计权衡\*\***: RocketMQ 传输牺牲了低延迟（消息队列引入约 5-50ms 延迟），换来了更好的可靠性（消息持久化、重试机制）和跨网络穿透能力（无需暴露 HTTP 端口）。适合跨数据中心、跨组织边界的 Agent 协作场景。

## 13.7 服务发现与注册

### Well-Known URI 模式

A2A 规范定义的服务发现方式: `baseUrl + /.well-known/agent-card.json`。客户端通过 HTTP GET 获取远程 Agent 的 AgentCard，无需任何注册中心。

### NacosA2aRegistry

`NacosA2aRegistry` 基于 Nacos 实现动态服务发现：

1. **注册**：将 AgentCard 和传输端点信息注册到 Nacos
2. **发现**：通过服务名查询 Nacos 获取可用的 Agent 列表
3. **订阅**：监听 AgentCard 变更事件，实现能力热更新

## 13.8 SPI 扩展——自定义传输层

A2A 传输层通过 Java SPI 完全可插拔：

1. 实现 `TransportWrapperBuilder` 接口
2. 在 `META-INF/services/io.agentscope.core.a2a.transport.TransportWrapperBuilder` 中注册
3. 配置 `TransportProperties` 指定传输类型和参数

#### 4. `AgentScopeA2aServer.build()` 自动发现并加载

这意味着你可以为 gRPC、WebSocket、Kafka 等任何传输协议编写 A2A 适配器，而无需修改框架核心代码。

**设计评价：** A2A 协议是框架最具前瞻性的设计——将 Agent 视为网络服务，使不同团队、不同语言开发的 Agent 可以互操作。三层架构（AgentCard 身份层 + JSON-RPC 语义层 + SPI 传输层）的解耦非常干净。但存在两个不足：一是每次请求创建新 Agent 实例开销较大，未来可引入实例池化；二是安全认证层（securitySchemes）尚处于声明阶段，未与实际鉴权逻辑打通。

### 思考题

1. A2aAgent 每次调用都创建新的 A2A Client，这种设计在什么并发量级下会成为瓶颈？如何改进？
2. 对比 JSON-RPC 和 RocketMQ 两种传输层，在什么业务场景下应选择 RocketMQ？延迟和可靠性如何权衡？
3. AgentCard 的 NacosAgentCardResolver 使用了缓存+监听模式。如果 Nacos 集群发生分区，可能出现什么一致性问题？

**参考：**

1. 瓶颈在 HTTP 连接建立开销。单次 A2A Client 创建约需 50-100ms（TCP+TLS+HTTP/2 协商），100+ QPS 时连接池耗尽。改进：①引入 A2A Client 连接池，复用长连接；②HTTP/2 多路复用，一个连接承载多个请求；③预热——应用启动时预创建 Client 放入池中。预计池化后可支撑 1000+ QPS。
2. 选择 RocketMQ 的场景：①跨数据中心——消息队列天然支持网络穿透，

无需暴露 HTTP 端口；②异步场景——Agent 间不需要实时响应（如日志分析、报告生成）；③需要消息持久化和重试——任务不能丢失。延迟权衡：JSON-RPC 约 10-50ms，RocketMQ 约 50-200ms，但后者提供至少一次投递保证。

3. Nacos 分区一致性问题经典 CAP 困境——分区期间，不同分区的 NacosAgentCardResolver 缓存可能指向不同版本的 AgentCard。具体表现：①Agent A 的卡片在分区 1 更新了 skills，分区 2 仍缓存旧版；②客户端调用了已下线的 Agent（卡片指向过时地址）。缓解：客户端增加 TTL 过期检查，强制定期刷新。

# 第 14 章 RAG 检索增强——让 Agent 拥有知识

## 14.1 RAG 原理与在 Agent 中的定位

大语言模型的训练数据有截止日期，且无法访问企业私有数据。RAG

(Retrieval-Augmented Generation) 通过"先检索、再生成"的方式弥补这一缺陷：



### 检索的理论基础——从关键词到语义

RAG 的检索质量取决于三个环节：**Embedding 模型** → **向量索引** → **分块策略**。

**Embedding 模型**将文本映射为高维向量（通常 768-1536 维），语义相近的文本在向量空间中距离更近。当前主流选择：

| 模型                              | 维度   | 中文支持 | 适用场景      |
|---------------------------------|------|------|-----------|
| text-embedding-3-small (OpenAI) | 1536 | 一般   | 英文为主、成本敏感 |
| bge-large-zh-v1.5               | 1024 | 优秀   | 中文场景首选    |
| multilingual-e5-large           | 1024 | 优秀   | 多语言混合     |
| DashScope text-embedding        | 1536 | 优秀   | 阿里云生态集成   |

**向量相似度**衡量两个向量的"接近程度"，常用余弦相似度（范围 -1 到 1）或点积。检索时计算 query 向量与所有文档向量的相似度，返回 Top-K 最相似的文档。

**分块策略**决定文档如何被切分为可检索单元。过大的块（>1000 token）会导致检索不精确——块中包含多个主题，相似度被稀释；过小的块（<100 token）会丢失上下文——检索到的片段信息不足。常见策略：

| 策略   | 原理                  | 适用场景   |
|------|---------------------|--------|
| 固定长度 | 按 token 数切割         | 通用、简单  |
| 句子边界 | 在句号/换行处切割           | 保持语义完整 |
| 语义分块 | 用 Embedding 检测语义转折点 | 高精度需求  |
| 递归分块 | 先大块，再按分隔符细分         | 结构化文档  |

### 精确率 vs. 召回率的权衡

RAG 检索本质上是信息检索问题，存在经典的精确率-召回率权衡：

- **高精确率**（只返回最相关的文档）：减少 LLM 上下文中的噪音，但可能遗漏关键信息
- **高召回率**（返回更多可能相关的文档）：确保不遗漏，但增加 LLM 处理的噪音和 token 消耗

AgentScope-Java 的 `RetrieveConfig` 通过 `topK` 和 `scoreThreshold` 参数控制这个权衡——`topK=5`, `scoreThreshold=0.7` 偏向精确率, `topK=20`, `scoreThreshold=0.3` 偏向召回率。

在 Agent 场景中, RAG 的价值不仅是"回答问题"——它是 Agent **感知外部世界**的关键通道。一个拥有 RAG 的 Agent 可以:

- 查询实时产品信息 (产品库)
- 检索历史客户案例 (案例库)
- 获取最新行业动态 (新闻库)

## 14.2 Knowledge 接口与实现体系

AgentScope-Java 的 RAG 系统以 `Knowledge` 接口为核心 (`agentscope-core/.../rag/Knowledge.java:40-53`) :

```
public interface Knowledge {
    Mono<Void> addDocuments(List<Document> documents);
    Mono<List<Document>> retrieve(String query, RetrieveConfig config);
}
```

两个方法, 语义清晰: 存入文档、检索文档。所有实现都必须遵循这个契约。

**5 种 Knowledge 实现:**

| 实现                  | 依赖       | 特点                 |
|---------------------|----------|--------------------|
| `SimpleKnowledge`   | 内置       | 本地向量检索，适合开发和测试     |
| `BailianKnowledge`  | 阿里云百炼    | 云端 RAG，支持多轮查询改写    |
| `RAGFlowKnowledge`  | RAGFlow  | 开源 RAG 引擎，支持深度文档解析 |
| `HayStackKnowledge` | HayStack | 深度学习 RAG 框架，灵活管道   |
| `DifyKnowledge`     | Dify     | LLM 应用开发平台内置 RAG   |

**多知识库聚合：** `ReActAgent.Builder` 支持传入多个 `Knowledge`，内部通过

`buildAggregatedKnowledge()` 创建匿名聚合 `Knowledge`

(`ReActAgent.java:1638-1679`) :

- `addDocuments`: 扇出到所有 KB
- `retrieve`: 并行检索所有 KB，按 ID 去重（保留高分），按 score 降序排列，截取 limit

### 14.3 GenericRAGHook——自动检索注入

`GenericRAGHook` (`agentscope-core/.../rag/GenericRAGHook.java:65-228`) 是

RAG 的"自动模式"，通过 Hook 机制在每次推理前自动检索并注入知识：

**执行流程：**

PreCallEvent 触发

1. `extractQueryFromMessages()` (LINE 135)  
反向遍历消息列表，找到最后一个 USER 消息作为查询
2. `knowledge.retrieve(query, config)` (LINE 141)  
从知识库检索相关文档
3. `createEnhancedMessages(retrievedDocs)` (LINE 150)  
格式化检索结果 → 追加为新的 USER 消息
4. `event.setInputMessages(enhancedMessages)` (LINE 153)  
修改事件的消息列表，后续推理使用增强后的消息

**知识注入格式** (LINE 215-228) :

```
<retrieved_knowledge>Use the following content from the knowledge base(s) if  
it is helpful:  
  
- Score: 0.950, Content: 员工年假政策: 工作满1年可享受5天年假...  
- Score: 0.823, Content: 年假申请流程: 通过 OA 系统提交申请...  
</retrieved_knowledge>
```

使用 XML 标签包裹，每条结果附带相关性分数，LLM 可以据此判断是否采用。

**优先级:** `priority()` 返回 50 (LINE 117) ，高优先级确保在其他 Hook 之前执行。

**容错:** `onErrorResume` (LINE 156) 捕获异常后仅记录日志，不中断 Agent 执行——即使 RAG 服务不可用，Agent 仍能正常工作。

# 14.4 KnowledgeRetrievalTools——Agent 自主检索

KnowledgeRetrievalTools (agentscope-core/.../rag/KnowledgeRetrievalTools.java:52-178) 将 RAG 暴露为一个工具，Agent 可以自主决定何时检索：

```
@Tool(name = "retrieve_knowledge",
      description = "Search the knowledge base for relevant information")
public String retrieveKnowledge(
    @ToolParam(name = "query") String query,
    @ToolParam(name = "limit") Integer limit,
    Agent agent) {
    // 从 Agent 的记忆中提取对话历史（多轮查询改写）
    List<Msg> history = extractHistory(agent);
    RetrieveConfig config = buildConfig(limit, history);
    return knowledge.retrieve(query, config)
        .map(this::formatDocumentsForTool).block();
}
```

## GENERIC vs AGENTIC 对比：

| 维度       | GENERIC（自动）            | AGENTIC（自主）           |
|----------|------------------------|-----------------------|
| 触发方式     | Hook 自动拦截 PreCallEvent | Agent 通过 ReAct 循环决定调用 |
| 检索时机     | 每次推理前                  | Agent 认为需要时           |
| Token 消耗 | 较高（每次都注入知识）            | 较低（按需检索）              |
| 推理延迟     | 增加（检索+注入）              | 仅在调用工具时增加             |
| 适用场景     | 知识密集型任务                | 偶尔需要知识的任务             |
| 控制粒度     | 粗（全自动）                 | 细（Agent 决定查询内容）       |

## 14.5 配置与使用

```
// 创建知识库
Knowledge knowledge = new SimpleKnowledge();

// 方式 1: GENERIC 模式 (自动注入)
ReActAgent agent = ReActAgent.builder()
    .knowledge(knowledge)
    .ragMode(RAGMode.GENERIC)           // 默认值
    .retrieveConfig(RetrieveConfig.builder()
        .limit(5)                       // 最多返回 5 条
        .scoreThreshold(0.5)           // 最低相关性分数
    ).build())
    .build();

// 方式 2: AGENTIC 模式 (Agent 自主决定)
ReActAgent agent = ReActAgent.builder()
    .knowledge(knowledge)
    .ragMode(RAGMode.AGENTIC)           // 注册为工具
    .build();
```

## 14.6 BailianKnowledge 云端 RAG

**BailianKnowledge** ([agentscope-extensions/.../BailianKnowledge.java:87-](#)

**294**) 集成阿里云百炼平台，支持多轮查询改写：

```
BailianKnowledge knowledge = BailianKnowledge.builder()
    .config(dashScopeConfig)
    .client(bailianClient)
    .indexId("your-index-id")
    .build();
```

**多轮查询改写：**当 Agent 进行多轮对话时，`convertToQueryHistory()` (LINE 227) 将对话历史转换为 `QueryHistoryEntry` 格式，百炼平台据此改写查询，使检索更精准。

# 14.7 RAG 调优指南

| 调优维度    | 策略                                      | 效果            |
|---------|-----------------------------------------|---------------|
| 分块策略    | 文档按段落分块（512-1024 token），重叠 50-100 token | 平衡上下文完整性和检索精度 |
| 检索数量    | `limit=3-5`（太少遗漏，太多噪声）                  | 最优信息密度        |
| 分数阈值    | `scoreThreshold=0.5-0.7`                | 过滤低相关结果       |
| 模式选择    | 知识密集用 GENERIC，偶尔用 AGENTIC               | 控制成本和延迟       |
| 多 KB 聚合 | 不同领域独立 KB，统一聚合                          | 检索质量优于混合大库    |
| 查询改写    | 利用对话历史（Bailian）                         | 多轮对话中检索更精准    |

## 思考题

- GENERIC 模式和 AGENTIC 模式在 token 消耗上的差异有多大？如何量化评估选择哪种模式？
- GenericRAGHook 的 onErrorResume 设计（RAG 不可用时降级为无知识推理）是否总是安全的？什么情况下不安全？
- 如果需要支持多语言知识库（中英混合检索），当前 Knowledge 接口需要哪些扩展？

**\*\*参考: \*\***

1. *GENERIC* 模式: 每轮自动检索+注入, 额外 token = 检索文档数 × 平均文档长度。*AGENTIC* 模式: LLM 自主决定检索时机, 额外 token = 检索次数 × 文档长度 + 每轮的"是否检索"决策 token。量化: 在知识密集场景 (>70%问题需要外部知识), *GENERIC* 更优 (省去决策开销); 在偶尔检索场景 (<30%), *AGENTIC* 更省 token。转折点约在 50%检索率附近。
  2. 不安全场景: ①医疗/法律等高风险领域——无知识推理可能产生专业错误建议; ②数据时效性关键场景——RAG 不可用意味着使用了过时信息; ③安全审查场景——RAG 可能包含安全策略, 跳过后 Agent 行为失控。改进: 提供 fallback 策略配置 (`THROW_ERROR` / `FALLBACK` / `REQUIRE_HUMAN`) 。
  3. 扩展点: ①Knowledge 接口增加 `language` 字段, 声明知识库语言; ②`query()`方法增加跨语言检索逻辑——先翻译 `query` 为目标语言再检索, 或使用多语言 *Embedding* 模型 (如 `multilingual-e5`) ; ③结果后处理——检索结果回译为 Agent 工作语言。框架层面: 引入 `LanguageRouter` 组件, 按语言分发到不同 Knowledge 实例。
-

# 第 15 章 Human-in-the-Loop——人机协作的艺术

## 15.1 HITL 的设计动机与场景

全自动 Agent 在生产环境中存在风险：LLM 可能做出错误决策、工具调用可能产生不可逆操作、敏感操作需要人工审批。HITL（Human-in-the-Loop）让人类在关键时刻介入 Agent 执行流程。

**HITL 设计模式分类：**根据人类介入的方式和时机，可以将 HITL 模式分为三类：

| 模式       | 介入方式            | 介入时机      | 框架实现                                  |
|----------|-----------------|-----------|---------------------------------------|
| **工具门控** | 暂停工具执行，等待人工输入   | 工具执行时     | `ToolSuspendException`<br>(15.2-15.5) |
| **推理审查** | 拦截推理结果，人工决定是否继续 | 推理完成后     | `Hook` 拦截 (15.6)                      |
| **人机协作** | 人类作为特殊"工具"参与推理  | Agent 请求时 | `HumanTool` (概念模式)                    |

工具门控是最精确的控制——只在特定工具的特定条件下挂起，其余流程不受影响。推理审查是更粗粒度的控制——拦截所有推理结果，适合全局安全策略。人机协作是最灵活的模式——Agent 可以主动向人类提问，获取信息或决策。本章重点介绍前两种模式，它们在框架中已有成熟实现。

### 典型 HITL 场景：

- 支付审批：Agent 生成支付指令 → 人工确认 → 执行（工具门控）
- 代码部署：Agent 准备部署脚本 → 人工审查 → 执行部署（推理审查）
- 数据删除：Agent 生成 DELETE SQL → 人工确认 → 执行（工具门控）
- 医疗诊断：Agent 提出诊断建议 → 医生确认 → 纳入病历（推理审查）

## 15.2 ToolSuspendException 机制

AgentScope-Java 的 HITL 核心机制是“工具挂起”——在工具执行过程中抛出

`ToolSuspendException`，Agent 暂停并等待人工输入。

### `ToolSuspendException` (`agentscope-`

`core/.../tool/ToolSuspendException.java:40-66`) :

```
public class ToolSuspendException extends RuntimeException {
    private final String reason;

    public ToolSuspendException() {
        this("Tool execution suspended");
    }

    public ToolSuspendException(String reason) {
        this.reason = reason;
    }

    public String getReason() { return reason; }
}
```

### 捕获机制 (`ToolExecutor.java:243-253`) :

```
.onErrorResume(ToolSuspendException.class, e -> {
    logger.debug("Tool '{}' suspended: {}", toolCall.getName(),
e.getReason());
    return Mono.just(ToolResultBlock.suspended(toolCall, e));
})
```

异常被捕获并转换为 `ToolResultBlock.suspended()`——一个带

`agentscope_suspended: true` 元数据的工具结果。

## 15.3 ReActAgent 中的挂起处理

`ReActAgent.acting()` (`ReActAgent.java:569-631`) 处理挂起结果：

```
// 分离成功和挂起的结果
List<...> successPairs = ...; // 未挂起的工具结果
List<...> pendingPairs = ...; // 挂起的工具结果

if (所有结果都挂起) {
    // 返回 SUSPENDED 状态, 包含挂起的工具调用信息
    return ActingResult.suspended(pendingPairs);
}

if (部分挂起) {
    // 成功的结果正常处理
    // 挂起的结果标记为待处理
    // 继续推理循环 (成功的结果可以作为上下文)
}
```

## 15.4 完整 HITL 流程

```
1. Agent 推理 → LLM 决定调用 approve_payment 工具
   |
2. approve_payment 工具内部:
   |   if (需要人工确认) {
   |       throw new ToolSuspendException("Payment requires human approval");
   |   }
   |
3. ToolExecutor 捕获异常 → ToolResultBlock.suspended()
   |
4. ReActAgent.acting() 检测到 SUSPENDED → 返回挂起状态
   |
5. Agent.call() 返回包含挂起信息的 Msg
```

6. 应用层展示给人工 → 人工确认/拒绝

7. 人工输入 → 构造 ToolResult 消息

```
Msg toolResult = Msg.builder()
    .role(Role.USER)
    .content(ToolResultBlock.of(toolUseId, "APPROVED"))
    .build();
```

8. 重新调用 Agent, 传入补充消息:

```
agent.call(List.of(originalUserMsg, suspendedAssistantMsg,
    toolResultMsg))
```

9. Agent 从中断点恢复, 继续推理

## 15.5 编写需要人工审批的工具

```
public class PaymentTools {
    private final ApprovalService approvalService;

    @Tool(name = "approve_payment", description = "Process payment after approval")
    public String approvePayment(
        @ToolParam(name = "amount") double amount,
        @ToolParam(name = "recipient") String recipient) {

        // 金额超过阈值需要人工确认
        if (amount > 10000) {
            throw new ToolSuspendException(
                "Payment of $" + amount + " to " + recipient +
                " requires manual approval (amount exceeds $10,000 limit)");
        }

        // 小额直接处理
        return paymentService.process(amount, recipient);
    }
}
```

## 15.6 安全审查 Hook

另一种 HITL 模式是通过 Hook 在推理后拦截, 而非工具层挂起:

```

public class SecurityReviewHook implements Hook {
    private final Set<String> sensitiveTools = Set.of("delete_data",
"execute_sql", "deploy");

    @Override
    public Mono<HookResult> onEvent(HookEvent event) {
        if (event instanceof PostReasoningEvent postEvent) {
            Msg reasoning = postEvent.getReasoningMessage();
            List<ToolUseBlock> toolCalls = reasoning.getToolUseBlocks();

            for (ToolUseBlock call : toolCalls) {
                if (sensitiveTools.contains(call.getName())) {
                    // 停止 Agent, 等待人工审查
                    postEvent.stopAgent(
                        "Sensitive tool '" + call.getName() + "' requires
review");
                }
            }
        }
        return Mono.just(HookResult.continue());
    }

    @Override
    public int priority() { return 10; } // 高优先级
}

```

两种模式的选择:

- **ToolSuspendException**: 在工具内部判断, 细粒度控制 (根据参数决定是否挂起)
- **Hook 拦截**: 在推理后统一拦截, 适合全局安全策略

---

## 思考题

1. ToolSuspendException 和 Hook 拦截两种 HITL 模式的适用场景分别是什么?
2. 如果一个工具调用需要多轮人工交互 (如审批流程), 当前的挂起-恢复机制是否足够?

### 3. 如何在 HITL 场景中防止 Agent 无限挂起？需要什么超时机制？

**\*\*参考：\*\***

1. *ToolSuspendException*: 适用于“工具本身需要审批”的场景（如支付工具、删除操作），工具开发者决定何时挂起。*Hook 拦截*: 适用于“业务策略需要审批”的场景（如特定条件下才拦截），运维/安全团队配置规则。前者是代码级控制，后者是配置级控制。实践中两者结合：关键工具用 *ToolSuspendException* 兜底，通用策略用 *Hook*。
  2. 不够——当前机制是单次挂起-恢复，不支持多轮交互。多轮审批场景：① 初审→补充材料→复审→终审，需要多次挂起恢复；② 审批被驳回后 Agent 需要修改参数重新提交。改进：在 *ToolResultBlock* 中增加 *conversationId*，支持同一工具调用的多次挂起；*Session* 中保存挂起上下文链，恢复时沿链回溯。
  3. 防无限挂起需要三级超时：① 工具级超时——单个 HITL 工具最长等待时间（如 24 小时），超时自动返回超时错误；② 会话级超时——整个 Agent 会话最长挂起时间（如 7 天），超时关闭会话；③ 业务级超时——通过 *Hook* 实现（如审批流程 SLA 48 小时）。超时后策略：*RETURN\_TIMEOUT* / *ESCALATE\_TO\_HUMAN* / *FORCE\_CONTINUE*。
-

# 第四篇：生产工程

---

## 第 16 章 状态持久化与容错

Agent 的执行过程可能持续数十分钟甚至数小时——一个复杂的推理任务可能包含 20+ 轮 ReAct 循环，涉及多次 LLM 调用和工具执行。如果进程崩溃、节点重启或需要弹性伸缩，Agent 必须能从断点恢复，而非从头开始。状态持久化解决了这个核心问题。

**为什么 Agent 需要状态持久化？** 三个驱动因素：(1) **崩溃恢复**——LLM 调用失败、工具超时、OOM 等异常在长时间运行中几乎必然发生；(2) **弹性伸缩**——K8s 环境下 Pod 可能被驱逐，Agent 需要能迁移到新节点继续执行；(3) **审计合规**——企业场景要求完整记录 Agent 的推理过程和决策依据。

**有状态 vs 无状态的权衡：**无状态 Agent（每次调用重建）简单但成本高昂——重复的 LLM 调用、重复的工具执行、重复的等待时间。有状态 Agent 虽然引入了持久化复杂度，但实现了“断点续传”，在长任务场景中节省 50-90% 的重复成本。AgentScope-Java 选择了有状态方案，并通过 `StateModule` + `Session` 的分层架构将持久化细节与业务逻辑解耦。

## 16.1 StateModule 接口

所有需要持久化的 Agent 组件实现 `StateModule` 接口：

```
public interface StateModule {
    Map<String, Object> getState();
    void setState(Map<String, Object> state);
}
```

`InMemoryMemory`、`PlanNotebook` 等核心组件都实现了此接口。Agent 的 `getState()` 方法会递归收集所有 `StateModule` 的状态。

## 16.2 Session 体系

`Session` 接口 (`agentscope-core/.../session/Session.java:53-145`) 定义了状态存储的契约：

```
public interface Session {
    void save(SessionKey sessionKey, String key, State value);
    void save(SessionKey sessionKey, String key, List<? extends State> values);
    <T extends State> Optional<T> get(SessionKey sessionKey, String key, Class<T> type);
    <T extends State> List<T> getList(SessionKey sessionKey, String key, Class<T> itemType);
    boolean exists(SessionKey sessionKey);
    void delete(SessionKey sessionKey);
    Set<SessionKey> listSessionKeys();
}
```

### 16.2.1 InMemorySession

最简单的实现，使用 `ConcurrentHashMap<SessionKey, Map<String, List<State>>>`。适合开发调试和单次会话场景。进程终止后数据丢失。

### 16.2.2 FileSystemSession (JsonSession)

将状态序列化为 JSON 文件存储在本地文件系统。每个 `SessionKey` 对应一个目录，每个 `key` 对应一个 JSON 文件。适合单机持久化，无需外部依赖。

### 16.2.3 RedisSession

支持三种 Redis 客户端：

- **JedisSession**：同步阻塞，简单直接
- **LettuceSession**：异步非阻塞，性能更好
- **RedissonSession**：功能丰富，支持分布式锁

数据结构：`Hash<SessionKey, Map<String, List<State>>>`，每个 state key 对应一个 Redis Hash field，值为 JSON 序列化。

适用场景：分布式部署、需要跨实例共享状态。

### 16.2.4 MysqlSession

最复杂的生产级实现 (`agentscope-extensions/.../MysqlSession.java:70-804`)：

**表结构：**

```
CREATE TABLE agent_session (  
    session_id VARCHAR(255),  
    state_key VARCHAR(255),  
    identifier VARCHAR(255),  
    state_data TEXT,  
    list_hash VARCHAR(64),  
    PRIMARY KEY (session_id, state_key, identifier)  
);
```

**增量写入优化** (`ListHashUtil`) :

`MysqlSession` 的关键优化是避免每次都全量重写状态列表:

```
// ListHashUtil.computeHash() 计算当前列表的哈希
String currentHash = ListHashUtil.computeHash(states);

// ListHashUtil.needsFullRewrite() 比较新旧哈希
if (ListHashUtil.needsFullRewrite(previousHash, currentHash)) {
    // 全量重写: 删除旧行 + 插入新行
} else {
    // 增量更新: 只更新变化的行
}
```

这对消息历史这种频繁追加的场景特别重要——大多数时候只需要 `INSERT` 新消息, 而不是 `DELETE + INSERT` 全部。

**输入验证:** `validateSessionId()`、`validateStateKey()`、`validateIdentifier()`

防止 SQL 注入。

## 16.3 Agent 状态保存与恢复

**保存** (`AgentBase.saveTo()`) :

```
agent.saveTo(mysqlSession, sessionId);
```

内部流程:

1. 调用 `agent.getState()` 收集所有 `StateModule` 的状态
2. 序列化为 `State` 对象
3. 调用 `session.save(sessionKey, key, state)`

**恢复** (`AgentBase.loadIfExists()`) :

```
agent.loadIfExists(mysqlSession, sessionId);
```

内部流程:

- 1. 调用 `session.get(sessionKey, key, State.class)`
- 2. 如果存在，反序列化并调用 `agent.setState(state)`
- 3. 如果不存在，保持初始状态

16.4 从优雅关闭中恢复

当 Agent 因 GracefulShutdown 被中断时，状态保存后返回

`shutdown_interrupted` 标志。客户端重试时：

```
Msg response = agent.call(userMsg).block();

if (response.getMetadata().containsKey("shutdown_interrupted")) {
    // Agent 被优雅关闭中断
    // 重新创建 Agent 并恢复状态
    ReActAgent newAgent = createAgent();
    newAgent.loadIfExists(mysqlSession, sessionId);

    // 去重：防止重复处理用户消息
    // GracefulShutdownHook.deduplicateIfResuming() 处理此逻辑

    response = newAgent.call(userMsg).block();
}
```

16.5 容错策略总结

| 故障类型       | 容错机制                    | 恢复策略                 |
|------------|-------------------------|----------------------|
| Agent 进程崩溃 | Session 持久化             | 从 Session 恢复状态       |
| 优雅关闭       | GracefulShutdown + 状态保存 | 客户端重试 + 去重           |
| 工具执行失败     | ToolResultBlock.error() | Agent 自动重试（ReAct 循环） |

|           |                                   |          |
|-----------|-----------------------------------|----------|
| LLM 调用超时  | ModelUtils.applyTimeoutAndRetry() | 自动重试     |
| RAG 服务不可用 | GenericRAGHook.onErrorResume      | 降级为无知识推理 |

### 思考题

1. MysqlSession 的增量写入优化 (ListHashUtil) 在什么场景下最有效？什么场景下退化为全量重写？
2. Session 接口缺少事务支持——多组件状态持久化不是原子操作。设计一个事务性 Session 方案。
3. 从 shutdown\_interrupted 恢复时，消息去重 (deduplicateIfResuming) 的局限性是什么？

**\*\*参考：\*\***

1. ListHashUtil 增量写入最有效的场景：消息列表增长型操作（如持续追加对话消息），只需写入新增部分。退化为全量重写的场景：①频繁修改历史消息（如编辑/删除中间消息）；②列表元素顺序变化；③Hash 冲突导致误判为变更——实际未变但全量重写。

2. 事务性 Session 方案：①引入 SessionTransaction 接口，begin()/commit()/rollback()；②Session.save()改为延迟写入——先写入事务缓冲区，commit 时原子性刷盘；③利用 MySQL 事务——将多个组件的 SQL 放入同一事务；④补偿事务——commit 失败时按逆序回滚已写入的组件。

3. 去重局限性：①只能基于消息 ID 去重，如果恢复后 Agent 生成了语义相同但 ID 不同的消息，无法去重；②时间窗口外的重复消息无法识别；③并发场景下，多 Agent 实例可能生成不同 ID 但相同内容的消息。根本解决：需要幂等键 (idempotency key) 而非仅消息 ID。

# 第 17 章 可观测性——全链路追踪与监控

可观测性是生产环境的“眼睛”——没有可观测性，Agent 的执行过程就是黑盒，问题发生时只能靠猜测。传统的应用可观测性基于三大支柱：**追踪**（Tracing，请求的端到端路径）、**指标**（Metrics，聚合的量化数据）和**日志**（Logging，离散的事件记录）。Agent 的可观测性在这三大支柱上有其独特需求：

- **追踪**需要覆盖 ReAct 循环的多次迭代（一个用户请求可能触发 5-20 轮 LLM 调用和工具执行），而非单次 API 调用
- **指标**需要包含 LLM 特有的维度（token 消耗、推理迭代次数、工具调用成功率），而非仅 CPU/内存
- **日志**需要记录推理过程中的关键决策点（为什么选择了这个工具？为什么终止了循环？），而非仅错误信息

AgentScope-Java 的可观测性设计以 Tracer 体系为核心，通过装饰器模式在 Agent/Model/Tool/Format 四个层次插入追踪逻辑，并支持 OpenTelemetry 标准和自定义实现。

## 17.1 Tracer 体系架构

AgentScope-Java 的追踪系统采用注册式设计：

```
graph TD
    TR["TracerRegistry 全局单例"]
    TR -->|register| Tracer["Tracer 实现"]
    TR -->|get| Current["当前 Tracer"]
    TR -->|enableTracingHook| Hook["Reactor 全局 Hook"]
    Hook -->|Hooks.onEachOperator| Lift["Operators.lift"]
    Lift -->|传播| Ctx["追踪上下文"]
```

**Tracer 接口** (`agentscope-core/.../tracing/Tracer.java:35-68`) 提供 5 个拦截点：

截点：

| 方法                              | 拦截点                              | 作用          |
|---------------------------------|----------------------------------|-------------|
| <code>`callAgent()`</code>      | <code>Agent.call()</code>        | 追踪 Agent 执行 |
| <code>`callModel()`</code>      | <code>Model.stream()</code>      | 追踪 LLM 调用   |
| <code>`callTool()`</code>       | <code>Toolkit.callTools()</code> | 追踪工具执行      |
| <code>`callFormat()`</code>     | <code>Formatter.format()</code>  | 追踪消息格式化     |
| <code>`runWithContext()`</code> | Reactor Context                  | 上下文传播       |

每个方法都采用装饰器模式——接收 `Supplier` 并在执行前后插入追踪逻辑：

```
default Mono<Msg> callAgent(AgentBase instance, List<Msg> inputMessages,
    Supplier<Mono<Msg>> agentCall) {
    return agentCall.get(); // 默认实现：直接透传
}
```

## 17.2 TelemetryTracer——OpenTelemetry 集成

`TelemetryTracer` (`agentscope-extensions-studio/.../TelemetryTracer.java:67-307`) 是 AgentScope-Java 内置的 OpenTelemetry 追踪实现：

**Span 结构：**

```

invoke_agent (Agent 执行)
├─ format (消息格式化)
├─ chat (LLM 调用)
│   └─ attributes: model, provider, input_tokens, output_tokens
│       └─ events: tool_choice, finish_reason
├─ execute_tool (工具执行)
│   └─ attributes: tool_name, tool_args, duration_ms
│       └─ events: tool_result
└─ invoke_agent (子 Agent 调用, 递归)

```

## 配置方式:

```

TelemetryTracer tracer = TelemetryTracer.builder()
    .enabled(true)
    .endpoint("http://localhost:4318/v1/traces")
    .addHeader("Authorization", "Bearer xxx")
    .build();

```

```

TracerRegistry.register(tracer);
TracerRegistry.enableTracingHook();

```

## 17.3 自定义 Tracer

```

public class MetricsTracer implements Tracer {
    private final MeterRegistry meterRegistry;

    @Override
    public Flux<ChatResponse> callModel(ChatModelBase instance,
        List<Msg> inputMessages, List<ToolSchema> toolSchemas,
        GenerateOptions options, Supplier<Flux<ChatResponse>> modelCall)
    {
        Timer.Sample sample = Timer.start(meterRegistry);
        return modelCall.get()
            .doOnComplete(() -> sample.stop(
                Timer.builder("agent.model.call")
                    .tag("model", instance.getModelName())
                    .register(meterRegistry)));
    }

    @Override
    public Mono<ToolResultBlock> callTool(Toolkit toolkit,
        ToolCallParam toolCallParam,
        Supplier<Mono<ToolResultBlock>> toolkitCall) {
        Counter counter = Counter.builder("agent.tool.calls")
            .tag("tool", toolCallParam.getToolName())
            .register(meterRegistry);
        return toolkitCall.get().doOnNext(result -> counter.increment());
    }
}

```

```
    }  
}
```

## 17.4 Hook 作为业务级监控

Hook 系统可以在不修改 Agent 代码的情况下添加监控逻辑：

```
public class MonitoringHook implements Hook {  
    @Override  
    public Mono<HookResult> onEvent(HookEvent event) {  
        if (event instanceof PostReasoningEvent postEvent) {  
            // 记录推理延迟  
            long duration = Duration.between(  
                postEvent.getStartTime(), Instant.now()).toMillis();  
            metrics.recordReasoningLatency(duration);  
        }  
  
        if (event instanceof PostActingEvent postEvent) {  
            // 记录工具调用审计  
            auditLog.record(postEvent.getToolName(),  
                postEvent.getArguments(), postEvent.getResult());  
        }  
  
        return Mono.just(HookResult.continue());  
    }  
}
```

## 17.5 关键指标

| 指标名                         | 类型      | 说明           |
|-----------------------------|---------|--------------|
| `agent.call.duration`       | Timer   | Agent 单次调用耗时 |
| `agent.model.call.duration` | Timer   | LLM 调用耗时     |
| `agent.model.tokens.input`  | Counter | 输入 token 数   |
| `agent.model.tokens.output` | Counter | 输出 token 数   |
| `agent.tool.calls`          | Counter | 工具调用次数       |

|                       |           |              |
|-----------------------|-----------|--------------|
| `agent.tool.duration` | Timer     | 工具执行耗时       |
| `agent.tool.errors`   | Counter   | 工具执行失败次数     |
| `agent.iterations`    | Histogram | ReAct 迭代次数分布 |

## 思考题

1. TelemetryTracer 为 Agent/Model/Tool/Format 四个阶段创建 Span——如果需要添加自定义 Span（如 RAG 检索），如何扩展？
2. 全局 Reactor Hook 对 JVM 中其他 Reactor 应用的影响如何评估和隔离？
3. 设计一个基于 Hook 的 Token 消耗计费系统，需要考虑哪些边界情况？

**\*\*参考： \*\***

1. 扩展方式：①实现 Tracer 接口，在自定义 Tracer 中创建新 Span 类型；②利用现有 Hook 机制——在 RAG 检索前后添加 Hook，Hook 内创建自定义 Span（通过 TracerRegistry 获取 Tracer 实例）；③更优雅的方式：TelemetryTracer 暴露 startSpan(operationName)公共方法，供任何业务组件调用。

2. 评估：在测试环境启用 Hook 后跑基准测试，对比 QPS 和 P99 延迟变化；隔离方案：①为 AgentScope 创建独立的 ReactorHooks 实例（需要框架改造）；②使用 Context.put()/get()替代全局 Hook 传递 traceId；③按 ClassLoader 隔离——AgentScope 使用独立 ClassLoader 加载 Reactor。

3. 边界情况：①流式输出的 token 计数——ReasoningChunkEvent 可能重复计数（同一内容多 Hook 触发）；②工具调用失败但已消耗 token——是否计入；③多模型调用（主模型+子 Agent 模型）的 token 归属；④HITL 挂起期间的 token——挂起前的消耗如何分摊；⑤缓存命中（如 Prompt Caching）的 token 折扣。方案：TokenEvent 统一事件模型，区分 prompt/completion/cached 三类 token。

# 第 18 章 Spring Boot 集成与微服务部署

AgentScope-Java 不是孤立运行的——在企业环境中，Agent 需要与配置中心、服务发现、数据库、消息队列等基础设施协同工作。Spring Boot 集成为 Agent 提供了成熟的依赖注入、配置外部化、健康检查和生命周期管理能力，使其从"可运行的示例"变为"可运维的服务"。

**集成设计原理：**Spring Boot 集成的核心挑战是 Agent 的生命周期与 Spring Bean 生命周期的对齐。Agent 不是无状态的 Service——它持有对话上下文、记忆状态、工具引用，这些资源的初始化顺序（模型 → 工具包 → Agent）和销毁顺序（Agent → 工具包 → 模型）必须严格控制。AgentScope-Java 通过 `@ConditionalOnClass` 按需加载和 `SmartLifecycle` 有序启停来解决这个问题。在 Bean 作用域上，Agent 通常采用 prototype 作用域（每次请求创建新实例），而 Model 和 Toolkit 采用 singleton 作用域（全局共享），这种设计既保证了 Agent 的会话隔离性，又实现了模型连接的复用。

## 18.1 自动配置

AgentScope-Java 提供 Spring Boot 自动配置，通过 `@ConditionalOnClass` 等条件注解按需加载：

**核心自动配置项：**

- **Model Bean**: 根据配置自动创建 DashScope/OpenAI/其他模型
- **Toolkit Bean**: 自动扫描 `@Tool` 注解并注册
- **AgentScopeProperties**: 统一配置属性

**初始化顺序**: Spring 容器启动时, 自动配置按 Model → Toolkit → Session → Tracer 的顺序创建 Bean。Agent 通过 `ObjectProvider<Toolkit>` 延迟获取工具包, 避免了循环依赖问题。

## 18.2 配置属性

```
agentscope:
  model:
    provider: dashscope          # LLM 提供商
    api-key: ${DASHSCOPE_API_KEY}
    model-name: qwen-plus
    temperature: 0.7
    max-tokens: 4096

  agent:
    max-iterations: 10          # ReAct 最大迭代次数
    auto-context:
      enabled: true
      token-ratio: 0.4
      last-keep: 10

  session:
    type: mysql                 # memory / filesystem / redis / mysql
    mysql:
      data-source: dataSource    # 引用 Spring DataSource

  tracing:
    enabled: true
    endpoint: http://localhost:4318/v1/traces
```

配置属性通过 `@ConfigurationProperties(prefix = "agentscope")` 绑定到

`AgentScopeProperties`, 支持 Spring Boot 的属性占位符 (`${...}`) 和类型安全校验。

## 18.3 多环境配置

```
agentscope:
  model:
    provider: dashscope
    model-name: qwen-turbo      # 开发用便宜模型
  session:
    type: memory               # 开发用内存 Session
agentscope:
  model:
    provider: dashscope
    model-name: qwen-max       # 生产用强模型
  session:
    type: mysql                # 生产用 MySQL Session
  tracing:
    enabled: true
    endpoint: http://jaeger:4318/v1/traces
```

多环境配置的核心原则：**开发环境优先速度和成本**（内存 Session + 便宜模型），**生产环境优先可靠性和可观测性**（MySQL Session + 强模型 + 全链路追踪）。

## 18.4 Docker 容器化

```
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY target/agent-app.jar app.jar
ENV JAVA_OPTS="-Xms512m -Xmx2g"
EXPOSE 8080
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

基础镜像选择 `eclipse-temurin:17-jre-alpine` 的考量：Alpine Linux 基于 musl libc，镜像体积仅 ~85MB（vs glibc 版 ~210MB），但需注意 musl 的 DNS 解析行为与 glibc 不同，在 K8s Service 发现中可能需要额外配置。如果遇到 DNS 问题，可切换到 `eclipse-temurin:17-jre-jammy`（基于 glibc）。

## 18.5 Kubernetes 部署

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: agent-service
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: agent
          image: agent-service:latest
          ports:
            - containerPort: 8080
          env:
            - name: DASHSCOPE_API_KEY
              valueFrom:
                secretKeyRef:
                  name: agent-secrets
                  key: api-key
          resources:
            requests: { memory: "1Gi", cpu: "500m" }
            limits: { memory: "2Gi", cpu: "2000m" }
          livenessProbe:
            httpGet: { path: /actuator/health/liveness, port: 8080 }
            initialDelaySeconds: 30
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: 8080 }
            initialDelaySeconds: 10
          lifecycle:
            preStop:
              exec:
                command: ["sh", "-c", "sleep 15"] # 配合 GracefulShutdown
```

### Agent 特有的部署考量：

1. **资源配额：**Agent 的内存消耗主要来自对话历史（每轮约 1-5KB，100 轮约 100-500KB）和 LLM 客户端连接池。2GB 上限通常足够单个 Agent 实例，但 AutoContextMemory 的压缩操作会短暂增加内存（压缩期间同时持有原始和摘要版本）。

2. **健康检查**：liveness 探测的 `initialDelaySeconds: 30` 给模型客户端初始化留出时间；readiness 探测 10 秒即可，因为 Agent 在模型客户端就绪后即可接受请求。
3. **会话亲和性**：默认 Deployment 不保证会话亲和性。对于使用 InMemorySession 的 Agent，必须启用 Session Affinity (`sessionAffinity: ClientIP`)；对于使用 Redis/MySQL Session 的 Agent，亲和性不是必需的但能减少状态恢复开销。

#### 优雅关闭与 K8s 的配合：

1. K8s 发送 SIGTERM → Pod 标记为 Terminating
2. Spring Boot `GracefulShutdown` 开始等待活跃请求完成
3. AgentScope-Java `GracefulShutdownManager` 在安全点保存状态
4. `preStop` hook 给 15 秒缓冲期
5. `terminationGracePeriodSeconds` 设置为 30-60 秒
6. 超时后 K8s 发送 SIGKILL

## 18.6 HPA 自动扩缩容

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: agent-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: agent-service
  minReplicas: 2
  maxReplicas: 10
```

```
metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 70
```

Agent 的扩缩容指标选择：

- **CPU**：LLM 调用是 I/O 密集，CPU 不是最佳指标
- **并发请求数**：更准确，但需要自定义指标
- **活跃 Agent 数**：最精确，需要通过 Actuator 暴露

**缩容风险**：HPA 缩容时可能驱逐正在执行 ReAct 循环的 Agent。必须配合

GracefulShutdown（参见第 11 章）确保状态保存后才能终止。建议

`scaleDown.stabilizationWindowSeconds` 设置为 300 秒，避免流量波动导致的频繁缩容。

---

## 思考题

1. Agent 的扩缩容指标选择——CPU、并发请求数、活跃 Agent 数，哪个最准确？为什么？
2. preStop hook 的 15 秒缓冲期和 terminationGracePeriodSeconds 的关系如何设计？给出具体数值。
3. Spring Boot Starter 的自动配置过于激进（一旦引入就启动 A2A Server）——设计一个更灵活的条件化配置方案。

**\*\*参考：\*\***

1. 最准确的是并发请求数——Agent 是 I/O 密集型，CPU 利用率低但并发连

接数高。CPU 不准确因为 Agent 大部分时间在等 LLM 响应 (CPU<10%)。活跃 Agent 数不准确因为 Agent 可能处于 HITL 挂起状态 (占用内存但不消耗资源)。推荐：并发请求数为主指标，配合活跃 Agent 数为辅助。

2. 具体数值：preStop=15s, shutdownTimeout=30s, terminationGracePeriodSeconds=60s。关系：K8s 发送 SIGTERM→ (0-15s) preStop hook 等待 Pod 从 Service 摘除→ (15s 起) 应用进入 GracefulShutdown→最多 30s 完成在途请求→ (45s) 如果仍有请求→K8s 等待至 60s 发送 SIGKILL。预留 15s 安全余量。

3. 条件化配置方案：①引入 @EnableAgentScope 注解，显式启用才加载自动配置；②配置开关 agentscope.enabled=false 默认不启动；③按模块拆分——@EnableA2aServer、@EnableTracer、@EnableSession 各自独立控制；④使用 Spring Boot 的 @ConditionalOnProperty 实现细粒度配置。

---

# 第 19 章 性能调优

## 19.1 Agent 执行性能分析

Agent 的执行时间是多个阶段的叠加：

总耗时 = 推理延迟 + 工具执行延迟 + 内存操作 + 格式化

典型分布 (5 轮 ReAct 循环)：

|         |                             |          |
|---------|-----------------------------|----------|
| LLM 调用： | $5 \times 3-10s = 15-50s$   | (80-90%) |
| 工具执行：   | $5 \times 0.1-2s = 0.5-10s$ | (5-15%)  |
| 格式化：    | $5 \times 1-5ms = 5-25ms$   | (<1%)    |
| 内存操作：   | $5 \times <1ms = <5ms$      | (<1%)    |

**瓶颈识别：** LLM 调用几乎总是主要瓶颈。优化重点应放在减少 LLM 调用次数和降低单次调用延迟上。

## 19.2 工具执行优化

### 并行执行

当 LLM 返回多个 tool\_call 时，`Toolkit.callTools()` 通过 `Flux.merge()` 并行执行：

```
// Toolkit 内部实现
Flux<ToolResultBlock> results = Flux.fromIterable(toolCalls)
    .flatMap(call -> executeTool(call)
        .subscribeOn(Schedulers.boundedElastic()),
        maxConcurrency); // 默认 256 并发
```

**调优建议：**

- I/O 密集型工具 (HTTP 调用、数据库查询)：增大并发度

- 有速率限制的 API：降低并发度或添加限流
- 有依赖关系的工具：分开到不同 ReAct 轮次（LLM 会自动处理）

## 超时与重试

`ModelUtils.applyTimeoutAndRetry()`（`agentscope-core/.../ModelUtils.java`）为模型调用配置超时和重试：

```
public static <T> Flux<T> applyTimeoutAndRetry(Flux<T> flux,
        Duration timeout, int maxRetries) {
    return flux.timeout(timeout)
        .retryWhen(Retry.backoff(maxRetries, Duration.ofSeconds(1))
            .filter(throwable -> !(throwable instanceof
                ToolSuspendException)))
        .onRetryExhaustedThrow((spec, signal) -> signal.failure());
}
```

### 调优建议：

- 简单查询：`timeout=30s`, `maxRetries=2`
- 复杂推理：`timeout=120s`, `maxRetries=3`
- 工具密集：`timeout=60s`, `maxRetries=1`（工具执行本身的超时单独配置）

# 19.3 内存管理优化

## AutoContextMemory 参数调优

| 参数           | 默认值 | 调优建议                           |
|--------------|-----|--------------------------------|
| `tokenRatio` | 0.4 | 0.3-0.5 之间；低 = 更激进压缩，高 = 更多上下文 |
| `lastKeep`   | 10  | 5-20 之间；取决于对话轮次密度              |

|        |                 |             |
|--------|-----------------|-------------|
| 压缩触发时机 | 超出 tokenRatio 时 | 不可配置，框架自动判断 |
|--------|-----------------|-------------|

**压缩对推理质量的影响：**

- tokenRatio = 0.7 → 几乎不压缩 → 高质量但高成本
- tokenRatio = 0.5 → 适度压缩 → 平衡点
- tokenRatio = 0.3 → 激进压缩 → 低成本但可能丢失关键信息
- tokenRatio = 0.1 → 极端压缩 → 仅保留摘要和最近几条

# 19.4 模型调用优化

## 模型选择策略

任务复杂度与模型匹配：

简单查询（天气、翻译）：  
→ qwen-turbo / gpt-4o-mini：快速、便宜

中等推理（数据分析、文本生成）：  
→ qwen-plus / gpt-4o：平衡

复杂推理（多步推理、工具编排）：  
→ qwen-max / gpt-4o：强大但慢

## 降级策略

```
public class FallbackModel extends ChatModelBase {
    private final ChatModelBase primary;
    private final ChatModelBase fallback;

    @Override
    public Flux<ChatResponse> doStream(List<Msg> msgs, GenerateOptions
options) {
        return primary.doStream(msgs, options)
            .onErrorResume(TimeoutException.class,
                e -> fallback.doStream(msgs, options));
    }
}
```

## Prompt Caching 优化

Prompt Caching（提示缓存）是降低 LLM 调用延迟和成本的最有效手段之一。

其原理是：当多次请求的提示前缀相同时，LLM 服务端可以复用已计算的 KV Cache，跳过前缀部分的注意力计算，直接从缓存位置开始生成。

**Agent 场景的缓存友好性：**Agent 的每次 LLM 调用都包含系统提示词 + 工具描述 + 对话历史，其中系统提示词和工具描述在会话期间保持不变，构成了天然的"缓存前缀"。在 5 轮 ReAct 循环中，前缀可能占总 token 的 60-80%，缓存命中可节省 50-70% 的输入 token 成本和 30-50% 的首 token 延迟。

**各提供商的缓存支持：**

| 提供商       | 机制                 | 触发条件             | 缓存时长    |
|-----------|--------------------|------------------|---------|
| Anthropic | `cache_control` 标记 | 显式标记缓存边界         | 5 分钟    |
| DeepSeek  | 自动前缀缓存             | 前缀匹配即命中          | 未知      |
| OpenAI    | Prompt Caching     | 自动 (>1024 token) | 5-10 分钟 |

**设计建议：**将系统提示词和工具描述放在消息列表的最前面，保持前缀稳定性。

避免在系统提示词中包含时间戳或随机数等动态内容，这会使缓存失效。对于多轮对话，早期消息的稳定性高于最新消息，因此 AutoContextMemory 的压缩策略（只压缩尾部消息）天然有利于缓存命中。

---

**思考题**

1. Agent 执行性能的瓶颈几乎总是 LLM 调用。除了减少调用次数，还有什么策略可以降低 LLM 调用延迟？
2. AutoContextMemory 的 tokenRatio 参数从 0.3 到 0.7，对推理质量和成本的影响如何量化？
3. 设计一个模型降级策略：当主模型不可用时，如何平滑切换到备用模型？

**\*\*参考：\*\***

1. 降延迟策略：①模型缓存——Prompt Caching（如 Anthropic/DeepSeek 的 `cache_control`），重复前缀免重算；②预测性预取——在 Agent 思考阶段预加载可能用到的工具描述；③流式输出早截断——检测到完整 JSON 结构即返回，不等全部 token 生成完；④模型蒸馏——用小模型处理简单请求，大模型只处理复杂请求；⑤边缘部署——将模型推理部署到离用户更近的节点。

2. tokenRatio 从 0.3 到 0.7 的影响：0.3 时工作记忆窗口小，早期消息快速被压缩，推理质量下降（丢失上下文），但 token 成本低；0.7 时工作记忆窗口大，保留更多原始消息，推理质量高，但 token 消耗翻倍以上。量化方法：在标准评测集上测试不同 ratio 下的工具调用准确率，绘制 ratio-accuracy 曲线，找拐点。经验值：0.4-0.5 是性价比最优点。

3. 降级策略设计：①健康检测——连续 3 次调用失败或 P99 延迟 > 阈值触发降级；②分级降级——第一级切同厂商小模型（如 GPT-4→GPT-3.5），第二级切跨厂商备选（如 OpenAI→DeepSeek）；③上下文适配——模型切换时重新格式化对话历史（不同模型 Formatter 不同）；④自动恢复——备用模型成功率 > 95% 持续 5 分钟后尝试切回主模型。

# 第 20 章 安全设计

LLM Agent 的安全与传统软件安全有着本质区别：传统软件的行为由代码逻辑完全决定，而 Agent 的行为由 LLM 推理结果决定——这意味着攻击者可以通过构造输入来“操控”代码执行路径。OWASP 在 2025 年发布的 LLM Top 10 风险清单中，与 Agent 直接相关的高危项包括：提示注入（LLM01）、敏感信息泄露（LLM06）、不安全的输出处理（LLM02）和供应链攻击（LLM05）。

**Agent 安全的三个维度：**本章从权限控制（谁能用什么工具）、输入防护（防止恶意指令操控 Agent）和数据安全（保护敏感信息不被泄露）三个维度展开，每个维度对应框架中的一个安全机制。

## 20.1 工具安全

### ToolGroup 权限隔离

`ToolGroup` (`agentscope-core/.../tool/ToolGroup.java:42-54`) 是工具权限隔离的基本单元：

```
// 创建工具组
toolkit.registration().group("sensitive")
    .description("Sensitive operations requiring approval")
    .active(false)           // 默认不激活
    .tools("delete_data", "execute_sql", "deploy")
    .apply();
```

**隔离机制** (`ToolGroupManager.java:230-244`)：

```
boolean isActiveTool(String toolName) {
    // 未分组的工具 → 始终可用
    if (!isGroupedTool(toolName)) return true;
```

```
// 分组工具 → 至少属于一个激活组才可用
return tools.get(toolName).stream()
    .anyMatch(group -> toolGroups.get(group).isActive());
}
```

**安全门** (`ToolExecutor.java:192-199`) :

```
if (!groupManager.isActiveTool(toolCall.getName())) {
    return ToolResultBlock.error(
        "Unauthorized tool call: '" + toolCall.getName() + "' is not
        available");
}
```

**动态激活**: Agent 可以通过 `reset_equipped_tools` 元工具自主激活工具组

(`MetaToolFactory.java:43-107`)。这个元工具的参数 `schema` 动态生成 `enum`，只包含存在的组名，防止越权。

## SkillBox 技能隔离

SkillBox 的 `ToolGroup` 默认 `active=false` (`SkillBox.java:143`)，技能加载后不会立即可用，需要显式激活。这防止了 Agent 在不知情的情况下获得新能力。

## 20.2 提示注入防护

提示注入 (Prompt Injection) 是 LLM Agent 面临的最大安全威胁之一。

AgentScope-Java 的多层防护:

**Hook 层面:**

```
public class ContentFilterHook implements Hook {
    private final List<Pattern> dangerousPatterns = List.of(
        Pattern.compile("(?i)ignore previous instructions"),
        Pattern.compile("(?i)system:"),
        Pattern.compile("(?i)you are now")
    );

    @Override
```

```

    public Mono<HookResult> onEvent(HookEvent event) {
        if (event instanceof PreCallEvent preEvent) {
            for (Msg msg : preEvent.getInputMessages()) {
                if (containsInjection(msg)) {
                    return Mono.just(HookResult.stopAgent(
                        "Potential prompt injection detected"));
                }
            }
        }
        return Mono.just(HookResult.continue());
    }
}

```

**ToolValidator 输入验证** (`ToolValidator.java:51-96`) :

每个工具调用前，`ToolValidator.validateInput()` 使用 JSON Schema Draft 2 验证 LLM 生成的参数。这防止了参数注入攻击——LLM 输出的参数必须符合预定义的 Schema。

## 20.3 Shell 工具安全

`ShellCommandTool` (`agentscope-`

`core/.../tool/coding/ShellCommandTool.java:65-542`) 实现了多层安全防护:

```

graph TD
    Input["命令输入"] --> L1["安全层 1: CommandValidator"]
    L1 -->|白名单通过| Execute["执行命令"]
    L1 -->|白名单失败| L2["安全层 2: ApprovalCallback"]
    L2 -->|人工批准| Execute
    L2 -->|人工拒绝| Reject["SecurityError"]
    Execute --> L3["安全层 3: baseDir 限制"]
    L3 -->|路径合法| Result["返回结果"]
    L3 -->|路径越权| Reject2["阻止执行"]

```

**使用方式:**

```
// 生产环境：白名单模式
ShellCommandTool shellTool = new ShellCommandTool(
    Set.of("ls", "cat", "grep", "wc"), // 只允许这 4 个命令
    null, // 无人工审批回调
    Paths.get("/data/workspace") // 限制工作目录
);

// 开发环境：审批模式
ShellCommandTool shellTool = new ShellCommandTool(
    Set.of(), // 无白名单
    command -> { // 人工审批回调
        logger.warn("Shell command approval: {}", command);
        return securityService.approve(command);
    },
    null
);
```

## 20.4 SkillFileFilter 路径遍历防护

`SkillBox.uploadSkillFiles()` (`SkillBox.java:750-756`) 实现了路径遍历防护：

```
Path targetPath = skillDir.resolve(resourcePath).normalize();

// 关键安全检查：规范化路径必须在技能目录内
if (!targetPath.startsWith(skillDir)) {
    logger.warn("Path traversal attempt detected: {}", resourcePath);
    continue; // 跳过此文件
}
```

`DefaultSkillFileFilter` (`SkillBox.java:780-815`) 额外限制可上传的目录和扩展名：

- 默认允许目录：`scripts/`, `assets/`
- 默认允许扩展名：`.py`, `.js`, `.sh`

# 20.5 数据安全

| 安全维度         | 风险              | 防护措施              |
|--------------|-----------------|-------------------|
| API Key 泄露   | 代码中硬编码          | 环境变量 + K8s Secret |
| Session 数据泄露 | MySQL/Redis 未加密 | 传输层 TLS + 静态加密    |
| 敏感数据记忆       | Memory 中保存密码等   | 脱敏 Hook + 定期清理    |
| A2A 通信窃听     | 明文 JSON-RPC     | HTTPS + 认证头       |
| 工具参数泄露       | 日志中记录敏感参数       | 日志脱敏 + 审计分级       |

## API Key 管理最佳实践：

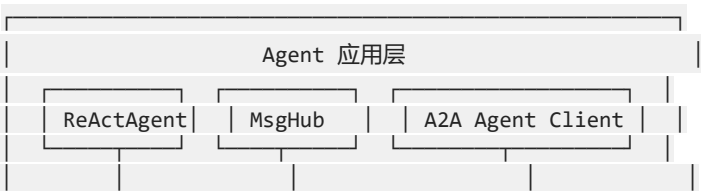
```
apiVersion: v1
kind: Secret
metadata:
  name: agent-secrets
type: Opaque
data:
  dashscope-api-key: <base64-encoded>
```

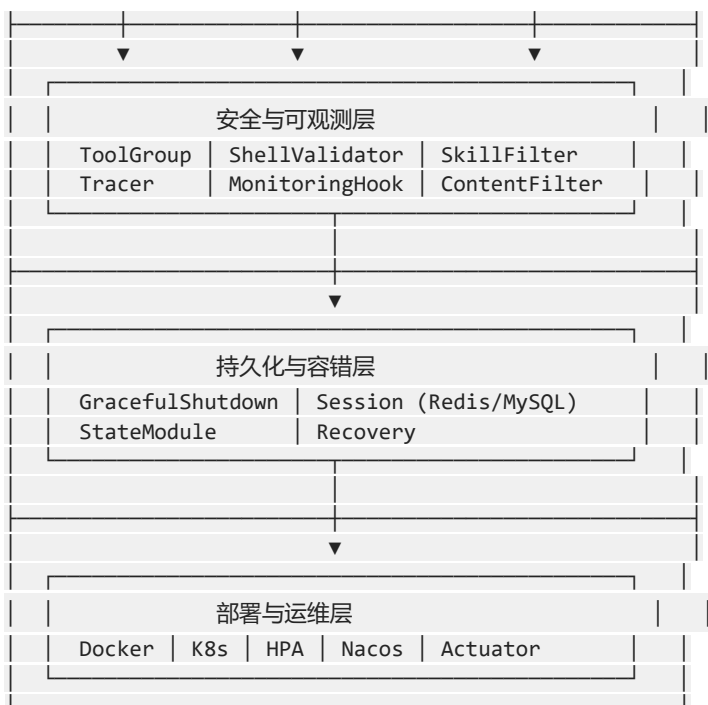
## Deployment 中通过 secretKeyRef 引用：

```
env:
- name: DASHSCOPE_API_KEY
  valueFrom:
    secretKeyRef:
      name: agent-secrets
      key: dashscope-api-key
```

## 第四篇总结：生产工程全景图

经过第 16-20 章的讨论，一个生产级 Agent 应用的完整技术栈如下：





从上到下：应用逻辑 → 安全与可观测 → 持久化与容错 → 部署与运维。每一层都是生产环境不可或缺的重要组成部分。

## 思考题

1. ToolGroup 的"未分组工具始终可用"规则是否构成安全风险？如何改进？
2. ShellCommandTool 的多层安全防护中，哪一层最可能被绕过？如何加固？
3. 在 A2A 通信中，当前缺少认证机制。设计一个基于 JWT 的 A2A 认证方案。

**\*\*参考：\*\***

1. 是安全风险——未分组工具对所有 Agent 可见，意味着新加入的 Agent 自动获得所有未分组工具权限，违反最小权限原则。改进：①默认拒绝——未分组工具不可用，必须显式分配到 ToolGroup；②白名单模式——ToolGroup 声明允许的工具列表而非禁止列表；③审计——记录未分组工具的调用日志，

定期 review。

2. 最可能被绕过的是 *SkillFileFilter*——它只检查文件路径，不检查文件内容。攻击方式：①符号链接攻击——`ln -s /etc/passwd /tmp/safe_file`，路径合法但内容敏感；②路径编码绕过——使用 URL 编码或 Unicode 规范化绕过路径匹配。加固：①解析符号链接后检查真实路径；②使用操作系统 *chroot/namespace* 隔离；③文件内容扫描。

3. JWT 认证方案：①AgentCard 中声明 *securitySchemes* (*type: http, scheme: bearer, bearerFormat: JWT*)；②A2A 请求头携带 *Authorization: Bearer <JWT>*；③JWT payload 包含：*agentId*、*permissions*（可调用的目标 Agent 列表）、*exp*；④*TransportWrapper* 在 *handleRequest()* 前校验 JWT；⑤Nacos 注册时发布公钥（JWKS 端点）；⑥Token 刷新——短期 JWT（5 分钟）+ Refresh Token。

---

# 第五篇：实战与生态

---

## 第 21 章 从零构建你的第一个 Agent

本章将从零开始，逐步构建一个完整的 Agent 应用。每个步骤不仅展示代码，更解释背后的设计选择——为什么这样写，而非仅仅展示怎么写。建议读者边读边动手，逐步理解从最简 Agent 到生产级 Agent 的演进路径。

### 21.1 环境搭建

**环境要求：** Java 17+, Maven 3.8+, API Key (DashScope 或 OpenAI)

在 `pom.xml` 中引入核心依赖：

```
<dependency>
  <groupId>io.agentscope</groupId>
  <artifactId>agentscope-core</artifactId>
  <version>1.0.11</version>
</dependency>
```

如果使用 DashScope 模型，还需要引入对应的扩展包。如果使用 OpenAI 兼容模型（包括 DeepSeek、Moonshot 等），核心包已内置支持，无需额外依赖。选择模型提供商时，核心考量是：**API 兼容性**（是否遵循 OpenAI 协议）和 **延迟**（国内模型通常延迟更低）。

## 21.2 第一个对话 Agent

最简 Agent 只需 5 行核心代码：

```
DashScopeChatModel model = DashScopeChatModel.builder()
    .apiKey(System.getenv("DASHSCOPE_API_KEY"))
    .modelName("qwen-plus").build();

ReActAgent agent = ReActAgent.builder()
    .name("assistant").model(model)
    .memory(new InMemoryMemory()).build();

Msg response = agent.call(Msg.userText("你好，请介绍一下你自己")).block();
System.out.println(response.getTextContent());
```

**代码解读：**这三步对应了 Agent 的三个核心要素——**大脑**（Model）、**记忆**

（Memory）和**身份**（name）。Builder 模式让配置一目了然：模型决定了推理能力，记忆决定了上下文理解，名称用于日志和追踪。`agent.call()` 返回

`Mono<Msg>`，`.block()` 将异步调用转为同步等待——在真实项目中应使用响应式链而非阻塞。

此时 Agent 只能对话，不能执行任何操作——它没有“手”。接下来添加工具。

## 21.3 添加工具

工具赋予 Agent 与外部世界交互的能力。`@Tool` 注解将普通 Java 方法声明为

Agent 可调用的工具：

```
public class WeatherTools {
    @Tool(name = "get_weather", description = "获取指定城市的天气信息")
    public String getWeather(
        @ToolParam(name = "city", description = "城市名称") String city)
    {
        return city + "今天晴，气温 25°C";
    }
}
```

```
}  
}
```

**设计要点：** `description` 是 LLM 决定"何时调用此工具"的唯一依据，写好 `description` 是工具设计最重要的部分——过于模糊（"获取信息"）会导致误调用，过于具体（"获取中国华北地区城市天气"）会限制泛化能力。理想描述应包含**动作**（获取）+ **对象**（天气）+ **输入**（城市名称）。

将工具注册到 Agent 的 Toolkit 中：

```
Toolkit toolkit = new Toolkit();  
toolkit.registerTool(new WeatherTools());  
  
ReActAgent agent = ReActAgent.builder()  
    .name("weather_assistant")  
    .sysPrompt("你是一个天气助手，帮助用户查询天气信息")  
    .model(model).toolkit(toolkit)  
    .memory(new InMemoryMemory()).build();
```

当用户问"北京今天天气怎么样？"时，Agent 的 ReAct 循环会自动执行：推理（需要天气信息）→ 调用工具（`get_weather("北京")`）→ 获取结果（"北京今天晴"）→ 生成回答。`sysPrompt` 引导 Agent 优先使用工具而非凭记忆回答，避免"幻觉"天气数据。

## 21.4 结构化输出

当 Agent 需要返回机器可读的结果（而非自由文本）时，使用结构化输出：

```
public record WeatherInfo(String city, int temperature, String condition) {}  
  
Msg response = agent.call(Msg.userText("北京天气"),  
    WeatherInfo.class).block();  
WeatherInfo info = response.getStructuredData(WeatherInfo.class);
```

**原理：** 框架在 LLM 请求中注入 JSON Schema 约束，强制模型按 `WeatherInfo` 的字段格式输出。这是 Function Calling 机制的复用——结构化输出本质上是

一个"无副作用的工具调用", LLM 填充参数而非执行操作。Java Record 是理想的数据载体, 因为其不可变性和自动生成的 accessor 与 JSON 反序列化天然兼容。

## 21.5 流式响应

对于需要实时反馈的场景 (如长文生成、多步推理), 使用流式输出:

```
agent.stream(Msg.userText("讲个故事"), StreamOptions.DEFAULT)
    .filter(e -> e.getType() == EventType.REASONING || e.getType() ==
        EventType.AGENT_RESULT)
    .doOnNext(e -> System.out.print(e.getMessage().getTextContent()))
    .blockLast();
```

**流式事件过滤:** `StreamEvent` 有多种类型 (REASONING、TOOL\_CALL、TOOL\_RESULT、AGENT\_RESULT), 过滤出 REASONING 和 AGENT\_RESULT 事件, 可以在用户界面展示"Agent 正在思考..."的实时反馈, 而不暴露工具调用的内部细节。这在 SSE (Server-Sent Events) 场景中特别有用——前端通过 `EventSource` 接收流式事件, 实现打字机效果。

## 21.6 MCP 工具集成

MCP (Model Context Protocol) 允许 Agent 动态发现和使用外部工具, 无需在编译时声明:

```
McpClientWrapper mcpClient = McpClientBuilder.create("filesystem")
    .stdioTransport("npx", "-y", "@modelcontextprotocol/server-filesystem",
        "/tmp")
    .buildAsync().block();

toolkit.registerMcpClient(mcpClient).block();
```

**与 @Tool 的选择：**@Tool 适合团队内部、Java 原生的工具（编译期类型检查、IDE 补全、调试方便）；MCP 适合第三方维护的工具、跨语言共享的场景、需要独立部署和扩缩容的工具服务。在本例中，filesystem MCP Server 提供了文件读写能力，Agent 可以直接操作 /tmp 目录——这意味着你不需要自己写文件操作工具，直接复用社区生态。

**从最简到生产：**以上 6 步构建了一个可运行的 Agent，但从开发到生产还需要：错误处理（工具失败降级）、AutoContextMemory（防上下文溢出）、HITL（关键操作审批）、Session 持久化（断点恢复）、Tracer 追踪（调试排障）——这些能力在前面各章已详细讨论，按需组合即可。

---

## 思考题

1. 从 5 行代码的最简 Agent 到生产级 Agent，中间需要增加哪些关键能力？按优先级排序。
2. MCP 工具集成与@Tool 注解在开发体验上有什么差异？什么场景应该优先选择 MCP？
3. 如果要为你的团队编写一个 Agent 开发脚手架（template 项目），需要包含哪些内容？

**\*\*参考：\*\***

1. 按优先级排序：①错误处理（工具调用失败/LLM 超时的降级策略）→ ②记忆管理（AutoContextMemory 防上下文溢出）→ ③HITL（关键操作人工审批）→ ④状态持久化（断点恢复）→ ⑤可观测性（Tracer 追踪调试）→ ⑥安全防护（ToolGroup 权限隔离）→ ⑦流式输出（SSE 用户体验）→ ⑧部署运

维（容器化/扩缩容）。

2. 开发体验差异：@Tool 注解——在 Java 代码中定义，编译期类型检查，IDE 自动补全，调试方便；MCP——JSON Schema 定义，运行时动态发现，跨语言复用，但调试需额外工具。优先 MCP 的场景：①工具由第三方团队维护（不同代码仓库）；②工具需要独立部署和扩缩容；③多 Agent 框架混合（如 Java+Python 共享同一 MCP Server）。

3. 脚手架内容：①项目模板——pom.xml（依赖管理）+ 标准目录结构；②配置模板——application.yml（模型/工具/记忆配置）+ 多环境 profile；③Agent 基类——封装常用模式（ReAct+HiTL+状态持久化）；④工具库——团队常用@Tool 集合；⑤测试框架——Mock LLM 响应+工具执行断言；⑥Dockerfile + K8s manifest；⑦CI/CD pipeline（含集成测试）。

---

# 第 22 章 多 Agent 游戏实战——Werewolf

## 22.1 游戏规则与 Agent 角色设计

9 名玩家，3 种阵营：

| 角色  | 数量 | 能力        | Agent 设计要点    |
|-----|----|-----------|---------------|
| 狼人  | 3  | 夜晚协商杀人    | 需要合作策略，隐藏身份   |
| 村民  | 3  | 无特殊能力     | 通过讨论推理狼人身份    |
| 预言家 | 1  | 每晚查验一人身份  | 掌握关键信息，决定何时揭露 |
| 女巫  | 1  | 解药+毒药各一次  | 关键决策：救谁、毒谁    |
| 猎人  | 1  | 被淘汰时可带走一人 | 选择最有价值的目标     |

## 22.2 系统提示词工程

每个角色有独立的系统提示词（通过 i18n 支持），核心设计原则：

- 狼人：**协调杀人，白天伪装， "Keep responses concise (2-3 sentences)"
- 预言家：**谨慎揭露，避免成为目标

- **女巫**：战略性使用药水
- **猎人**：死亡时的射杀决策
- **村民**：通过讨论推理

```
// WerewolfWebGame.java LINE 122-164
ReActAgent agent = ReActAgent.builder()
    .name(name) // "甲", "乙", ... 或
"Alice", "Bob", ...
    .sysPrompt(prompts.getSystemPrompt(role, name)) // 角色特定提示
    .model(model) // 共享 DashScope qwen3-
max
    .memory(new InMemoryMemory()) // 每个 Agent 独立记忆
    .toolkit(new Toolkit()) // 空工具集 (纯推理角色)
    .build();
```

## 22.3 MsgHub 群组讨论的三种用法

游戏中共创建三个 MsgHub 实例，展示了 MsgHub 的三种使用模式：

### 22.3.1 模式一：狼人夜间讨论（autoBroadcast + 禁用 + 并行投票）

```
// WerewolfWebGame.java LINE 208-268
try (MsgHub werewolfHub = MsgHub.builder()
    .name("WerewolfDiscussion")
    .participants(werewolves)
    .announcement(werewolfDiscussionPrompt)
    .enableAutoBroadcast(true) // 讨论时互相可见
    .build()) {

    werewolfHub.enter().block();

    // 阶段 1: 讨论 (autoBroadcast ON) — 2 轮顺序发言
    for (int i = 0; i < 2; i++) {
        for (Player wolf : werewolves) {
            wolf.getAgent().call().block();
            // 每个 Agent 的回复自动广播给其他狼人
        }
    }
}
```

```

    }

    // 阶段 2: 投票 (autoBroadcast OFF) — 秘密投票
    werewolfHub.setAutoBroadcast(false);

    FanoutPipeline votingPipeline = FanoutPipeline.builder()
        .addAgents(werewolves).concurrent().build();
    List<Msg> votes = votingPipeline.execute(votingPrompt,
        VoteModel.class).block();

    // VoteModel: {targetPlayer, reason} — 结构化投票
}

```

**关键设计：**讨论阶段 autoBroadcast ON（狼人互相看到意见），投票阶段 OFF（投票保密），FanoutPipeline 实现并行投票。

### 22.3.2 模式二：白天公开讨论（autoBroadcast ON + 多轮）

```

// WerewolfWebGame.java LINE 430-478
try (MsgHub discussionHub = MsgHub.builder()
    .name("DayDiscussion")
    .participants(alivePlayers)
    .enableAutoBroadcast(true)
    .build()) {

    discussionHub.enter().block();

    for (int round = 1; round <= MAX_DISCUSSION_ROUNDS; round++) {
        // 注入轮次提示
        if (round > 1) {
            Msg roundPrompt = prompts.createDiscussionPrompt(gameState,
                round);
            alivePlayers.forEach(p ->
                p.getAgent().getMemory().addMessage(roundPrompt));
        }

        // 顺序发言 (autoBroadcast 广播给所有人)
        for (Player player : alivePlayers) {
            player.getAgent().call().block();
        }
    }
}

```

### 22.3.3 模式三：白天投票（autoBroadcast OFF + FanoutPipeline + 结构化输出）

```
// WerewolfWebGame.java LINE 481-543
try (MsgHub votingHub = MsgHub.builder()
    .name("DayVoting")
    .participants(alivePlayers)
    .enableAutoBroadcast(true) // enter() 需要初始化
    .build()) {

    votingHub.enter().block();
    votingHub.setAutoBroadcast(false); // 立即关闭—投票保密

    FanoutPipeline votingPipeline = FanoutPipeline.builder()
        .addAgents(alivePlayers).concurrent().build();

    // 结构化输出：每个 Agent 返回 VoteModel {targetPlayer, reason}
    List<Msg> votes = votingPipeline.execute(votingPrompt,
        VoteModel.class).block();

    // 统计票数
    Player eliminated = utils.countVotes(votes, gameState);
}
```

## 22.4 结构化输出的角色决策机制

每个特殊角色使用不同的结构化输出模型：

```
// 女巫决策—LINE 282-311
Msg healDecision = witch.getAgent()
    .call(healPrompt, WitchHealModel.class).block();
WitchHealModel healModel =
    healDecision.getStructuredData(WitchHealModel.class);
if (Boolean.TRUE.equals(healModel.useHealPotion())) {
    victim.setAlive(true); // 使用解药
    witch.setWitchHasHealPotion(false);
}

// 预言家查验—返回 SeerCheckModel {targetPlayer, reason}
// 猎人射杀—返回 HunterShootModel {willShoot, targetPlayer, reason}
// 投票—返回 VoteModel {targetPlayer, reason}
```

## 22.5 SSE 流式 Web 界面

```
// WerewolfWebController.java
@GetMapping(value = "/start", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<ServerSentEvent<String>> startGame(...) {
    return gameEventEmitter.asFlux()
        .map(event -> ServerSentEvent.<String>builder()
            .id(String.valueOf(event.id()))
            .event(event.type().name())
            .data(toJson(event))
            .build());
}
```

GameEventEmitter 使用 `Sinks.Many<GameEvent>` 作为事件总线，游戏各阶段产生的消息通过它推送到前端。

## 22.6 游戏流程完整数据流

```
graph TD
    Start["游戏开始: initializeGame()"] --> Loop["循环 round 1..MAX_ROUNDS"]
    Loop --> Night["nightPhase()"]
    Night --> Wolf["狼人讨论<br/>MsgHub autoBroadcast"]
    ON→OFF<br/>FanoutPipeline 并行投票
    Night --> Seer["预言家查验<br/>SeerCheckModel 结构化输出"]
    Night --> Witch["女巫决策<br/>WitchHealModel+PoisonModel"]
    Wolf --> Check1["checkGameEnd?"]
    Seer --> Check1
    Witch --> Check1
    Check1 -->|否| Day["dayPhase()"]
    Check1 -->|是| Win["announceWinner()"]
    Day --> Discuss["白天讨论<br/>MsgHub autoBroadcast ON, 3 轮"]
    Day --> Vote["白天投票<br/>MsgHub OFF + FanoutPipeline"]
    Day --> Hunter["猎人射杀<br/>HunterShootModel"]
    Discuss --> Check2["checkGameEnd?"]
    Vote --> Check2
    Hunter --> Check2
```

Check2 -->|否| Loop

Check2 -->|是| Win

## 思考题

1. Werewolf 游戏中，三个 MsgHub 实例的 autoBroadcast 策略各不相同。  
这种"按需开关"的设计模式可以应用到哪些业务场景？
2. FanoutPipeline 并行投票如何保证投票的独立性？如果 Agent 之间有依赖关系（如狼人需要协商），如何处理？
3. 如果要将 Werewolf 游戏从 9 人扩展到 20 人，当前架构有哪些瓶颈？如何优化？

**\*\*参考：\*\***

1. 按需开关设计模式适用于：①审批流程——讨论阶段 `autoBroadcast=true`（公开讨论），投票阶段 `=false`（秘密投票）；②谈判场景——己方讨论可见（on），对对方不可见（off）；③协作编辑——编辑阶段各自独立（off），review 阶段共享（on）；④多团队项目——组内讨论可见，跨组讨论需中转。
2. 投票独立性保证：FanoutPipeline 并行发送请求，每个 Agent 收到相同的输入但不共享中间状态，各自独立生成投票。狼人协商的处理：投票前用 MsgHub 进行狼人私有讨论（`autoBroadcast=true`），达成共识后再进入 FanoutPipeline 投票——讨论阶段是串行的（先讨论后投票），投票阶段才是并行的。
3. 瓶颈：①LLM 并发——20 人需要 20 个 LLM 调用，9 人版本可以串行但 20 人必须并行化；②MsgHub 广播风暴—— $N^2$  消息量（20 人=380 条/轮）；③上下文膨胀——每个 Agent 记忆包含 19 人的发言历史。优化：  
①Agent 池化——复用 Agent 实例而非每轮新建；②分组 MsgHub——按阵

营分组广播; ③记忆压缩——AutoContextMemory + 摘要压缩他人发言;  
④异步投票——不等待所有人完成即开始计票。

---

# 第 23 章 生产级多 Agent 系统实战——Boba Tea Shop

## 23.1 架构设计

Boba Tea Shop 是一个"云边奶茶铺"智能客服系统，采用 Supervisor + A2A + MCP 架构：

```
graph TD
    UI["Vue 3 UI"] -->|SSE| Supervisor["Supervisor Agent<br/>AutoContextMemory + MySQLSession<br/>A2aAgentTools 路由"]
    Supervisor -->|A2A/JSON-RPC| Business["Business Sub-Agent<br/>MCP Client → MCP Server<br/>(order/feedback)"]
    Supervisor -->|A2A/JSON-RPC| Consult["Consult Sub-Agent<br/>Knowledge/Bailian RAG<br/>Mem0Memory"]
    Nacos["Nacos 服务发现<br/>AgentCard + MCP Server 注册"] -.->|发现| Supervisor
    MySQL["MySQL Session 持久化"] -.->|存取| Supervisor

    style UI fill:#e1f5fe
    style Supervisor fill:#fff3e0
    style Business fill:#e8f5e9
    style Consult fill:#f3e5f5
```

## 23.2 Supervisor Agent——路由与委派

**核心设计：**每次请求创建新 Agent 实例，实现完全隔离：

```
// SupervisorAgent.java LINE 75-95
public Flux<Event> stream(Msg msg, String sessionId, String userId) {
    Toolkit toolkit = new Toolkit();
    toolkit.registerTool(tools); // A2aAgentTools
}
```

```

// AutoContextMemory: 自动上下文压缩
AutoContextConfig config = AutoContextConfig.builder()
    .tokenRatio(0.4) // 40% 上下文窗口用于工作记忆
    .lastKeep(10)    // 保留最近 10 条消息不被压缩
    .build();
AutoContextMemory memory = new AutoContextMemory(config, model);

// MySQL 持久化
MysqlSession mysqlSession = new MysqlSession(
    dataSource, System.getenv("DB_NAME"), null, true);

ReActAgent agent = createAgent(toolkit, memory);
agent.loadIfExists(mysqlSession, sessionId); // 恢复上次会话

return agent.stream(msg)
    .doFinally(signal -> agent.saveTo(mysqlSession, sessionId)); // 保
存状态
}

```

### 路由工具 (A2aAgentTools.java:29-84):

```

@Tool(description = "Agent for handling consultation-related requests...")
public String callConsultAgent(
    @ToolParam(name = "context") String context,
    @ToolParam(name = "userId") String userId) {
    context = "<userId>" + userId + "</userId>" + context;
    Msg msg =
Msg.builder().content(TextBlock.builder().text(context).build()).build();
    A2aAgent consultAgent = consultAgentProvider.getObject(); // 原型作用域
    return combineAgentResponse(consultAgent.call(msg).block()); // 同步
A2A 调用
}

```

### 路由逻辑通过系统提示词定义（而非硬编码）：

你是一个奶茶店智能客服主管，负责将用户请求路由到合适的子 Agent：

- feedback\_agent：用户反馈、投诉、差评
- consult\_agent：产品咨询、活动信息、冲泡指导
- business\_agent：下单、查单、改单

## 23.3 Business Sub-Agent + MCP Server

Business Sub-Agent 通过 MCP 协议连接后端业务服务：

```
// AgentScopeRunner.java LINE 121-144 — MCP 客户端初始化
private void initializeMcpOnce() {
    if (!mcpInitialized) {
        synchronized (this) {
            if (!mcpInitialized) {
                NacosMcpServerManager mcpServerManager = new
NacosMcpServerManager(aiService);
                NacosMcpClientWrapper mcpClientWrapper =
                    NacosMcpClientBuilder.create("business-mcp-server",
mcpServerManager)
                        .build();
                toolkit.registerMcpClient(mcpClientWrapper).block(); // 注
册 MCP 工具
                mcpInitialized = true;
            }
        }
    }
}
}
```

**MCP Server** (`McpServerConfig.java:66-77`) 使用 Spring WebFlux SSE 传输:

```
@Bean
public McpSyncServer mcpSyncServer(WebFluxSseServerTransportProvider
transportProvider) {
    return McpServer.sync(transportProvider)
        .serverInfo("business-mcp-server", "0.0.1")
        .capabilities(ServerCapabilities.builder().tools(true).build())
        .build(); // + registerOrderTools() + registerFeedbackTools()
}
```

## 23.4 Consult Sub-Agent + RAG + 长期记忆

Consult Sub-Agent 集成了知识库和长期记忆:

```
// AgentScopeRunner.java LINE 55-82
ReActAgent.Builder builder = ReActAgent.builder()
    .name("consult_agent")
    .sysPrompt(consultAgentInstruction)
    .memory(new AutoContextMemory(autoContextConfig, model))
    .hooks(List.of(new MonitoringHook()))
    .model(model)
    .toolkit(toolkit)
    .knowledge(knowledge) // RAG 知识库
    .ragMode(RAGMode.AGENT_CONTROL) // Agent 自主决定何时检索
    .longTermMemory(mem0Memory) // Mem0 长期记忆
    .longTermMemoryMode(LongTermMemoryMode.AGENT_CONTROL);
```

## 23.5 Nacos 服务发现

所有组件通过 Nacos 注册和发现：

```
// NacosA2aServiceConfig.java LINE 29-44
@Bean
public AiService nacosA2aService() throws NacosException {
    Properties properties = new Properties();
    properties.put(PropertyKeyConst.SERVER_ADDR, serverAddress);
    properties.put(PropertyKeyConst.NAMESPACE, namespace);
    return AiFactory.createAiService(properties);
}

// A2aAgentConfiguration.java LINE 26-43
@Bean
public A2aAgent consultAgent(AiService a2aService) {
    return A2aAgent.builder()
        .name("consult_agent")
        .agentCardResolver(new NacosAgentCardResolver(a2aService)) // Nacos
发现
        .build();
}
```

## 23.6 完整请求流转

```
sequenceDiagram
    participant User as 用户
    participant UI as Vue 3 UI
    participant Sup as Supervisor
    participant Biz as Business Agent
    participant Mem as Mem0
    participant MCP as MCP Server

    User->>UI: "我想点一杯珍珠奶茶，老样子"
    UI->>Sup: POST /api/assistant/chat (SSE)
    Sup->>Sup: AutoContextMemory 加载历史
    Sup->>Sup: MysqlSession.loadIfExists()
    Sup->>Biz: A2A callBusinessAgent
    Biz->>Mem: memory-search("老样子")
    Mem-->>Biz: 用户偏好(半糖/少冰/加椰果)
    Biz->>MCP: order-create
    MCP-->>Biz: 订单确认
    Biz-->>Sup: 订单信息
```

```
Sup->>Sup: MysqlSession.saveTo()
Sup-->>UI: SSE 推送回复
UI-->>User: "已为您下单..."
```

## 23.7 Docker + Kubernetes 部署

Boba Tea Shop 提供完整的容器化部署方案：

- **4 个 Docker 镜像**：supervisor-agent, business-sub-agent, consult-sub-agent, business-mcp-server
- **Helm Chart**：一键部署到 K8s
- **Nacos**：服务注册与发现
- **MySQL**：Session 持久化
- **健康检查**：Spring Boot actuator 端点

| 维度       | Quickstart     | Werewolf                   | Boba Tea Shop               |
|----------|----------------|----------------------------|-----------------------------|
| Agent 数量 | 1              | 9                          | 3+ (A2A)                    |
| 协作模式     | 无              | MsgHub +<br>FanoutPipeline | Supervisor (A2A)            |
| 工具使用     | @Tool          | 结构化输出（投票/决策）               | MCP + RAG + Mem0            |
| 记忆管理     | InMemoryMemory | InMemoryMemory             | AutoContextMemory<br>+ Mem0 |
| 状态持久化    | 无              | 无                          | MysqlSession                |

|     |        |             |                     |
|-----|--------|-------------|---------------------|
| 分布式 | 单机     | 单机          | A2A + Nacos         |
| 部署  | main() | Spring Boot | Docker + K8s + Helm |
| 前端  | 无      | HTML + SSE  | Vue 3 + SSE         |
| 复杂度 | ★      | ★★★         | ★★★★★               |

从 Quickstart 到 Boba Tea Shop 的递进关系：

- 1. Quickstart: 单 Agent + 基础工具 → 理解 ReAct 循环
- 2. Werewolf: 多 Agent + MsgHub + 结构化输出 → 理解多 Agent 协作
- 3. Boba Tea Shop: 分布式 + MCP + RAG + 持久化 + 部署 → 理解生产级系统

思考题

- 1. Boba Tea Shop 使用 Supervisor 模式路由用户请求到子 Agent。如果子 Agent 之间需要直接通信（例如 Business Agent 需要咨询 Consult Agent），如何设计避免循环调用？
- 2. 每次 A2A 请求创建新 Agent 实例的设计，在奶茶店场景下的资源开销如何估算？如果高峰期每秒 100 单，需要什么级别的服务器配置？
- 3. 将 Boba Tea Shop 的 MySQL Session 替换为 Redis Session，对系统行为有什么影响？在什么场景下 Redis 是更好的选择？

**\*\*参考：\*\***

- 1. 避免循环调用：①引入消息头 *depth* 计数器——子 Agent 间调用

depth+1, depth>3 拒绝; ②Supervisor 作为唯一路由——子 Agent 不直接互调, 请求先回 Supervisor 再路由; ③DAG 依赖声明——子 Agent 声明依赖关系, 框架检测循环并拒绝; ④超时熔断——子 Agent 调用超时 (如 30s) 自动降级。

2. 资源估算: 单次 A2A 请求约消耗 50-100ms CPU + 2KB 内存 (Agent 实例+上下文)。100 QPS 时: 8 核 16GB 服务器可支撑 (每核处理 12-13 QPS), 但需考虑 LLM API 并发限制 (如 OpenAI 默认 500 RPM/key)。关键瓶颈不在 Agent 实例化, 而在 LLM API 调用并发——需要多个 API key 轮询或使用高并发模型服务。

3. MySQL→Redis 的影响: ①Redis 是纯内存存储, 重启数据丢失——需要配置 AOF 持久化; ②Redis 无 SQL 查询能力——无法按 sessionId+userId 复合查询历史; ③Redis 单 value 大小限制 (512MB) ——超长对话可能超限。Redis 更优场景: ①短会话 (<1 小时) ——内存速度远快于磁盘; ②高并发读取——Redis 单线程但 QPS 可达 10 万+; ③自动过期——TTL 天然适合会话超时清理。

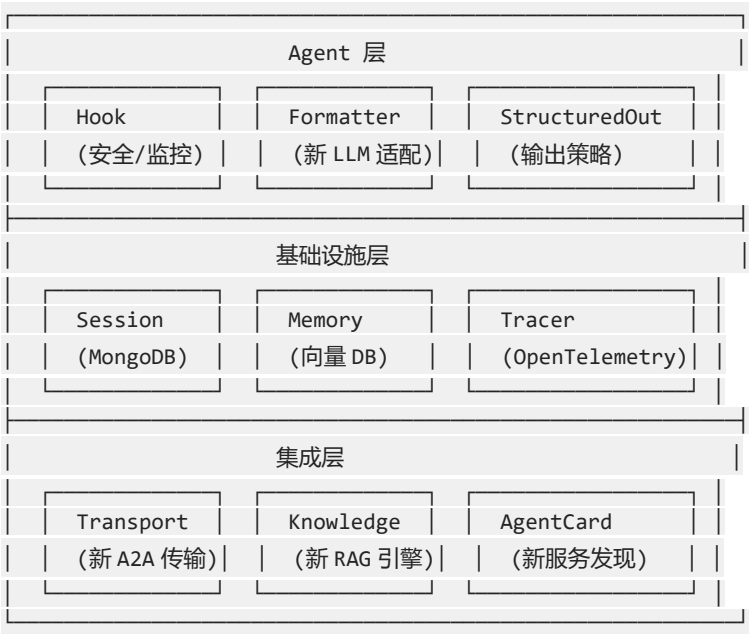
---

# 第 24 章 框架扩展开发指南

AgentScope-Java 采用"核心 + 扩展"架构——核心模块定义接口和默认实现，扩展模块通过 SPI 和实现接口来添加新能力。本章详细讲解如何开发自定义扩展。

## 24.1 扩展点全景图

AgentScope-Java 可扩展点：



## 24.2 自定义 Formatter——适配新 LLM Provider

### 24.2.1 理解 Formatter 体系

Formatter 接口 (`agentscope-core/.../formatter/Formatter.java:45-135`) 是

LLM 适配的核心:

```
public interface Formatter<TReq, TResp, TParams> {  
    List<TReq> format(List<Msg> msgs);          // 内部消息 → Provider 消息  
    ChatResponse parseResponse(TResp response, // Provider 响应 → 统一响应  
                               Instant startTime);  
    void applyOptions(TParams paramsBuilder,    // 应用生成选项  
                     GenerateOptions options, GenerateOptions defaultOptions);  
    void applyTools(TParams paramsBuilder,      // 应用工具描述  
                   List<ToolSchema> tools);  
    default void applyToolChoice(TParams builder, ToolChoice choice) {}  
}
```

三个泛型参数:

- **TReq**: Provider 的消息类型 (如 `OpenAIMessage`)
- **TResp**: Provider 的响应类型 (如 `OpenAIResponse`)
- **TParams**: Provider 的请求构建器类型 (如 `OpenAIRequest.Builder`)

### 24.2.2 适配步骤 (以 Grok 为例)

#### 第一步: 创建 DTO

```
// GrokMessage.java  
public record GrokMessage(  
    String role,          // "system" | "user" | "assistant" | "tool"  
    String content,  
    List<GrokToolCall> tool_calls,  
    String tool_call_id,  
    String name  
) {}  
  
// GrokResponse.java
```

```

public record GrokResponse(
    String id,
    List<GrokChoice> choices,
    GrokUsage usage
) {}

// GrokRequest.java
public record GrokRequest(
    String model,
    List<GrokMessage> messages,
    List<GrokTool> tools,
    Double temperature,
    Integer max_tokens
) {}

```

## 第二步：创建 BaseFormatter

```

public abstract class GrokBaseFormatter
    extends AbstractBaseFormatter<GrokMessage, GrokResponse,
    GrokRequest.Builder> {

    @Override
    public ChatResponse parseResponse(GrokResponse response, Instant
    startTime) {
        // 将 Grok 响应转换为统一的 ChatResponse
        GrokChoice choice = response.choices().get(0);
        return ChatResponse.builder()
            .text(choice.message().content())
            .toolCalls(convertToolCalls(choice.message().tool_calls()))
            .usage(convertUsage(response.usage()))
            .finishReason(choice.finish_reason())
            .startTime(startTime)
            .build();
    }
}

```

## 第三步：创建 ChatFormatter

```

public class GrokChatFormatter extends GrokBaseFormatter {

    @Override
    protected List<GrokMessage> doFormat(List<Msg> msgs) {
        List<GrokMessage> result = new ArrayList<>();
        for (Msg msg : msgs) {
            switch (msg.getRole()) {
                case SYSTEM -> result.add(new GrokMessage("system",
                    extractText(msg), null, null, null));
                case USER -> result.add(new GrokMessage("user",
                    extractText(msg), null, null, null));
                case ASSISTANT -> {
                    GrokMessage assistant = new GrokMessage("assistant",
                        extractText(msg),

```

```

        convertToGrokToolCalls(msg),
        null, null);
        result.add(assistant);
    }
    // ToolResult 处理...
}
}
return result;
}

@Override
public void applyOptions(GrokRequest.Builder builder,
    GenerateOptions options, GenerateOptions defaultOptions) {
    options.getTemperature().ifPresent(builder::temperature);
    options.getMaxTokens().ifPresent(builder::maxTokens);
}

@Override
public void applyTools(GrokRequest.Builder builder, List<ToolSchema>
tools) {
    builder.tools(convertToGrokTools(tools));
}
}

```

#### 第四步：创建 ChatModel

```

public class GrokChatModel extends ChatModelBase {
    private final WebClient webClient;
    private final GrokChatFormatter chatFormatter;
    private final GrokMultiAgentFormatter multiAgentFormatter;

    @Override
    protected Flux<ChatResponse> doStream0(List<Msg> msgs,
        GenerateOptions options, List<ToolSchema> tools) {
        GrokRequest.Builder builder = GrokRequest.builder()
            .model(modelName);

        chatFormatter.applyOptions(builder, options, defaultOptions);
        chatFormatter.applyTools(builder, tools);

        List<GrokMessage> formatted = chatFormatter.format(msgs);
        builder.messages(formatted);

        Instant startTime = Instant.now();

        return webClient.post()
            .uri("/v1/chat/completions")
            .bodyValue(builder.build())
            .retrieve()
            .bodyToFlux(GrokResponse.class)
            .map(resp -> chatFormatter.parseResponse(resp, startTime));
    }
}

```

```
    }  
}
```

### 第五步：创建 MultiAgentFormatter

多 Agent 场景需要处理角色映射（如 `agent_a`  $\rightarrow$  `assistant`）：

```
public class GrokMultiAgentFormatter extends GrokChatFormatter {  
    private final ConversationMerger merger = new ConversationMerger();  
  
    @Override  
    protected List<GrokMessage> doFormat(List<Msg> msgs) {  
        // 1. 合并连续同角色消息  
        List<Msg> merged = merger.merge(msgs);  
        // 2. 映射角色名  
        List<Msg> mapped = mapRoles(merged);  
        // 3. 调用父类格式化  
        return super.doFormat(mapped);  
    }  
  
    private List<Msg> mapRoles(List<Msg> msgs) {  
        // Grok 要求严格 USER/ASSISTANT 交替  
        // 非 USER 角色都映射为 ASSISTANT  
        return msgs.stream()  
            .map(msg -> msg.getRole() == Role.USER ? msg :  
                Msg.builder().role(Role.ASSISTANT)  
                    .content(msg.getContent()).build())  
            .toList();  
    }  
}
```

### 24.2.3 各 Provider 的格式差异

| Provider  | 角色约束                               | 多模态             | 工具调用                          | 思考块            |
|-----------|------------------------------------|-----------------|-------------------------------|----------------|
| OpenAI    | 宽松                                 | URL +<br>Base64 | 原生<br>parallel_tool_calls     | 无              |
| Anthropic | 严格<br><br>USER/ASSISTANT<br><br>交替 | Base64 only     | tool_use content<br><br>block | thinking block |

|           |                      |                 |              |                   |
|-----------|----------------------|-----------------|--------------|-------------------|
| Gemini    | 功能角色<br>(model/user) | Inline data     | functionCall | thought           |
| DashScope | 宽松                   | URL +<br>Base64 | 原生           | thinking (qwen3)  |
| Ollama    | OpenAI 兼容            | Base64          | OpenAI 兼容    | 无                 |
| DeepSeek  | OpenAI 兼容            | 不支持             | OpenAI 兼容    | reasoning_content |

**关键差异：**Anthropic 的严格角色交替是最难处理的。如果 Agent 两次连续调用工具（中间无 USER 消息），`AnthropicChatFormatter` 需要插入空 USER 消息来满足约束。

## 24.3 自定义 Tracer——OpenTelemetry 集成

### 24.3.1 Tracer 接口

```
public interface Tracer {
    default Mono<Msg> callAgent(AgentBase instance,
        List<Msg> inputMessages, Supplier<Mono<Msg>> agentCall) {
        return agentCall.get();
    }

    default Flux<ChatResponse> callModel(ChatModelBase instance,
        List<Msg> inputMessages, List<ToolSchema> toolSchemas,
        GenerateOptions options, Supplier<Flux<ChatResponse>> modelCall)
    {
        return modelCall.get();
    }

    default Mono<ToolResultBlock> callTool(Toolkit toolkit,
        ToolCallParam toolCallParam,
        Supplier<Mono<ToolResultBlock>> toolKitCall) {
        return toolKitCall.get();
    }

    default <TReq, TResp, TParams> List<TReq> callFormat(
        AbstractBaseFormatter<TReq, TResp, TParams> formatter,
        List<Msg> msgs, Supplier<List<TReq>> formatCall) {
```

```

        return formatCall.get();
    }

    default <T> T runWithContext(ContextView ctx, Supplier<T> inner) {
        return inner.get();
    }
}

```

## 24.3.2 实现自定义 Tracer

```

public class OpenTelemetryTracer implements Tracer {
    private final Tracer otelTracer; // io.opentelemetry.api.trace.Tracer

    @Override
    public Mono<Msg> callAgent(AgentBase instance,
        List<Msg> inputMessages, Supplier<Mono<Msg>> agentCall) {
        Span span = otelTracer.spanBuilder("invoke_agent")
            .setAttribute("agent.name", instance.getName())
            .setAttribute("agent.type", instance.getClass().getSimpleName())
            .startSpan();

        return Mono.deferContextual(ctx -> {
            // 从 Reactor Context 中获取 OTEL Context
            io.opentelemetry.context.Context otelCtx =
                ctx.getDefault("otel-context",
                    io.opentelemetry.context.Context.current());
            span.setParent(otelCtx);
            return agentCall.get();
        })
        .doOnSuccess(msg -> {
            span.setAttribute("agent.result.length",
                msg.getTextContent().length());
            span.end();
        })
        .doOnError(span::recordException)
        .contextWrite(Context.of("otel-context",
            io.opentelemetry.context.Context.current().with(span)));
    }

    @Override
    public Flux<ChatResponse> callModel(ChatModelBase instance,
        List<Msg> inputMessages, List<ToolSchema> toolSchemas,
        GenerateOptions options, Supplier<Flux<ChatResponse>> modelCall)
    {
        Span span = otelTracer.spanBuilder("chat")
            .setAttribute("model.name", instance.getModelName())
            .startSpan();

        return modelCall.get()
            .doOnNext(resp -> {
                resp.getUsage().ifPresent(usage -> {

```

```

        span.setAttribute("model.input_tokens",
usage.getInputTokens());
        span.setAttribute("model.output_tokens",
usage.getOutputTokens());
    });
    })
    .doOnComplete(span::end)
    .doOnError(span::recordException);
}

@Override
public Mono<ToolResultBlock> callTool(Toolkit toolkit,
    ToolCallParam toolCallParam,
    Supplier<Mono<ToolResultBlock>> toolKitCall) {
    Span span = otelTracer.spanBuilder("execute_tool")
        .setAttribute("tool.name", toolCallParam.getToolName())
        .startSpan();

    return toolKitCall.get()
        .doOnNext(result -> {
            span.setAttribute("tool.suspended", result.isSuspended());
            span.setAttribute("tool.error", result.isError());
        })
        .doOnComplete(span::end)
        .doOnError(span::recordException);
}
}

```

### 24.3.3 注册与启用

```

// 在 Spring Boot 配置中
@Bean
public Tracer openTelemetryTracer() {
    OpenTelemetry otel = OpenTelemetrySdk.builder()
        .setTracerProvider(
            SdkTracerProvider.builder()
                .addSpanProcessor(
                    BatchSpanProcessor.builder(
                        OtlpGrpcSpanExporter.builder()
                            .setEndpoint("http://jaeger:4317")
                            .build())
                    .build())
                .build())
        .build();

    return new OpenTelemetryTracer(otel.getTracer("agentscope"));
}

@PostConstruct
public void init() {
    TracerRegistry.register(openTelemetryTracer());
}

```

```

TracerRegistry.enableTracingHook();
}

```

## 24.4 自定义 Session——MongoDB 实现

### 24.4.1 Session 接口实现

```

public class MongoSession implements Session {
    private final MongoClient<Document> collection;

    public MongoSession(MongoDatabase database, String collectionName) {
        this.collection = database.getCollection(collectionName);
    }

    @Override
    public void save(SessionKey sessionKey, String key, State value) {
        validateSessionId(sessionKey.getSessionId());
        validateStateKey(key);

        Document doc = new Document()
            .append("session_id", sessionKey.getSessionId())
            .append("state_key", key)
            .append("state_data", serializeState(value))
            .append("updated_at", new Date());

        collection.updateOne(
            Filters.and(
                Filters.eq("session_id", sessionKey.getSessionId()),
                Filters.eq("state_key", key)),
            new Document("$set", doc),
            new UpdateOptions().upsert(true));
    }

    @Override
    public <T extends State> Optional<T> get(SessionKey sessionKey,
        String key, Class<T> type) {
        Document doc = collection.find(Filters.and(
            Filters.eq("session_id", sessionKey.getSessionId()),
            Filters.eq("state_key", key))).first();

        if (doc == null) return Optional.empty();
        return Optional.of(deserializeState(doc.getString("state_data"),
type));
    }

    @Override
    public void delete(SessionKey sessionKey) {
        collection.deleteMany(

```

```

        Filters.eq("session_id", sessionKey.getSessionId()));
    }

    // ... 其他方法实现
}

```

## 24.4.2 列表增量的实现

参考 `MysqlSession` 的 `ListHashUtil` 模式：

```

@Override
public void save(SessionKey sessionKey, String key, List<? extends State>
values) {
    String currentHash = ListHashUtil.computeHash(values);

    // 检查是否需要全量重写
    Document existing = collection.find(Filters.and(
        Filters.eq("session_id", sessionKey.getSessionId()),
        Filters.eq("state_key", key),
        Filters.eq("is_list", true))).first();

    if (existing != null) {
        String previousHash = existing.getString("list_hash");
        if (previousHash != null && previousHash.equals(currentHash)) {
            return; // 无变化, 跳过
        }
    }

    // 全量重写
    collection.deleteMany(Filters.and(
        Filters.eq("session_id", sessionKey.getSessionId()),
        Filters.eq("state_key", key)));

    for (int i = 0; i < values.size(); i++) {
        Document doc = new Document()
            .append("session_id", sessionKey.getSessionId())
            .append("state_key", key)
            .append("index", i)
            .append("state_data", serializeState(values.get(i)))
            .append("is_list", true)
            .append("list_hash", currentHash);
        collection.insertOne(doc);
    }
}

```

## 24.5 自定义 Memory——向量数据库 Memory

### 24.5.1 长期记忆接口

```
public interface LongTermMemory {  
    Mono<Void> record(List<Msg> msgs);    // 记录消息  
    Mono<String> retrieve(Msg msg);    // 检索相关记忆  
}
```

### 24.5.2 Milvus 长期记忆实现

```
public class MilvusLongTermMemory implements LongTermMemory {  
    private final MilvusClient client;  
    private final EmbeddingService embeddingService;  
    private final String collectionName;  
  
    @Override  
    public Mono<Void> record(List<Msg> msgs) {  
        return Mono.fromRunnable(() -> {  
            for (Msg msg : msgs) {  
                float[] embedding =  
embeddingService.embed(msg.getTextContent());  
                client.insert(InsertParam.newBuilder()  
                    .withCollectionName(collectionName)  
                    .withFields(List.of(  
                        new StringField("msg_id",  
UUID.randomUUID().toString()),  
                        new FloatVecField("embedding", embedding),  
                        new StringField("content", msg.getTextContent()),  
                        new StringField("role", msg.getRole().name()),  
                        new StringField("timestamp",  
Instant.now().toString())  
                    ))  
                    .build());  
            }  
        }).subscribeOn(Schedulers.boundedElastic()).then();  
    }  
  
    @Override  
    public Mono<String> retrieve(Msg msg) {  
        return Mono.fromCallable(() -> {  
            float[] queryEmbedding =  
embeddingService.embed(msg.getTextContent());  
            SearchResults results = client.search(SearchParam.newBuilder()  
                .withCollectionName(collectionName)  
                .withVectors(List.of(queryEmbedding))  
                .withTopK(5)
```

```

        .withExpr("timestamp > '" +
            Instant.now().minus(Duration.ofDays(30)).toString() +
            "'")
        .build());

    StringBuilder sb = new StringBuilder();
    sb.append("<long_term_memory>\n");
    for (SearchResult result : results.getResults().get(0)) {
        sb.append("-
    ").append(result.getEntity().get("content")).append("\n");
    }
    sb.append("</long_term_memory>");
    return sb.toString();
}).subscribeOn(Schedulers.boundedElastic());
}
}

```

### 24.5.3 使用方式

```

ReActAgent agent = ReActAgent.builder()
    .name("assistant")
    .model(model)
    .memory(new AutoContextMemory(config, model))
    .longTermMemory(new MilvusLongTermMemory(milvusClient, embeddingService))
    .longTermMemoryMode(LongTermMemoryMode.AGENT_CONTROL)
    .build();

```

## 24.6 自定义 Hook——安全审查、内容过滤、审计日志

### 24.6.1 安全审查 Hook

```

public class SecurityAuditHook implements Hook {
    private final AuditLogger auditLogger;
    private final ContentScanner scanner;

    @Override
    public Mono<HookResult> onEvent(HookEvent event) {
        return switch (event) {
            case PreCallEvent pre -> auditInput(pre);
            case PostReasoningEvent post -> auditReasoning(post);
            case PostActingEvent post -> auditToolExecution(post);
            default -> Mono.just(HookResult.continue());
        };
    }
}

```

```

    }

    private Mono<HookResult> auditInput(PreCallEvent event) {
        for (Msg msg : event.getInputMessages()) {
            ScanResult result = scanner.scan(msg.getTextContent());
            if (result.isBlocked()) {
                auditLogger.blocked("input", result.getReason());
                return Mono.just(HookResult.stopAgent(
                    "Input blocked: " + result.getReason()));
            }
        }
        return Mono.just(HookResult.continue());
    }

    private Mono<HookResult> auditReasoning(PostReasoningEvent event) {
        Msg reasoning = event.getReasoningMessage();

        // 检查工具调用是否安全
        for (ToolUseBlock toolCall : reasoning.getToolUseBlocks()) {
            if (isDangerousTool(toolCall.getName())) {
                auditLogger.alert("dangerous_tool", toolCall.getName());
                // 可以选择停止 Agent 或仅记录
                event.stopAgent("Dangerous tool blocked: " +
                    toolCall.getName());
            }
        }
        return Mono.just(HookResult.continue());
    }

    private Mono<HookResult> auditToolExecution(PostActingEvent event) {
        auditLogger.toolExecution(
            event.getToolName(),
            event.getArguments(),
            truncate(event.getResult(), 500),
            event.getDurationMs());
        return Mono.just(HookResult.continue());
    }

    @Override
    public int priority() { return 5; } // 非常高的优先级
}

```

## 24.6.2 内容过滤 Hook

```

public class PiiFilterHook implements Hook {
    private final Pattern emailPattern = Pattern.compile("\\b[\\w.-]+@[\\w.-]+\\.\\w+\\.\\b");
    private final Pattern phonePattern = Pattern.compile("\\b[3-9]\\d{9}\\b");
    private final Pattern idCardPattern =

```

```
Pattern.compile("\\b\\d{17}[\\dXx]\\b");
```

```
@Override
public Mono<HookResult> onEvent(HookEvent event) {
    if (event instanceof PreCallEvent pre) {
        List<Msg> filtered = pre.getInputMessages().stream()
            .map(this::filterMsg)
            .toList();
        pre.setInputMessages(filtered);
    }
    return Mono.just(HookResult.continue());
}

private Msg filterMsg(Msg msg) {
    String text = msg.getTextContent();
    text = emailPattern.matcher(text).replaceAll("[EMAIL_REDACTED]");
    text = phonePattern.matcher(text).replaceAll("[PHONE_REDACTED]");
    text = idCardPattern.matcher(text).replaceAll("[ID_REDACTED]");
    return Msg.builder().role(msg.getRole())
        .content(TextBlock.builder().text(text).build()).build();
}

@Override
public int priority() { return 60; } // 在 GenericRAGHook 之后执行
}
```

### 24.6.3 Token 消耗统计 Hook

```
public class TokenCountingHook implements Hook {
    private final AtomicLong totalInputTokens = new AtomicLong();
    private final AtomicLong totalOutputTokens = new AtomicLong();
    private final AtomicInteger totalIterations = new AtomicInteger();

    @Override
    public Mono<HookResult> onEvent(HookEvent event) {
        if (event instanceof PostReasoningEvent post) {
            post.getTokenUsage().ifPresent(usage -> {
                totalInputTokens.addAndGet(usage.getInputTokens());
                totalOutputTokens.addAndGet(usage.getOutputTokens());
            });
            totalIterations.incrementAndGet();
        }
        return Mono.just(HookResult.continue());
    }

    public TokenStats getStats() {
        return new TokenStats(
            totalInputTokens.get(),
            totalOutputTokens.get(),
            totalIterations.get());
    }
}
```

```

    public record TokenStats(long inputTokens, long outputTokens, int
iterations) {
        public double estimatedCost(double inputPrice, double outputPrice) {
            return inputTokens * inputPrice / 1_000_000
                + outputTokens * outputPrice / 1_000_000;
        }
    }
}

```

## 24.7 自定义 TransportWrapper——新增 A2A 传输层

A2A 协议支持多种传输方式，通过 SPI 机制注册。

### 24.7.1 SPI 注册机制

`TransportWrapperBuilder` 是 SPI 接口 (`agentscope-extensions-a2a-server/.../TransportWrapperBuilder.java`) :

```

public interface TransportWrapperBuilder {
    String getTransportType();
    TransportWrapper<?, ?> build(AgentCard agentCard,
        RequestHandler requestHandler, Executor executor);
}

```

注册方式：在 `META-`

`INF/services/io.agentscope.core.a2a.server.transport.TransportWrapperBuilder` 文件中写入实现类全名。

### 24.7.2 实现 Kafka 传输层

```

public class KafkaTransportWrapperBuilder implements TransportWrapperBuilder
{
    @Override
    public String getTransportType() {
        return "KAFKA";
    }
}

```

```

    @Override
    public TransportWrapper<?, ?> build(AgentCard agentCard,
        RequestHandler requestHandler, Executor executor) {
        return new KafkaTransportWrapper(agentCard, requestHandler,
            executor);
    }
}

public class KafkaTransportWrapper
    implements TransportWrapper<ConsumerRecord<String, String>,
        CompletableFuture<Void>> {

    private final KafkaConsumer<String, String> consumer;
    private final KafkaProducer<String, String> producer;
    private final JsonRpcTransportWrapper jsonRpcWrapper; // 复用 JSON-RPC

```

处理

```

    @Override
    public String getTransportType() { return "KAFKA"; }

    @Override
    public CompletableFuture<Void> handleRequest(ConsumerRecord<String,
        String> record,
        Map<String, String> headers, Map<String, Object> metadata) {
        // 1. 从 Kafka 消息中提取 JSON-RPC 请求
        String requestBody = record.value();

        // 2. 委托给 JsonRpcTransportWrapper 处理
        Object response = jsonRpcWrapper.handleRequest(requestBody, headers,
            metadata);

        // 3. 将响应发送到 Kafka 回复 Topic
        if (response instanceof Flux<?> flux) {
            // 流式响应: 逐个发送
            flux.subscribe(event -> producer.send(
                new ProducerRecord<>(record.key() + "-reply",
                    toJson(event))));
        } else {
            // 非流式响应: 直接发送
            producer.send(new ProducerRecord<>(record.key() + "-reply",
                toJson(response)));
        }

        return CompletableFuture.completedFuture(null);
    }
}

```

# 24.8 发布扩展模块的最佳实践

## 24.8.1 模块结构

```
agentscope-extensions/  
└─ agentscope-extensions-milvus/  
    └─ pom.xml # 依赖 agentscope-core  
        └─ src/main/  
            └─ java/io/agentscope/core/memory/milvus/  
                └─ MilvusLongTermMemory.java # 核心实现  
                └─ MilvusMemoryConfig.java # 配置类  
                └─ MilvusMemoryAutoConfiguration.java # Spring Boot 自动配置  
            └─ resources/  
                └─ META-INF/  
                    └─ spring/  
                        └─  
                            org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

## 24.8.2 命名规范

| 组件   | 命名模式                                                   | 示例                                 |
|------|--------------------------------------------------------|------------------------------------|
| 模块   | `agentscope-extensions-{name}`                         | `agentscope-extensions-milvus`     |
| 包    | `io.agentscope.core.{type}.{name}`                     | `io.agentscope.core.memory.milvus` |
| 实现类  | `{Product}{Interface}Impl` 或<br>`{Product}{Interface}` | `MilvusLongTermMemory`             |
| 配置   | `{Product}{Type}Config`                                | `MilvusMemoryConfig`               |
| 自动配置 | `{Product}{Type}AutoConfiguration`                     | `MilvusMemoryAutoConfiguration`    |

## 24.8.3 依赖管理

```
<parent>  
  <groupId>io.agentscope</groupId>  
  <artifactId>agentscope-extensions</artifactId>  
  <version>${agentscope.version}</version>  
</parent>
```

```

<dependencies>
  <!-- 只依赖 core, 不依赖其他扩展 -->
  <dependency>
    <groupId>io.agentscope</groupId>
    <artifactId>agentscope-core</artifactId>
    <version>${agentscope.version}</version>
    <scope>provided</scope>
  </dependency>

  <!-- 第三方依赖 -->
  <dependency>
    <groupId>io.milvus</groupId>
    <artifactId>milvus-sdk-java</artifactId>
  </dependency>
</dependencies>

```

## 24.8.4 测试规范

```

class MilvusLongTermMemoryTest {
  private MilvusLongTermMemory memory;

  @BeforeEach
  void setUp() {
    // 使用 Testcontainers 启动 Milvus
    MilvusContainer container = new
MilvusContainer("milvusdb/milvus:v2.3.0");
    container.start();
    memory = new MilvusLongTermMemory(
      createClient(container), embeddingService, "test_collection");
  }

  @Test
  void shouldRecordAndRetrieve() {
    List<Msg> msgs = List.of(Msg.userText("我叫张三"));
    memory.record(msgs).block();

    String result = memory.retrieve(Msg.userText("我叫什么? ")).block();
    assertThat(result).contains("张三");
  }
}

```

---

# 24.9 扩展开发检查清单

发布扩展模块前的自检清单：

| 检查项              | 说明                                                              |
|------------------|-----------------------------------------------------------------|
| 接口实现完整性          | Session 的所有方法都有实现，不抛 <code>UnsupportedOperationException</code> |
| 空值与边界处理          | <code>null</code> 输入、空列表、超长字符串等边界情况                             |
| 线程安全             | <code>ConcurrentHashMap</code> 或同步块保护共享状态                       |
| 资源清理             | <code>`close()`</code> 方法释放连接、线程池等资源                            |
| 错误传播             | 使用 <code>Mono.error()</code> 而非抛异常（响应式契约）                       |
| 输入验证             | 防注入、防路径遍历、防超限                                                   |
| 日志规范             | 使用 SLF4J，关键操作有 <code>DEBUG/WARN</code> 日志                       |
| Spring Boot 自动配置 | <code>@ConditionalOnClass</code> + 配置属性                         |
| 测试覆盖             | 核心路径 80%+ 覆盖率                                                   |
| 文档               | README + 使用示例 + 配置说明                                            |

## 思考题

1. 自定义 `Formatter` 时，最常遇到的 `Provider` 格式差异是什么？如何系统性地处理这些差异？
2. 设计一个 `MongoDB Session` 实现——需要考虑哪些 `MongoDB` 特有的优化（如批量写入、`TTL` 索引）？

### 3. 自定义 Hook 的安全审计模式（如内容过滤、PII 脱敏）与 Formatter 层的处理有什么区别？应该在哪一层实现？

**\*\*参考：\*\***

1. 最常遇到的差异：①思考输出格式——Anthropic 用 *thinking block*, OpenAI 用 *reasoning\_content*, Google 无原生支持；②工具调用 ID——有的用 *string*, 有的用 *integer*；③多模态内容——图片/文件的传递方式各异 (*URL vs Base64 vs File ID*)。系统性处理：建立 *Provider* 差异矩阵 (7 家 × 10 维度)，通用 *Formatter* 处理共性，差异点通过 *ProviderProfile* 配置驱动。

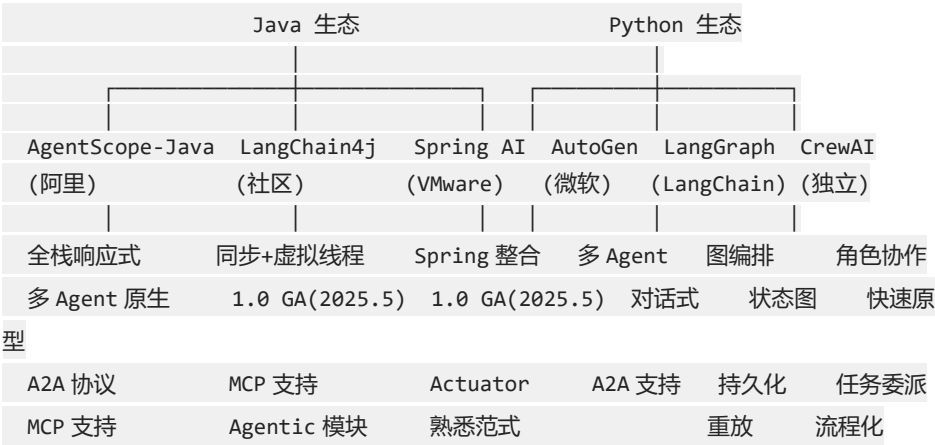
2. MongoDB Session 优化：①批量写入——*bulkWrite()* 替代单条 *save()*, 减少网络往返；②TTL 索引——*db.sessions.createIndex({expireAt:1}, {expireAfterSeconds:0})* 自动过期清理；③Change Streams——监听 *session* 变更实现实时同步；④分片——按 *sessionId* 哈希分片，水平扩展；⑤文档模型——嵌套式存储（消息内嵌 *session* 文档），减少 *join* 查询。

3. 区别：*Formatter* 层处理的是协议格式（将消息转换为 LLM 能理解的格式），关注数据转换；*Hook* 层处理的是业务语义（内容是否合规、是否脱敏），关注业务规则。PII 脱敏应在 *Hook* 层实现——①脱敏是业务策略，与 LLM 格式无关；②*Formatter* 不应修改内容语义，只管格式转换；③*Hook* 可插拔，不同部署环境用不同脱敏策略。

---

# 第 25 章 框架对比与选型指南

## 25.1 Agent 框架全景图（2025-2026）



## 25.2 AgentScope-Java vs LangChain4j

### 25.2.1 架构差异

| 维度       | AgentScope-Java                                       | LangChain4j                |
|----------|-------------------------------------------------------|----------------------------|
| 编程范式     | 全栈响应式 (Reactor Mono/Flux)                             | 同步阻塞 + 虚拟线程                |
| Agent 模型 | ReActAgent (显式推理+行动循环)                                | AiService (声明式接口)          |
| 多 Agent  | 原生支持<br>(Supervisor/Handoffs/Routing/Workflow/MsgHub) | Agentic 模块<br>(2025 新增, 仍在 |

|           |                                         |                             |
|-----------|-----------------------------------------|-----------------------------|
|           |                                         | 发展)                         |
| 工具系统      | @Tool + MCP + SubAgentTool + ToolGroup  | @Tool + MCP                 |
| 记忆系统      | 短期 + AutoContextMemory + 长期 (Mem0/ReMe) | 短期 +<br>ChatMemory +<br>RAG |
| 流式        | 原生 Flux 流 (token → event → SSE)         | TokenStream 适配<br>器         |
| A2A 协议    | 完整实现 (JSON-RPC + RocketMQ)              | 不支持                         |
| 状态持久化     | 4 种 Session (Memory/FS/Redis/MySQL)     | 无内置方案                       |
| 优雅关闭      | GracefulShutdownManager                 | 无                           |
| 规划系统      | PlanNotebook                            | 无                           |
| 技能系统      | SkillBox (动态加载)                         | 无                           |
| Spring 集成 | 自动配置 + Nacos + Sentinel                 | 自动配置<br>(Quarkus 也支持)       |

## 25.2.2 代码风格对比

**LangChain4j** 风格——声明式接口：

```
@AiService
interface Assistant {
    String chat(String message);

    @SystemMessage("You are a weather assistant")
    TokenStream chatStream(@UserMessage String message);
}
```

**AgentScope-Java** 风格——命令式构建：

```
ReActAgent agent = ReActAgent.builder()
    .name("assistant").model(model)
    .toolkit(toolkit).memory(memory)
    .build();
Msg response = agent.call(Msg.userText("hello")).block();
```

LangChain4j 的声明式风格更简洁，AgentScope-Java 的构建器风格更灵活。

前者适合简单场景，后者适合需要精细控制的复杂场景。

25.2.3 性能对比

| 场景         | AgentScope-Java   | LangChain4j           |
|------------|-------------------|-----------------------|
| 单次 LLM 调用  | Reactor 开销略高      | 同步直接，略快               |
| 100 并发请求   | ~10 线程            | ~100 虚拟线程 (Java 21+)  |
| 并行工具调用     | Flux.merge() 原生并发 | 需手动 CompletableFuture |
| 流式推送       | 全链路零拷贝            | TokenStream 适配        |
| 多 Agent 协作 | 原生优化              | 需自行编排                 |

**结论：**低并发简单场景下 LangChain4j 可能略快；高并发复杂场景下

AgentScope-Java 的响应式架构优势明显。

25.3 AgentScope-Java vs AutoGen (Python)

| 维度         | AgentScope-Java                             | AutoGen                        |
|------------|---------------------------------------------|--------------------------------|
| 语言         | Java 17+                                    | Python 3.10+                   |
| 多 Agent 模式 | Supervisor/Handoffs/Routing/Workflow/MsgHub | 对话式 (AgentChat) + 底层运行时 (Core) |
| A2A 协议     | 完整实现                                        | 官方参考实现 (同源)                    |

|      |                                   |                                       |
|------|-----------------------------------|---------------------------------------|
| 状态管理 | Session 持久化 + 状态恢复                | 检查点 (Checkpoint)                      |
| 企业特性 | 优雅关闭 + 安全沙箱 + 审计                  | 基础支持                                  |
| 生态   | Java 企业生态 (Spring/Nacos/Sentinel) | Python ML 生态<br>(HuggingFace/PyTorch) |
| 性能   | JVM 优化 + 响应式                      | GIL 限制 + asyncio                      |
| 部署   | Docker + K8s (JVM 成熟方案)           | Docker + K8s (需处理<br>Python 环境)       |

**关键差异：**AutoGen 的多 Agent 协作以"对话"为核心——Agent 之间通过消息对话协作。AgentScope-Java 则提供了更多元的协作模式（Supervisor 委派、Handoffs 转移、Workflow 编排等），适合更结构化的业务场景。

## 25.4 AgentScope-Java vs CrewAI (Python)

| 维度   | AgentScope-Java                         | CrewAI                             |
|------|-----------------------------------------|------------------------------------|
| 协作模型 | 多种模式<br>(Supervisor/Handoffs/MsgHub...) | 角色驱动 (Role → Goal → Backstory)     |
| 任务编排 | StateGraph + Pipeline                   | Sequential → Hierarchical → Custom |
| 工具   | @Tool + MCP + SubAgentTool              | @Tool + MCP                        |
| 记忆   | 短期 + AutoContext + 长期<br>(Mem0/ReMe)    | 短期 + 长期 + 实体记忆                     |
| 生产部署 | 企业级 (Session + Shutdown + K8s)          | CrewAI+ (托管平台)                     |

|      |    |                  |
|------|----|------------------|
| 上手速度 | 中等 | 快（10 分钟可跑通 Demo） |
|------|----|------------------|

CrewAI 的核心抽象是"团队"——定义角色、分配任务、设定流程。这更接近人类团队协作的直觉。AgentScope-Java 则更偏向工程化的编排控制。

## 25.5 AgentScope-Java vs Semantic Kernel (.NET/Java)

| 维度      | AgentScope-Java         | Semantic Kernel                          |
|---------|-------------------------|------------------------------------------|
| 语言      | Java                    | C#（主）+ Java（次）                           |
| 核心抽象    | Agent + Pipeline + Hook | Kernel + Plugin + Planner                |
| 规划      | PlanNotebook            | Handlebars Planner /<br>Stepwise Planner |
| 记忆      | Memory 体系               | Volatile/Text/Vector<br>Memory           |
| 企业集成    | Spring + Nacos          | Azure + Microsoft 365                    |
| 多 Agent | 原生多种模式                  | AgentGroupChat（基础）                       |
| MCP     | 支持                      | 支持                                       |
| A2A     | 完整实现                    | 不支持（微软自有协议）                              |

**注：**2026 年初，微软已将 AutoGen 和 Semantic Kernel 合并为统一的 **Microsoft Agent Framework**（RC 阶段），新特性将优先在统一框架中开发。

# 25.6 选型决策矩阵

```
graph TD
    Java{团队使用 Java?}
    Java -->|是| MultiAgent{需要多 Agent 协作?}
    Java -->|否| Python| FastProto{需要快速原型?}
    MultiAgent -->|是| ASJ[AgentScope-Java]
    MultiAgent -->|否| LC4J[LangChain4j]
    FastProto -->|是| CrewAI[CrewAI]
    FastProto -->|否| LangGraph[LangGraph]
```

| 场景                | 推荐框架                        | 理由                                |
|-------------------|-----------------------------|-----------------------------------|
| 企业级 Java Agent 系统 | AgentScope-Java             | 多 Agent 原生、A2A、优雅<br>关闭、生产部署      |
| 简单 Java AI 集成     | LangChain4j                 | 轻量、声明式、学习曲线低                      |
| Spring Boot + AI  | Spring AI 或 AgentScope-Java | 前者 Spring 深度集成，后者<br>多 Agent 能力更强 |
| 多 Agent 研究/实验     | AutoGen                     | 对话式协作、Python 生态                   |
| 快速多 Agent 原型      | CrewAI                      | 10 分钟出 Demo、角色驱动                  |
| 复杂工作流编排           | LangGraph                   | 图编排、状态持久化、重放                      |
| 企业级 .NET 系统       | Semantic Kernel / MAF       | 微软生态深度集成                          |
| 数据驱动的知识系统         | LlamaIndex                  | RAG 能力最强                          |

# 25.7 AgentScope-Java 的独特价值定位

在众多框架中，AgentScope-Java 的独特价值在于：

1. **Java 企业级的唯一全栈多 Agent 方案**：LangChain4j 和 Spring AI 在多 Agent 方面仍在追赶
2. **A2A 协议的完整实现**：其他 Java 框架均不支持 Agent-to-Agent 互操作
3. **生产级运维能力**：Graceful Shutdown、状态持久化、全链路追踪——这些“无聊但关键”的能力
4. **渐进式复杂度**：从 5 行代码的最简 Agent 到分布式多 Agent 系统，同一套 API

**局限：**

1. 社区规模远小于 LangChain4j 和 Spring AI
2. 文档和教程相对不足
3. 响应式编程门槛较高
4. 仅 Java 生态，无 Python 桥接

---

|                        |                             |
|------------------------|-----------------------------|
| 第 1 章：Agent 范式演进       | ← 理论基础：为什么需要 Agent          |
| ↓                      |                             |
| 第 2 章：LLM 工作原理         | ← 工程约束：LLM 能做什么、不能做什么       |
| ↓                      |                             |
| 第 3 章：响应式编程            | ← 技术选型：为什么用 Reactor         |
| ↓                      |                             |
| [第 4-11 章：核心架构]        | ← 实现细节：AgentScope-Java 如何工作 |
| ↓                      |                             |
| [第 12-15 章：多 Agent 协作] | ← 协作模式：多个 Agent 如何配合        |
| ↓                      |                             |
| [第 16-20 章：生产工程]       | ← 运维保障：如何上生产                |
| ↓                      |                             |
| [第 21-23 章：实战案例]       | ← 动手实践：从零到生产                |
| ↓                      |                             |
| 第 25 章：框架对比            | ← 选型参考：何时用 AgentScope-Java  |

理论 → 约束 → 技术选型 → 实现 → 协作 → 运维 → 实战 → 选型，形成完整的知识闭环。

---

## 思考题

1. 如果你的团队使用 Java 且需要构建多 Agent 系统，你会选择 AgentScope-Java 还是 LangChain4j？给出至少 3 个理由。
2. AgentScope-Java 的全栈响应式架构在什么场景下是劣势？在什么场景下是优势？
3. 如果要将 AgentScope-Java 与 Python 生态的 AutoGen 通过 A2A 协议互联，需要解决哪些技术挑战？

**\*\*参考：\*\***

1. 选 AgentScope-Java 的理由：①原生 A2A 协议支持——跨 Agent 互联开箱即用，LangChain4j 需自建；②全栈响应式——Reactor 天然适配流式 LLM 场景，LangChain4j 的流式支持是后加的；③企业级特性——GracefulShutdown/Session 持久化/ToolGroup 安全，LangChain4j 需大量自研。选 LangChain4j 的理由（反面参考）：生态更丰富/社区更大/与 Spring AI 兼容。
2. 劣势场景：①团队无 Reactor 经验——学习曲线陡峭，响应式调试困难；②简单脚本场景——单 Agent+少量工具，响应式带来的复杂度不值得；③与同步框架混用——Reactor 阻塞调用需调度器切换，引入额外开销。优势场景：①高并发在线服务——同时服务数千用户对话；②流式交互——SSE/WebSocket 实时推送；③多 Agent 协作——FanoutPipeline 并行编排。
3. 技术挑战：①协议适配——Python A2A 实现与 Java 的 JSON-RPC 序列化

差异（如日期/枚举处理）；②类型系统——Python 动态类型 vs Java 强类型，AgentCard 的 skills 字段映射需要中间层；③传输层——Java 默认 JSON-RPC over HTTP，Python 可能用 WebSocket，需统一传输协议；④异步模型——Python asyncio vs Java Reactor，桥接需要适配器；⑤部署——Python Agent 和 Java Agent 混合编排的运维复杂度。

---

## 后记

本书基于 AgentScope-Java v1.0.11 源码的逐行分析，涵盖了从理论基础到生产部署的完整知识体系。

AgentScope-Java 的核心设计决策——全栈响应式、ReAct 范式、A2A 协议、渐进压缩记忆——在 Java Agent 框架生态中独树一帜。它不是 LangChain4j 的 Java 移植，而是一个从企业级生产需求出发的原生设计。

框架仍在快速演进中。随着 A2A 协议标准化、MCP 生态扩展、更多 LLM Provider 适配，AgentScope-Java 有望成为 Java 企业级 Agent 开发的事实标准。

本书的所有源码引用均基于 v1.0.11 版本，如遇版本更新导致行号偏移，请对照类名和方法名定位。

---

# 参考文献

1. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). **ReAct: Synergizing Reasoning and Acting in Language Models**. arXiv:2210.03629.
2. Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). **Lost in the Middle: How Language Models Use Long Contexts**. arXiv:2307.03172.
3. Google (2024). **Agent-to-Agent (A2A) Protocol Specification**.  
<https://github.com/google/A2A>
4. Anthropic (2024). **Model Context Protocol (MCP) Specification**.  
<https://modelcontextprotocol.io>
5. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). **Reflexion: Language Agents with Verbal Reinforcement Learning**. NeurIPS 2023.
6. Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K. W., & Lim, E. P. (2023). **Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models**. ACL 2023.
7. Project Reactor (2024). **Reactor Core Reference Documentation**.  
<https://projectreactor.io/docs/core/release/reference/>

8. Alibaba (2024). **AgentScope-Java GitHub Repository**.  
<https://github.com/agentscope/agentscope-java>

9. Hewitt, C., Bishop, P., & Steiger, R. (1973). **A Universal Modular ACTOR Formalism for Artificial Intelligence**. IJCAI 1973.

10. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). **Attention Is All You Need**. NeurIPS 2017.

## 术语表

| 术语       | 英文                     | 释义                                   |
|----------|------------------------|--------------------------------------|
| Agent    | Agent / 智能体            | 能够感知环境、做出决策并执行行动的自主实体                |
| ReAct    | Reasoning + Acting     | 交替进行推理（Thought）和行动（Action）的 Agent 范式 |
| LLM      | Large Language Model   | 大语言模型，如 GPT-4、Claude、Qwen 等          |
| Pipeline | Pipeline / 管道          | 多个 Agent 按顺序或并行编排的执行模式               |
| MCP      | Model Context Protocol | 模型上下文协议，标准化工具                        |

|              |                                |                                           |
|--------------|--------------------------------|-------------------------------------------|
|              |                                | 集成协议                                      |
| A2A          | Agent-to-Agent Protocol        | Agent 间通信协议，支持跨进程/跨网络 Agent 互操作           |
| HITL         | Human-in-the-Loop              | 人机协作模式，关键决策点需人工审批                         |
| RAG          | Retrieval-Augmented Generation | 检索增强生成，先检索知识库再生成回答                        |
| KV Cache     | Key-Value Cache                | Transformer 推理中缓存已计算的 Key/Value 矩阵，避免重复计算 |
| Formatter    | Formatter / 格式化器               | 将统一消息模型转换为特定 LLM API 格式的适配层               |
| ToolGroup    | ToolGroup / 工具组                | 工具的权限分组，控制 Agent 可见的工具子集                  |
| SkillBox     | SkillBox / 技能盒                 | 动态技能加载系统，按需激活/停用工具组                       |
| PlanNotebook | PlanNotebook / 规划笔记本           | 结构化规划系统，支持任务分解、状态追踪和场景提示                  |
| MsgHub       | MsgHub / 消息中心                  | 群组广播通信组件，支持多 Agent 对等讨论                   |
| Session      | Session / 会话                   | Agent 状态持久化的抽象，支持跨调用恢复                    |
| Tracer       | Tracer / 追踪器                   | 全链路追踪组件，记录 Agent 执行的每个阶段                  |

|                    |                      |                                       |
|--------------------|----------------------|---------------------------------------|
| AutoContextMemory  | AutoContextMemory    | 6 级渐进压缩记忆，自动管理上下文窗口                   |
| Mono               | Mono                 | Reactor 中 0 或 1 个元素的异步容器              |
| Flux               | Flux                 | Reactor 中 0 到 N 个元素的异步流               |
| 背压                 | Backpressure         | 消费者向生产者反馈处理速率的机制，防止数据积压               |
| 优雅关闭               | Graceful Shutdown    | 优雅关闭，在途请求完成后再停止服务                     |
| ContentBlock       | ContentBlock / 内容块   | 消息中多模态内容的统一抽象 (sealed interface)      |
| ThinkingBlock      | ThinkingBlock / 思考块  | 记录 LLM 推理过程的内容块，API 调用时过滤但 Memory 中保留 |
| AgentCard          | AgentCard / Agent 名片 | A2A 协议中 Agent 的身份和能力描述文档              |
| ConversationMerger | ConversationMerger   | 多 Agent 对话消息合并器，满足不同 API 的格式约束        |
| Sealed Interface   | Sealed Interface     | Java 17+ 特性，限制接口只能被同包/同模块的类实现，支持编译期穷举 |
| FanoutPipeline     | FanoutPipeline       | 扇出管道，将同一输入并行发送给多个 Agent               |

|                      |                            |                                      |
|----------------------|----------------------------|--------------------------------------|
| SequentialPipeline   | SequentialPipeline         | 顺序管道，Agent 按序依次执行                    |
| SubAgentTool         | SubAgentTool               | 将子 Agent 封装为工具的适配器                   |
| ToolSuspendException | ToolSuspendException       | 工具挂起异常，用于 HITL 场景中中断 Agent 执行流       |
| SPI                  | Service Provider Interface | Java 服务提供者接口，实现可插拔扩展                 |
| Embedding            | Embedding / 向量化            | 将文本映射为高维向量，用于语义检索                    |
| Token                | Token / 词元                 | LLM 处理文本的最小单位，约 0.75 个英文单词或 0.5 个中文字 |
| SSE                  | Server-Sent Events         | 服务器推送事件，HTTP 流式传输协议                  |
| StateGraph           | StateGraph / 状态图           | 有限状态机，用于 Handoffs 模式中定义 Agent 间的转移关系 |
| Nacos                | Nacos                      | 阿里巴巴开源的服务注册与发现、配置管理平台                |
| Spring Boot Starter  | Spring Boot Starter        | Spring Boot 自动配置模块，引入即生效             |

## 作者简介

### 陈 瑶

拥有超过二十年 IT 信息化与数字化转型经验，  
曾主导大中型银行核心系统、电商平台、  
教育信息化平台及新一代智能体架构设计。

在技术管理领域深耕多年，  
擅长从 0 到 1 构建技术团队，  
从 1 到 N 推动规模化增长。

近年来专注于 LLM Agent 技术的工程化落地，  
是 AgentScope-Java 框架的深度实践者与布道者。

## 本书特色

- 基于 AgentScope-Java v1.0.11 源码逐行分析
- 25 章覆盖理论基础→核心架构→生产部署全链路
- 包含精确源码引用（文件路径+行号），论述有据可查
- 13 幅 Mermaid 架构图，直观展示系统设计
- 25 组思考题与参考答案，适合自学与教学
- 涵盖 A2A 协议、MCP 集成、Graceful Shutdown 等独家内容
- 框架对比与选型指南（LangChain4j/AutoGen/Spring AI）